2024

Architectural and Training Regime Modifications to Transformer Models for low data Applications

Azwad Tamir
University of Central Florida

ARCHITECTURAL AND TRAINING REGIME MODIFICATIONS TO TRANSFORMER MODELS FOR LOW DATA APPLICATIONS

by

AZWAD TAMIR
B.S. in Electrical and Electronics Engineering, Bangladesh University of Engineering and Technology, 2017
M.S. in Electrical Engineering, University of Central Florida, 2023

A dissertation submitted in partial fulfilment of the requirements
for the degree of Doctor of Philosophy
in the Department of Electrical and Computer Engineering
in the College of Engineering and Computer Sciences
at the University of Central Florida
Orlando, Florida

Fall Term
2024

Major Professor: Jiann-Shiun Yuan

ABSTRACT

A significant catalyst of the recent Artificial Intelligence and Machine Learning revolution is attributed to large language models (LLMs), which include many prominent AI systems such as ChatGPT, Bard, and Gemini. The majority of high-performance LLM models are based on the transformer architecture and the primary factor behind the recent enhancement in the performance of these models is attributed to the increase of layer quantity and dimensionality, resulting in an exponential rise in the total number of learnable parameters, necessitating substantial data for effective training. This may pose a significant barrier in certain application domains, such as bioinformatics, pharmaceutical research, and medicinal applications, where data is constrained by its sensitive nature, confidentiality restrictions, and personal privacy laws. The purpose of this study is to design architectural level and training loop modifications to the transformer architecture to make it more suitable for low-data applications. Our proposed modifications include selective transfer learning, cross transfer learning, hierarchical system design, convolutional transformers, knowledge graph fusion models, etc. To test the efficacy of the proposed techniques, we have applied the ideas to various applications, including Enzyme Commission (EC) number prediction from full-scale protein sequences. Here, we have linked four transformer ProtBert modules in a hierarchical manner and selectively pretrained them to obtain an accuracy increase from 89.93% to 97.88% and an F1 score boost from 0.9323 to 0.9787 compared to conventional transformer architectures. We have also designed a multimodal, knowledge graph integrated transformer model that could predict fetal drug-drug interaction events from drug SMILES and achieve accuracy increases of up to 6.78% on inductive test settings. Lastly, we have developed a parallel transformer modular architecture for predicting Gene Ontology (GO) terms from full-scale protein sequences and obtain better results with less training data and computational resources compared to other conventional transformer models.

Dedicated to my beloved parents.

ACKNOWLEDGMENTS

I extend my sincerest gratitude to Dr. Jiann-Shiun Yuan, my advisor, whose unwavering support, expertise, kindness, and guidance have been invaluable throughout my Ph.D. journey. Dr. Yuan provided me with the opportunity to showcase my research capabilities and displayed immense patience in nurturing my progress. Additionally, I express my deep appreciation to my dissertation committee members for their excellent suggestions and valuable time.

I am profoundly thankful to my family and friends, whose unwavering encouragement and support made this work achievable. Particularly, I am forever indebted to my parents for their immense sacrifices and steadfast dedication to my aspirations. Their consistent support and motivation propelled me forward in my academic pursuits, consistently reminding me of the broader purpose and ensuring I stayed on course.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1: INTRODUCTION

Large Language Models or LLMs in short has been one of the major forms of artificial Intelligence or Machine Learning models in modern times. They have been shown to achieve high accuracy and performance and consists many state of the art models like ChatGPT, Bard, Gemini etc. that are driving the AI revolution. At the core of most LLM models is the transformer architecture which is an encoder-decoder framework based on the attention mechanism capable of understanding both long term and short-term patterns in the input data.

The transformer architecture initially emerged in Natural Language Processing (NLP) and has since diversified into various domains such as image processing, audio and video classification, big data analysis, biomedical sequence research, and more. It has become a dominant model across a wide array of applications.

Recent advancements in transformer models, particularly in their encoder-decoder architecture, have been driven by increases in layer count and dimensionality. This exponential growth in model parameters necessitates substantial training data for effective learning. While NLP benefits from ample publicly available datasets and easily obtainable data from sources like social media and news, fields such as bioinformatics and biomedical research face significant challenges due to limited data availability, confidentiality constraints, and privacy laws.

The exponential growth of Machine Learning models in recent times is illustrated in Figure 1.1. It shows the remarkable increase in the number of parameters of the models going from around 100 million parameters for GPT-1 to 1.76 trillion parameters for GPT-4 in a few short years [1]. Here, learnable parameters refer to the weights of the machine learning model which are being updated by backpropagation in each training iteration.

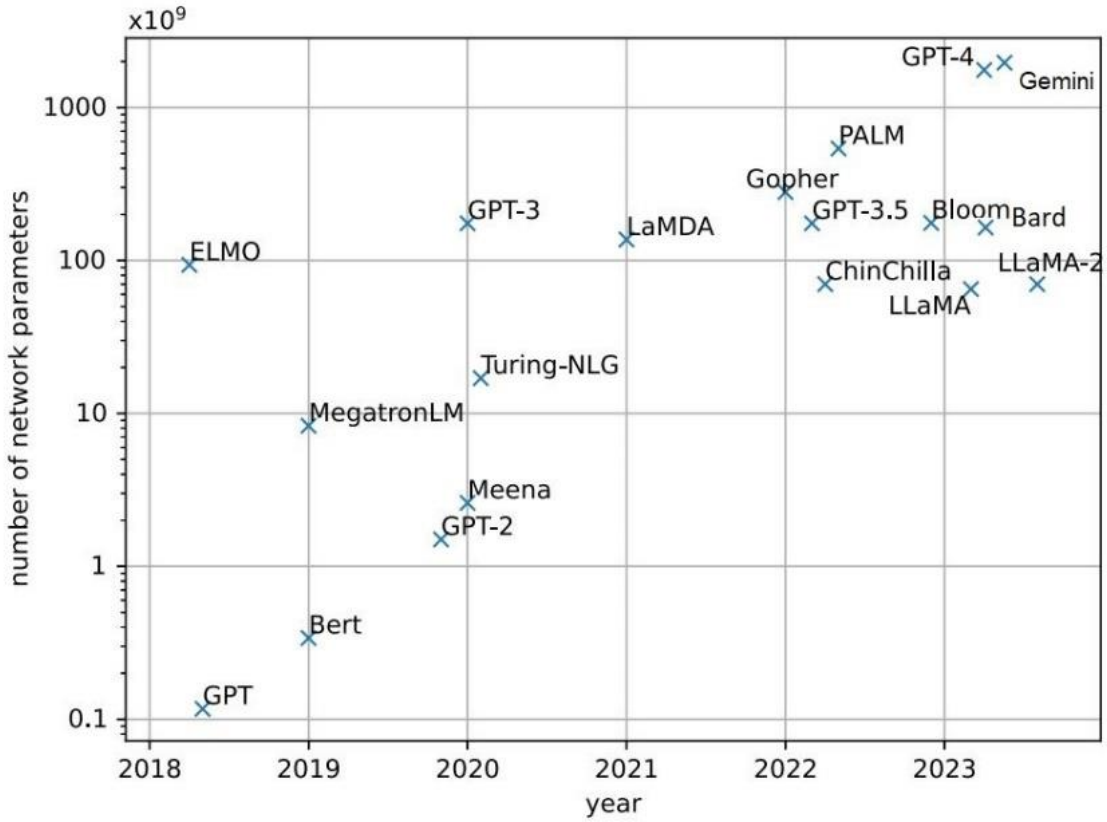

Figure 1.1: Exponential Growth of ML models in recent years. Figure adapted from [1]

This study aims to enhance the transformer architecture for low-data applications through architectural modifications and improvements in the training processes. The proposed techniques include selective and cross transfer learning, hierarchical system design, convolutional transformers, and knowledge graph fusion models. Experimental results demonstrate stable training outcomes with limited data, achieving superior performance while conserving computational resources.

Transformer

The transformer architecture was initially introduced in the paper titled "Attention is All You Need" [2]. It is a notable deep learning model extensively utilized in various domains, including natural language processing (NLP), computer vision (CV), and speech processing. Originally developed for machine translation as a sequence-to-sequence model [3], later studies have shown that Transformer-based pre-trained models (PTMs) [4] reliably attain state-of-the-art performance across many tasks. The Transformer architecture has become the benchmark in natural language processing, especially for pre-trained models.

In addition to language-related applications, the Transformer has been widely utilized in Computer Vision (CV) [5, 6, 7], audio processing [8, 9, 10], and fields such as chemistry [11] and life sciences [12]. The popularity of the Transformer has catalyzed the creation of other versions, commonly termed X-formers, each providing enhancements over the original Transformer from diverse perspectives in recent years [13]. The following are some significant enhancements:

1. **Model efficiency:** This presents a considerable problem in deploying the Transformer, principally due to the computational and memory complexity linked to the self-attention mechanism, particularly for extended sequences. Strategies to improve efficiency encompass the implementation of lightweight attention, including sparse attention variants, and the use of Divide-and-conquer strategies that employ recurrent and hierarchical mechanisms.

2. **Model Generation:** The Transformer's adaptable architecture and limited assumptions regarding the structural bias of input data present difficulties in training with small-scale datasets. Strategies to enhance performance in these situations encompass the implementation of structural bias or regularization approaches, along with pre-training on extensive unlabeled datasets.

3. **Model Adaptation:** This change seeks to optimize the transformer for various downstream activities.

The purpose of this study is to explore and investigate different types of modifications made to the transformer model to make it perform better for low data applications.

CHAPTER 2: BACKGROUND

The original Transformer, presented by Vaswani et al. in 2017 [2], is a sequence-to-sequence model comprising an encoder and a decoder, both constructed from a series of identical blocks. Each encoder block primarily consists of a multi-head self-attention mechanism and a position-wise feed-forward network (FFN). A residual link is implemented around each module to construct a more complex model, succeeded by a Layer Normalization module. Unlike the encoder blocks, the decoder blocks integrate cross-attention modules alongside the multi-head self-attention modules and the position-wise feedforward networks. The self-attention modules in the decoder are altered to inhibit any position from attending to subsequent positions. Figure 2 depicts the overall architecture of the original Transformer.

Recurrent neural networks, encompassing long short-term memory and gated recurrent neural networks, were previously recognized as the preeminent methodologies in sequence modeling and transduction tasks, including language modeling and machine translation. Continuous endeavors persist to enhance recurrent language models and encoder-decoder architectures [14, 15, 16].

These recurrent models generally process symbol positions within input and output sequences. They provide a series of concealed states derived from the preceding hidden state and the input at the current point. Nonetheless, their sequential structure constrains parallelization among training examples, which poses challenges with extended sequences due to memory limitations that hinder batch processing across examples. Innovations such as factorization approaches [17] and conditional computation [18] have markedly enhanced computational efficiency and model performance. Nevertheless, the inherent limitation of sequential computation remained.

Attention processes are essential in efficient sequence modeling and transduction, facilitating the modeling of dependencies irrespective of their location in input or output sequences [19, 20]. In

the majority of instances, these attention strategies enhance recurrent networks [21].

The Transformer is a model architecture that eliminates recurrence and solely utilizes an attention method to capture global interdependence between input and output. The Transformer enables significantly enhanced parallelization and has set new standards in translation quality, necessitating fewer computer resources than prior techniques [13].

The objective of reducing sequential computing also supports models such as the Extended Neural GPU [22], ByteNet [23], and ConvS2S [24], all of which employ convolutional neural networks as essential elements to concurrently calculate hidden representations across all input and output positions. These models exhibit a linear increase in computational complexity for ConvS2S and a logarithmic increase for ByteNet as the positional distance expands, hence complicating the learning of relationships between remote positions [25]. Conversely, the Transformer minimizes this to a fixed number of processes, however, this results in diminished effective resolution due to the averaging of attention-weighted positions, a limitation that Multi-Head Attention seeks to address.

Self-attention, or intra-attention, is an attention process that associates several points within a singular sequence to derive its representation. It has demonstrated efficacy in multiple tasks, including reading comprehension, abstractive summarization, textual entailment, and the acquisition of task-independent phrase representations [26, 21, 27, 28].

End-to-end memory networks utilize a recurrent attention mechanism instead of sequence-aligned recurrence and have proven effective in tasks such as basic language question responding and language modeling [29].

The Transformer is the inaugural transduction model that utilizes solely self-attention to generate representations of its input and output, eschewing sequence-aligned RNNs or convolutional meth-

ods. In the following sections, we will present the Transformer, elucidate the reasoning for self-attention, and emphasize its benefits compared to other related algorithms and models [30, 23, 24].

Architecture

The majority of advanced neural models for sequence transduction adhere to an encoder-decoder architecture [31, 19, 3]. Within this paradigm, the encoder processes a sequence of symbol representations (x1, ..., xn) as input and transforms it into a succession of continuous representations given by, z = (z1, ..., zn). The decoder subsequently utilizes these representations z to produce an output sequence (y1, ..., ym) incrementally, one symbol at a time.

In the generation process, the model functions in an auto-regressive manner [32], generating each symbol depending on previously produced symbols, which are subsequently reintroduced into the model as supplementary inputs for ensuing layers. The Transformer employs a generic architecture that incorporates stacked self-attention mechanisms and point-wise, completely connected layers for both the encoder and decoder, as depicted in the left and right sections of Figure 2.1. The subsequent subsections define the specifics of each encoder and decoder architecture, as well as the multihead attention mechanisms.

Encoder

The encoder is comprised up of a stack of N identical layers, two sub-layers in each layer. A multi-head self-attention mechanism is used in the first sub-layer, and a basic feed-forward network that functions independently at each location in the sequence is used in the second sub-layer.

Each of these two sub-layers has residual connections used around it, meaning that each sub-layer's

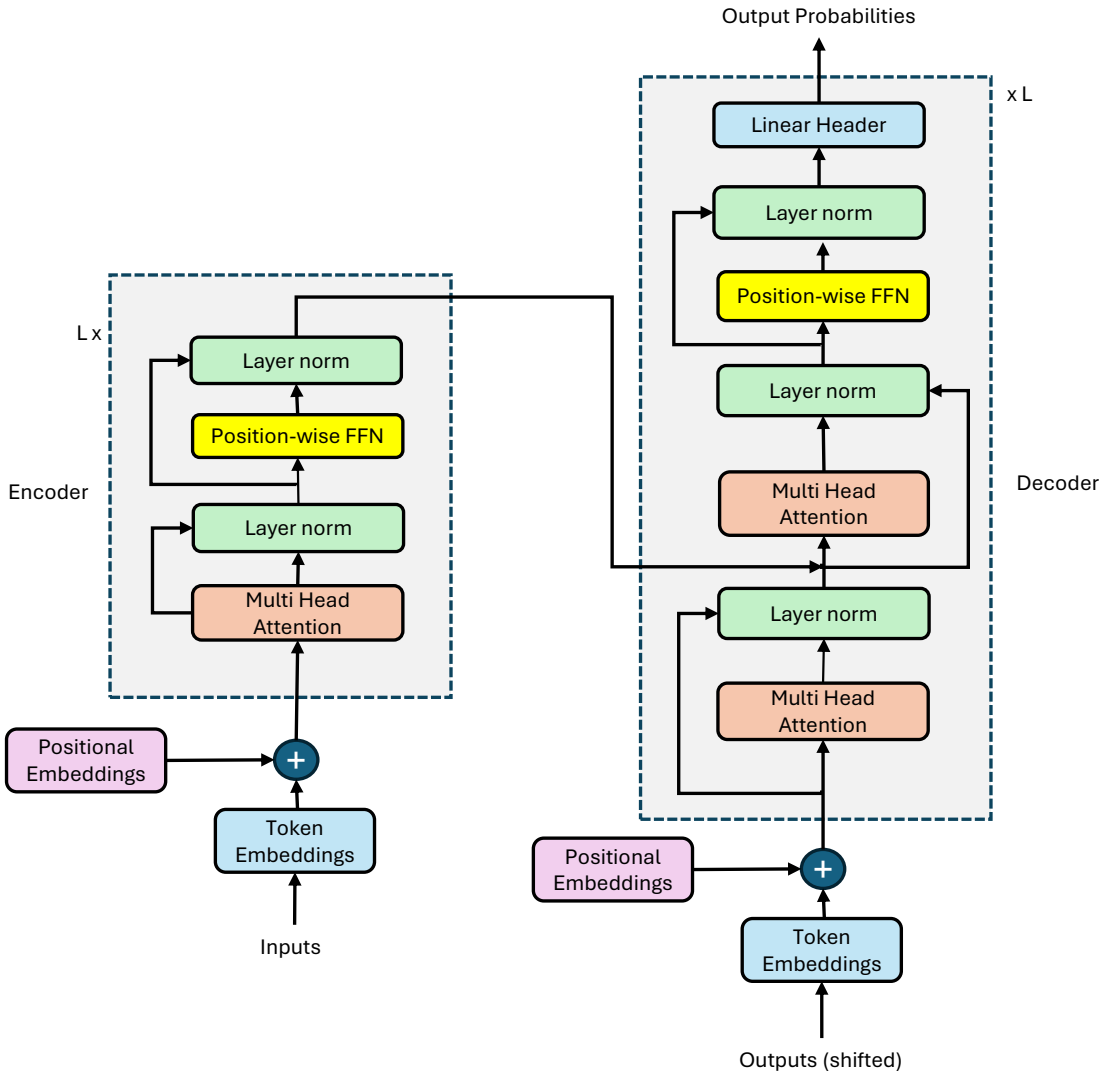

Figure 2.1: The detailed structure of an encoder-decoder transformer. The figure is adapted from [2]

output is first adjusted via a residual connection and then normalized using layer normalization. LayerNorm (x + Sublayer(x)) is how this is expressed, with Sublayer(x) standing for the function that the sub-layer itself performs. All of the model's sub-layers, including the embedding layers, generate outputs with a dimensionality of d_{model} in order to support these residual connections.

Decoder

The decoder module is made up of stacking N identical layers on top of one another. Three sub-layers make up each decoder layer: the first two resemble those in the encoder, while the third sub-layer handles multi-head attention over the output of the encoder stack. Every sub-layer in the decoder has residual connections after layer normalization, much like in the encoder. Furthermore, the decoder's self-attention sub-layer is altered to inhibit attention to positions that follow. Predictions for one position are guaranteed to rely solely on the known outputs at the positions that came before it because of the masking and the offsetting of the output embeddings' by one position. This makes sure that the model does not know the output before generating it.

Attention

An attention function generates an output by receiving a query and a collection of key-value pairs. The query, keys, values, and output are all instances of one dimensional vectors. A weighted total of the values is computed to generate the output; each weight is obtained from a compatibility function that computes how well the query matches the matching key.

Scaled Dot Product Attention

Figure 2.2 shows the Scaled Dot-Product Attention mechanism that is implemented in transformers. The inputs to this module are values of dimension d_v along with queries and keys with dimension d_k. To determine the weights for the values, first the dot products of the query with all the keys are computed followed by the division of each result by the square root of d_k. The outputs are then passed through a Softmax layer to determine the weights for the values.

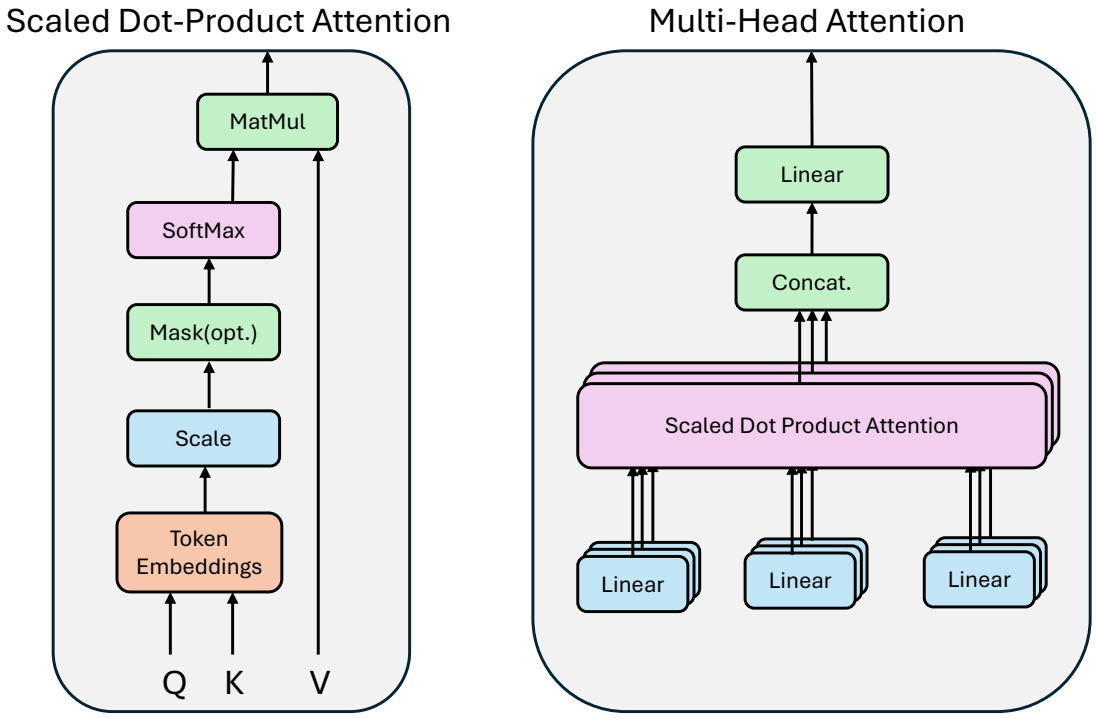

Figure 2.2: Illustration of Scaled Dot-Product and Multi-Head attention Modules. The figure is adapted from [2]

In practice, the queries are processed simultaneously by combining them into a matrix Q. The keys and values are also combined in a similar manner into matrices K and V. Next, equation 2.1 is used to compute the output matrix.

$$Attention(Q,K,V) = softmax(\frac{QK^T}{\sqrt{d}})V \tag{2.1}$$

Additive attention and dot-product (multiplicative) attention are the predominant forms of attention mechanisms utilized in contemporary applications. Dot-product attention resembles this strategy, incorporating a scaling factor that is the inverse of the square root of d_k. Additive attention computes the compatibility function with a feed-forward network comprising a single hidden layer.

Despite both techniques exhibiting comparable theoretical complexity, dot-product attention is more economical in terms of speed and space in practice, owing to optimized matrix multiplication algorithms. For minimal levels of d_k, both systems exhibit comparable performance.

Nonetheless, for elevated values, additive attention outperforms unscaled dot-product attention. This occurs due to the fact that, with substantial d_k, the dot products increase significantly, causing the softmax function to enter regions characterized by minimal gradients. The dot products are divided by the square root of d_k to normalize it and in turn resolving this issue.

Multi-Head Attention

Rather than employing a singular attention function with keys, values, and queries as vectors of dimension d_{model}, it is advantageous to linearly project the queries, keys, and values h times into dimensions d_k, d_k, and d_v, respectively, with distinct learnt linear projections. The attention function is thereafter executed in parallel on the projected queries, keys, and values, yielding d_v-dimensional output values. The outputs are concatenated and subsequently projected to yield the final values, as illustrated in Figure 2.2.

Multi-head attention allows the model to simultaneously focus on input from diverse representation subspace at multiple positions. Utilizing a solitary attention head would result in averaging, potentially obscuring significant features.

$$MultiHead(Q,K,V) = concat(h_1,h_2,...,h_h)W^o \tag{2.2}$$

$$h_i = Attention(QW_q^i, KW_i^K, VW_i^V) \tag{2.3}$$

The calculation of MultiHead attention is presented in equations 2.2 and 2.3 [2]. In this context, W_q^i and W_i^K possess dimensions d_{model} by d_k, while W_i^V has dimensions d_{model} by d_v. The lower dimensionality of each head results in an overall computing cost equivalent to that of single-head attention with full dimensionality.

The Multi-Head Attention modules are utilized in three distinct locations within the transformer architecture:

1. **Encoder-Decoder Attention:** The queries from the preceding decoder layer are input into the current encoder-decoder layers, whereas the keys and values are derived from the encoder's output. This configuration enables each position in the decoder to attend to all positions in the input sequence, analogous to conventional encoder-decoder attention methods employed in sequence-to-sequence models.

2. **Encoder Self-Attention:** These levels utilize keys, values, and queries all originating from the same source—the output of the preceding encoder layer. This enables each place in the encoder to focus on all other points in the preceding layer of the encoder.

3. **Decoder Self-Attention:** Equivalent to the encoder, these layers enable each location in the decoder to attend to all preceding positions, including the present one. To preserve the auto-regressive feature and inhibit backward information flow, the value is assigned negative infinity, whereas all values in the softmax input correspond to prohibited connections. This is executed within the scaled dot-product attention mechanism, as illustrated in Figure 2.2.

Position-Wise Feed-Forward Network

$$FFN(x) = max(0,\ xW_1 + b_1)W_2 + b_2 \tag{2.4}$$

Each layer of our encoder and decoder, in addition to the attention sub-layers, incorporates a fully connected feed-forward network that is applied independently and uniformly to each position. This network has two linear transformations interspersed with a ReLU activation in between, as delineated in equation 2.4 [2].

Although the linear transformations remain uniform across several places, they possess unique parameters for each layer. This can also be characterized as two convolutions utilizing a kernel size of 1. The input and output dimensions are equivalent to d_{model}, whereas the inner layer possesses a dimensionality four times that of d_{model}.

Input Embeddings

Similar to other sequence transduction models, input and output tokens are converted into vectors of dimension d_{model} by learned embeddings. A conventional learnt linear transformation is used, succeeded by a Softmax function to translate the decoder's output into predicted probabilities for the subsequent token. In the initial model, an identical weight matrix is employed for both embedding layers and the pre-Softmax linear transformation, adhering to the methodology outlined in [23]. In the embedding layers, these weights are normalized by the square root of d_{model}.

Positional Encoding

The fundamental transformer model is devoid of recurrence and convolution; therefore, it is essential to integrate information regarding the tokens' relative or absolute positions to leverage the sequence order. This is accomplished by incorporating "positional encoding" into the input embedding at the foundation of both the encoder and decoder stacks. The positional encoding possess the same dimension, d_{model}, as the embeddings, enabling their summation. Numerous alternatives for

Table 2.1: Computational complexities of self attention network compared to other similar algorithms

Layer Type	**Complexity**	**Sequential Operations**	**Maximum Path Length**
Self Attention	O($n^2 . d$)	O(1)	O(1)
Recurrent	O($n . d_2$)	O(n)	O(n)
Convolutional	O($k . n . d_2$)	O(1)	O($log_K(n)$)
Self-Attention (restricted)	*O(r . n . d)*	O(1)	O(n/r)

positional encoding exist, encompassing both learned and fixed types [9].The positional encoding is constructed using a mixture of sine and cosine functions, as demonstrated in equations 2.5 and 2.6 respectively [2].

$$PE_{(pos\,2i)} = sin(pos/10000^{2i/d_{model}}) \tag{2.5}$$

$$PE_{(pos\,2i+1)} = cos(pos/10000^{2i/d_{model}}) \tag{2.6}$$

Here, "pos" denotes the position, while "i" signifies the dimension. Every dimension of the positional encoding corresponds to a sinusoidal function. The wavelengths of these sinusoids constitute a geometric progression from 2π to 10000.2π. This function was chosen because it facilitates the model's ability to effectively learn to focus on relative positions. For each constant offset k, PE_{pos+k} can be represented as a linear function of PE_{pos}.Learned positional embeddings [24] were also evaluated in the original research [2], revealing that both methodologies produced essentially comparable outcomes. The sinusoidal variant was chosen since it may facilitate the model's gen-

eralization to sequence lengths beyond those seen during training.The computational complexity of the self-attention mechanism is presented in Table 2.1 [1], alongside other prior systems. The restricted self-attention layer can be computed with little complexity compared to other analogous algorithms.

In Table 2.1, n denotes the sequence length, d signifies the dimension of the representation, k indicates the kernel size for the convolutional layers, and r represents the size of the attention neighborhood in the restricted self-attention module.

Self-Attention

This section presents a comparison between self-attention layers and the recurrent and convolutional layers commonly employed to transform one variable-length sequence of symbol representations (x1, x2, ..., xn) into another sequence of equivalent length (z1, z2, ..., zn), where x_i and z_i are n-dimensional real vectors, analogous to the hidden layers in standard sequence transduction encoders or decoders. Three major considerations for self-attention are outlined below:

1. **Computational Complexity per Layer:** This quantifies the total computation necessary for each layer.

2. **Parallelization Potential:** This evaluates the extent to which computing can be parallelized, as represented by the least number of sequential operations required.

3. **Path Length for Extended Dependencies:** Acquiring long-range dependency is essential for numerous sequence transduction tasks. The efficacy of learning these relationships is affected by the length of the paths that forward and backward signals must navigate inside the network. Shorter pathways between any points in the input and output sequences enhance the learning of long-range relationships [25].

Self-attention is an essential element of the Transformer, providing a versatile method for processing variable-length inputs. It can be seen as a fully connected layer with weights produced dynamically depending on paired input relationships. Table 2.1 juxtaposes the complexity, sequential processes, and maximum path length of self-attention against three prevalent layer types. The benefits of self-attention are delineated as follows:

1. It possesses an equivalent maximum path length to completely connected layers, rendering it adept at simulating long-range interdependence. In comparison to completely connected layers, it exhibits greater parameter efficiency and superior capability in managing variable-length inputs.

2. Convolutional layers possess a restricted receptive field, generally necessitating a deep network architecture to attain a global receptive field. Conversely, the fixed maximum path length of self-attention enables it to effectively model long-range relationships with a uniform layer count.

3. The fixed amount of sequential operations and maximum path length render self-attention more parallelizable and superior for long-range modeling compared to recurrent layers.

Table 2.1 illustrates that a self-attention layer interlinks all places with a fixed number of sequential operations, whereas a recurrent layer necessitates O(n) sequential operations. Regarding computational complexity, self-attention layers exhibit superior speed compared to recurrent layers when the sequence length n is less than the representation dimensionality d, a scenario frequently encountered in sentence representations utilized in advanced machine translation models, such as word-piece and byte-pair representations. To improve computational efficiency for jobs with extensive sequences, self-attention can be constrained to a region of size r surrounding each output position in the input sequence, therefore augmenting the maximum path length to O(n/r).

A solitary convolutional layer with a kernel width k<n fails to establish connections between all pairs of input and output positions. Accomplishing this necessitates the arrangement of O(n/k) convolutional layers for contiguous kernels, or O($log_K(n)$) layers for dilated convolutions [23], hence extending the length of the longest pathways between any two points inside the network. Convolutional layers are typically more computationally intensive than recurrent layers, by a factor of k. Nonetheless, separable convolutions [33] markedly diminish this complexity to O($k \,.\, n \,.\, d + n \,.\, d^2$). Even when k=n, the complexity of a separable convolution is analogous to that of a self-attention layer coupled with a point-wise feed-forward layer, as utilized in the transformer architecture.

Furthermore, self-attention may result in models that are more interpretable. Individual attention heads distinctly learn to execute various tasks, with numerous exhibiting behaviors associated with the syntactic and semantic structure of phrases.

Masked Self-Attention

Inside the Transformer decoder module, self-attention is restricted such that queries at each location may only attend to key-value pairs up to and including that position. To facilitate simultaneous training, a masking function is implemented to the unnormalized attention matrix, $\hat{A} = exp(\sqrt{d_k} QK^T)$, where positions that are not valid are masked by assigning $\hat{A}_{ij} = -\infty$ if j>i. This form of self-attention is typically referred to as autoregressive or causal attention.

Finally, in the context of cross attention, the queries are obtained from the outputs of the preceding decoder layer, whilst the keys and values are produced from the outputs of the encoder.

Types of Transformers

The Transformer architecture can typically be employed in three distinct manners. The details of these structures are discussed below:

1. **Encoder-Decoder:** In this type of architecture, the complete Transformer architecture, as illustrated in Figure 2.1 is used. This is frequently utilized for sequence-to-sequence tasks, such as neural machine translation. Examples of this model type include BART, T5, Flan, among others.

2. **Encoder Only:** The encoder is utilized only, where the model output captures the latent features of the input sequence. This methodology is frequently utilized for Natural Language Understanding (NLU) applications such as text classification and sequence labeling. Prominent examples include BERT, ALBERT, and RoBERTa.

3. **Decoder Only:** The configuration utilizes solely the decoder, eliminating the encoder-decoder cross-attention module. This setup is commonly employed for sequence generation tasks, including language modeling. Common models that adhere to this structure include GPT, ChatGPT, XLNet, PaLM, and LaMDA.

Inductive Bias

The Transformer model is often contrasted with convolutional and recurrent networks. Convolutional networks are recognized for their inductive biases of translation invariance and locality via shared local kernel functions. Likewise, recurrent networks integrate inductive biases of temporal invariance and localization through their Markovian framework [34]. Conversely, the Transformer design imposes minimal assumptions on the structural information of data, rendering it a universal

and adaptable framework. Nonetheless, this absence of structural bias may result in overfitting on limited datasets. Graph Neural Networks (GNNs) utilizing message forwarding represent a closely associated category of networks. The Transformer can be seen as a Graph Neural Network (GNN) established on a complete directed graph (including self-loops), with each input represented as a node within the graph. The primary distinction between Transformers and GNNs is that Transformers do not incorporate any prior information of the structure of the input data; their message-passing mechanism is exclusively based on similarity assessments of the content.

Sparse Attention

Self-attention is a crucial part of the Transformer architecture; nonetheless, it poses two practical challenges:

1. **Complexity:** Self-attention functions with a complexity of $O(n^2 . d)$. Thus, the attention module may serve as a bottleneck during the processing of lengthy sequences.

2. **Absence of Structural Bias:** Self-attention does not presuppose any intrinsic structural biases in inputs. It depends on acquiring even the sequential information from the training data. Consequently, Transformers (lacking pretraining) are susceptible to overfitting on small or moderately big datasets.

Numerous enhancements have been suggested since the inception of the original transformer to address the challenges associated with the attention mechanism:

1. **Sparse Attention:** This method incorporates a bias towards sparsity in the attention mechanism, with the objective of reducing complexity.

2. **Linearized Attention:** This approach separates the attention matrix utilizing kernel feature maps and calculates attention in a reversed sequence to attain linear complexity.

3. **Prototype and Memory Compression:** Techniques in this area diminish the quantity of queries or key-value pairs in memory to decrease the dimensions of the attention matrix.

4. **Low-rank Self-Attention:** This method utilizes the low-rank characteristic of self-attention to diminish computing demands.

5. **Attention with Prior:** This research avenue investigates the integration or substitution of conventional attention processes with prior attention distributions.

6. **Enhanced Multi-Head Mechanism:** This study concentrates on augmenting the efficacy of the multi-head attention mechanism. In the conventional self-attention mechanism, each token must attend to all other tokens. It has been shown that in trained Transformers, the learnt attention matrix A is frequently highly sparse over the majority of cases [35]. Consequently, it is possible to reduce computing complexity by implementing structural biases that limit the amount of query-key pairs to which each query is exposed. Under this constraint, the similarity score of query-key pairings, determined by predetermined patterns, is presented in equation 2.7 [2].

$$\hat{A} = \begin{Bmatrix} q_i k_j^T & if\ token\ i\ attends\ to\ token\ j \\ -\infty & if\ token\ i\ does\ not\ attend\ to\ token\ j \end{Bmatrix} \tag{2.7}$$

Here, $\hat{A}$ represents the attention matrix in its unnormalized state.

Viewed from another perspective, standard attention resembles a complete bipartite graph in which each query assimilates information from all memory nodes to refine its representation. Sparse

attention, conversely, resembles a sparse graph in which specific connections between nodes are excluded. These methodologies can be categorized into two types according to their mechanisms for identifying sparse connections: position-based and content-based sparse attention.

Transfer Learning

Transfer learning (TL) in machine learning (ML) involves the refinement of a model that has been pre-trained on one task and then deployed to a new, related task. Creating a new machine learning model is resource-intensive and time-consuming, necessitating significant data, computational resources, and numerous iterations prior to implementation. Transfer learning provides an effective alternative by pretraining existing models on analogous tasks using novel data. A model trained to recognize photographs of dogs can be modified to detect cats by utilizing a smaller dataset that highlights the distinguishing characteristics between dogs and cats.

Benefits of Transfer Learning

Transfer learning (TL) offers numerous advantages to researchers engaged in the development of machine learning applications:

1. **Improved Efficiency:** Training machine learning models is laborious and resource-demanding as they assimilate patterns and acquire knowledge from extensive datasets. In transfer learning, pre-trained models preserve core knowledge, weights, and functions, facilitating expedited adaption to novel tasks. This method necessitates reduced datasets and diminished processing resources while attaining similar or enhanced outcomes.
2. **Enhanced Accessibility:** Constructing deep-learning neural networks generally requires

substantial data, computational resources, and time. TL overcomes these obstacles, enabling enterprises to utilize ML for particular applications. Current models can be modified at lower prices, facilitating applications in various domains such as medical imaging analysis, environmental monitoring, or facial recognition with minor alterations.

3. **Superior Performance:** TL-derived models frequently demonstrate superior robustness in diverse and difficult contexts. They are proficient in managing real-world variability and noise as a result of exposure to various settings throughout their initial training. This adaptability results in enhanced performance and flexibility in uncertain circumstances.

Types of Transfer Learning

The method selected for facilitating transfer learning (TL) is contingent upon the model's domain, the specific task it must execute, and the accessibility of training data. The different types of Transfer learning are outlined below:

1. **Transductive Transfer Learning:** Transductive transfer learning entails the transfer of knowledge from a particular source domain to a corresponding target domain, with a primary emphasis on the target domain. This strategy is especially advantageous when there is scarce or absent labeled data for the target domain. In transductive transfer learning, the model leverages previously acquired knowledge to predict target data. The model may find patterns more effectively due to the mathematical similarities between the target data and the source data. For instance, adopting a sentiment analysis model trained on product reviews to evaluate movie reviews exemplifies transductive transfer learning. Despite the differences in context and particular between product reviews and movie reviews, they exhibit structural and linguistic commonalities. The algorithm rapidly adapts its comprehension of sentiment from the product domain to the film domain.

2. **Inductive Transfer Learning:** Inductive transfer learning transpires when the source and target domains are same, yet the tasks required of the model vary. The pre-trained model is already acquainted with the source data, hence expediting training for new tasks. In natural language processing (NLP), models are initially trained on an extensive corpus of texts and subsequently refined through inductive transfer learning for specific tasks, such as sentiment analysis. In computer vision, models such as VGG are pre-trained on large image datasets and subsequently fine-tuned for tasks like object detection.

3. **Unsupervised Transfer Learning:** Unsupervised transfer learning employs an approach similar to inductive transfer learning to cultivate new competencies. It is utilized when solely unlabeled data is accessible in both the source and target domains. The model acquires common features from unlabeled input to enhance its generalization capabilities while addressing a specific purpose. This method is advantageous when acquiring tagged source data is difficult or expensive. For instance, contemplate training a model to recognize various sorts of motorcycles in traffic photos. The model initially learns from a substantial collection of unlabeled vehicle photos, identifying similarities and distinguishing characteristics among different vehicle categories such as cars, buses, and motorcycles. Subsequently, the model is presented with a limited, targeted collection of motorbike photos. Its performance has markedly improved due to past unsupervised learning.

The dissertation is divided into three projects where the ideas have been demonstrated on real world applications in the field of bioinformatics and biomedical engineering. These projects are outlined in detail in chapter 3, 4 and 5, while chapter 6 presents the conclusion statement, along with a summary and analysis of the outcomes from the three studies. It also encompasses a discussion and potential future work.

CHAPTER 3: PROT_EC: A TRANSFORMER BASED DEEP LEARNING SYSTEM FOR ACCURATE ANNOTATION OF ENZYME COMMISSION NUMBERS

The advancements in next-generation sequencing technologies have given rise to large-scale, open-source protein databases consisting of hundreds of millions of sequences. However, to make these sequences useful in biomedical applications, they need to be painstakingly annotated by curating them from literature. To counteract this problem, many automated annotation algorithms have been developed over the years including deep learning models, especially in recent times. In this work[1][36], we propose a transformer-based deep-learning model that can predict the Enzyme Commission numbers of an enzyme from full-scale sequences with state-of-the-art accuracy compared to other recent machine learning annotation algorithms. The system does especially well on clustered split dataset which consists of training and testing samples derived from different distributions that are structurally dissimilar from one other. This proves that the model is able to understand deep patterns within the sequences and can accurately identify the motifs responsible for the different enzyme commission numbers. Moreover, the algorithm is able to retain similar accuracy even when the training size is significantly reduced, and also, the model accuracy is independent of the sequence length making it suitable for a wide range of applications consisting of varying sequence structures.

[1]Part of the materials in this chapter is published in IEEE/ACM Transactions on Computational Biology and Bioinformatics

Related Work

The rapid development of sequencing technologies has made it practical and cost-effective to have large databases of protein sequences both in the private domain and also under publicly available open-source licenses. This large increase in the availability of protein sequences gave rise to protein databases like Uniprot [37] consisting of hundreds of millions of data points. And with every passing day, more than a hundred thousand proteins are being added to these continually growing global public protein databases [37]. However, in order to make these protein databases useful to their full extent, they need to be correctly annotated by the scientific community. To help out with this problem, significant effort is put in by domain experts and curators to manually annotate these protein sequences, but this process is very slow and only about 0.03% of the proteins available in the public databases could be successfully annotated in this manner [37].

The pressure to find faster methods to annotate large volumes of proteins has resulted in the rise of numerous automated and semi-automated systems. Several types of annotation systems evolved over the years that used different techniques to label the proteins. One method was to compare the sequence of the target protein to other already annotated proteins to find similarities between them. The target was given the same annotation as the known protein if significant patches of sequence between them matched up with each other. BLAST [38] was one of the most successful tools that used this type of method to annotate protein sequences.

Another popular method for protein annotation that came later on used functional motifs to annotate target proteins. It assigned a particular function to a protein if it contains the motif associated with that function. Later versions of these models started using profile Hidden Markov models (HMMs) which made them more robust and able to find soft matches [39, 40]. These models computed the probability of a new sequence having a particular function if the probability went above a certain threshold. They are fast and efficient leading to many prominent protein databases

like Pfam and InterPro using them to annotate their large protein databases [41, 42]. However, these models, despite being quick and accurate, do have their limitations. Firstly, they compute the probability of each function motif individually thus being less efficient and accurate when there exist functional groups that share a common feature. Moreover, the signatures used to detect a particular function in a protein sequence often require involvement from human curators and are not fully automated, thus making the whole process somewhat slow and arduous. These limitations warrant the need for further study into more effective annotation systems in order to process a large number of proteins that are added to the databases every day.

Machine learning algorithms or more specifically a variant of it known as deep learning, which could be characterized as models having more than one hidden layer, have gained a lot of attention in recent times for their success in a wide range of different applications. Early deep learning models worked on applications in computer vision and machine translation but they soon expanded into other fields including medical imaging and bioinformatics.

The operation of a deep learning model generally consists of two stages. The first is the training step where labeled and semi-labeled data is used to learn the weights of the model followed by the inference step where the learned weights in the model are used to determine the label of an unknown sample. As the lower layers in a model are involved with more generalized functions, they could be shared between similar tasks. This method is known as transfer learning, where, instead of training from scratch for a new task, the weights of the model could be initialized from a similar pretrained application. After initialization, the model weights are trained on the new data in a step known as fine-tuning. This process produces better accuracy with less amount of training data, requiring lower training time and computational costs [43].

Following the success of deep learning models in various fields and applications, a number of studies have been carried out to apply them in protein functional prediction and classification

tasks [44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55]. Moreover, deep learning models have also been implemented on protein structure prediction [56, 57, 58, 59, 60] and protein design [61, 62, 63, 64, 65]. A milestone in protein annotation began when researchers started to feed large amounts of raw unstructured data into large deep-learning models. This spared the models from human biases, helped identify patterns that haven't been found yet, and alleviated the system from bottlenecks caused by the slow curation process. Recent advances in this area include DeepEC [66] which is an ensemble CNN model that annotates protein sequences with EC numbers. However, it could only annotate sequences that are less than a thousand units long and requires to have at least 10 datapoints of a single class to work properly. Other major contributions consist of CNN-based ensemble models to annotate unaligned protein domain sequences [67], using a transformer-based model to make an improvement in both accuracy and computational requirements [68], training different models to understand the structure of protein sequences [69] and building a transformer-based model to predict GO terms [70]. However, these studies do not deal with full-scale protein sequences which are often important for Bioinformatics applications. This problem has been counteracted in Proteinfer [71], which uses both a single and an ensembled CNN architecture to annotate full-scale protein sequences with GO terms and Enzyme Commission (EC) numbers.

The purpose of this study is to build upon Proteinfer [71] by proposing a transformer-based deep learning system that is able to take full-scale protein sequences and annotate them with EC numbers with better accuracy compared to Proteinfer, which we used as our primary benchmark. The proposed algorithm is able to predict all four stages of the hierarchical EC number of an enzyme given the input sequence.

The main novelties of our study are given below:

- Achieving state-of-the-art accuracy on EC number annotation of full-scale protein sequences.
- Using a single model instead of an ensemble of models to make the system lighter and easier

to use.

- The model requires much less time and computational resources for training compared to other models.
- The proposed model is equipped with fully Automated hyperparameter tuning, which makes it easier to train, requiring no development or validation dataset.
- Better understanding of deep patterns within the protein sequences which is evident from the narrow gap in accuracy between the random and clustered dataset splits.
- The ability of the model to be able to perform very well even with a small amount of data.

This chapter is organized into four sections. The first section provides an introduction to the project with relevant literature reviews. The second section describes the model architecture in detail and provides a description of the methodology. The third section shows the results obtained and compares them to other models. The fourth section discusses the various issues and technical information relating to the study. Finally, the fifth section concludes the work and provides further discussions with potential future research directions.

Model

This section describes the detailed model architecture and operating procedure along with the description of the dataset and some useful terminologies.

Enzyme Commission (EC) numbers

Enzymes are a type of protein that catalyzes a biological chemical reaction. They are classified by the type of reactions they catalyze in the form of a hierarchical system known as the EC number which is made up of four integers separated by dots [72].

- The first number, which ranges from 1 to 7, signifies the broad type of reactions that the enzyme catalyzes.
- The second EC number represents the sub-class of the first EC number.
- The third number denotes the sub-subclass.
- Finally, the fourth number stands for the sub-subclass serial number for the enzyme.

So, the EC number of an enzyme serves as its primary identifier and is therefore of much importance to the scientific community.

Dataset

There are several open-source public repositories that host a large amount of protein data. The biggest of them is Uniprot [73] which contains a protein sequence database known as UinprotKB made up of two sections: SwissProt, which consists of more than five hundred thousand reviewed protein sequences, and TrEMBL, which is made up of around 200 million unreviewed sequences. The Swissprot part of UniprotKB is used as the dataset for this work as they are manually curated from the literature and reviewed by domain experts as opposed to TrEMBL which is unreviewed and is computationally annotated with the help of various automated systems like ARBA [73] and UniRule [74].

Each protein sequence in the UniProt database contains several cross-referenced annotations like GO terms and EC numbers and they are verified using a 6-stage process with multiple credible sources like Pfam [42] and TIGRFAM [75]. The information from all these sources is fed into a mother database called InterPro [41], which could track the exact source of each annotation present in the SwissProt database.

The dataset consists of around 550,000 protein sequences which were split into training, development, and testing sets using two techniques. In the first case, the datapoints were split randomly between the three sets. For the second method called clustered split, UniRef50 [76] was used to split the data in such a way that sequences that are similar in structure are contained within a given split. This forces the training set to be made up of sequences that are structurally different from the development and testing sets ensuring that the test results outputted by the algorithm are fair and not contaminated by unrelated correlations between the sequences. Hence, the algorithm has to understand the exact patterns in the sequences responsible for each EC number to make accurate predictions. This type of method to create a clustered split produces a challenging test case and was the same method used in Proteinfer [71], which we used as our primary benchmark.

Both the random and clustered dataset splits of SwissProt were made publicly available in a data repository by the authors of proteinfer [71]. We used the exact dataset by downloading them from their website so that we can directly compare the test results outputted by our algorithm with that of Proteinfer [71]. The downloaded datapoints each contained a sequence and their corresponding GO terms and EC number annotations. We discarded the GO terms and only took the protein sequences and their corresponding EC numbers as only these are required in this work. The repository contained 438,522 datapoints among which 208,823 sequences are enzymes. These are filtered from the rest and are used for training and testing our model. A few of the enzymes did not have all four EC numbers available and the composition of enzymes in the dataset with each EC number in the training, development, and testing sets for the random and clustered splits are

Table 3.1: Availability of EC numbers in the Random split

Split	Proteins	EC1	EC2	EC3	EC4
Train	438,522	208,823	206,865	200,978	178,302
Dev	55,453	26,724	26,487	25,784	23,010
Test	54,289	25,892	25,665	24,992	22,183

Table 3.2: Availability of EC numbers in the Clustered split

Split	Proteins	EC1	EC2	EC3	EC4
Train	182,965	87,520	86,667	84,273	74,943
Dev	180,309	86,230	85,537	83,156	73,641
Test	18,3475	87,661	86,784	84,299	74,892

given in Table 5.1 and Table 5.2 respectively. In the case of the random split, 80% of the sequences are put in the training set with around 10% in the development and testing sets each. On the other hand, for the clustered split, the data is equally divided between the three splits making each split about one-third of the total dataset.

A number of other datasets were also used to compare the performance of Prot_EC with other models. Among these include the New dataset and the Cofactor dataset which was created by the authors of DEEPre [50] and downloaded directly from their repository. Also, the Uniprot and Held out dataset made in ECPred [45] was used to test the performance of Prot_EC with various other deeplearning models. The details of how these datasets were made are available in the DEEPre and ECPred publications [50, 45].

Transformers

Transformers are a type of deep learning algorithm that was first introduced in the paper entitled "Attention is All you Need" [2], where it was used for natural Language Processing (NLP) tasks. At the heart of the model were Attention blocks which the algorithm used for most of its computations. The model consisted of two parts: An encoder and a decoder. The encoder was responsible for converting input text into a vector representation while the decoder block converted the vector representation into an output text. The transformer is a sequence model as it takes the input tokens one by one and outputs another sequence in the same manner.

Later on, the transformer model was extended to other applications like the Vision Transformer [7], which is used to classify images. The different transformer types came in three categories. The first is the encoder models like ALBERT [77], BERT [78], DistillBERT [79], and ELECTRA [80] containing only an encoder. The second is the decoder models like CTRL [81], GPT-2 [82] and Transformer-XL [83], having only a decoder and the third is an encoder-decoder model like BART [84], and T5 [85] which consists of both an encoder and a decoder like the original transformer model [2].

ProtBert

The building blocks of our model are made up of a modified version of ProtBert [69], which is an encoder-only model pretrained on the BFD100 dataset consisting of 2,122 million protein sequences. The pretrained version of ProtBert was downloaded from the HuggingFace website [86]. During pretraining, portions of the sequence given by the masking probability were hidden and the model was trained to predict the masked tokens. This makes the model understand and learn deep patterns in the data along with the relationship between the different amino acids in the

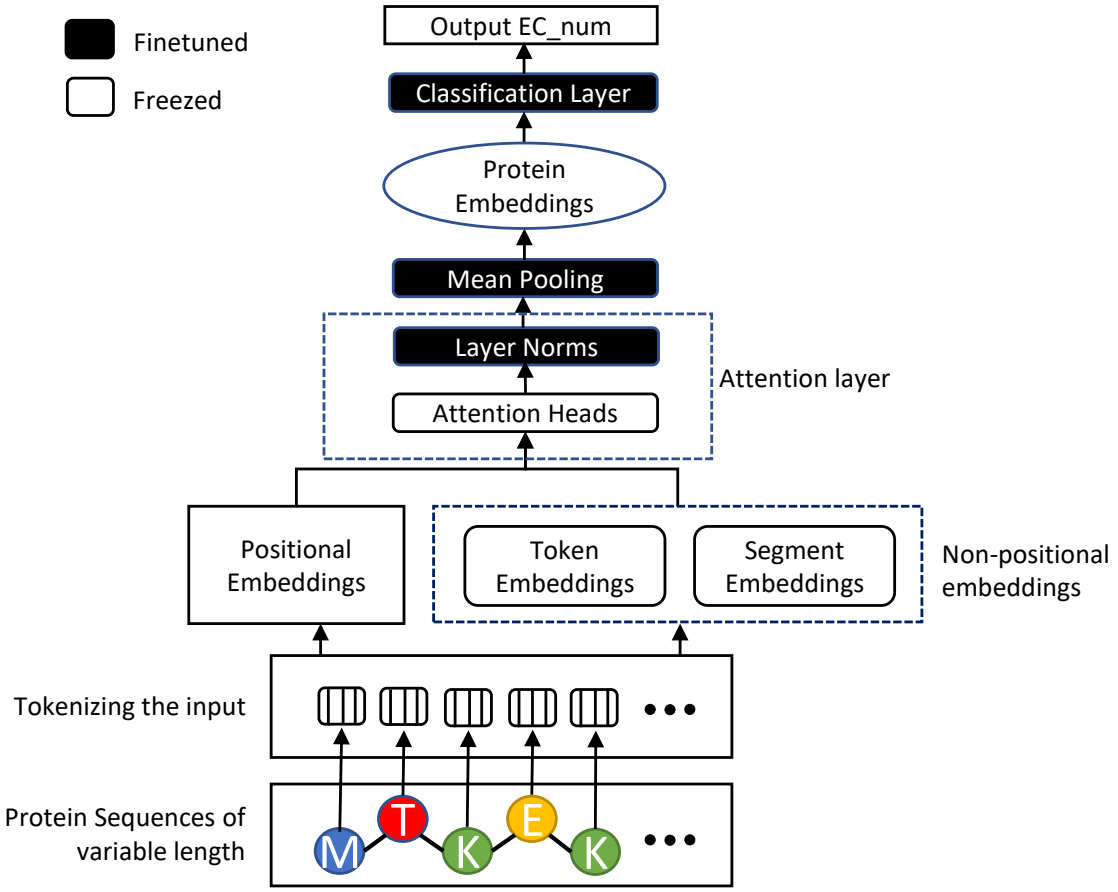

Figure 3.1: Modified architecture of the ProtBert Module that was used in Prot_EC. The figure also indicates which layers were finetuned during the training process.

protein sequence [69]. The ProtBert module consisted of 30 hidden layers each having 1024 nodes and an intermediate hidden layer size of 4096. There were 16 attention heads and the model had 420 Million parameters in total. It was pretrained in [69] with the help of 128 Nodes and 1024 TPUs for 800 thousand steps on sequences with a maximum sequence length of 512 followed by another 200 thousand steps on sequences consisting of a maximum length of 2,000. A masking probability of 15% was used along with a Lamb optimizer. A learning rate of 0.002 was selected with a weight decay rate of 0.01.

In order to modify the ProtBert module and use it in our work, we downloaded the pretrained weights and finetuned them on enzyme sequences. However, when it comes to large transformer models like ProtBert, even finetuning it requires a large amount of data and computational resources. To counteract this, we have implemented selective fine-tuning where some of the layers of the model are tuned while others are left to their pretrained values. This technique produces better accuracy on large models where the amount of data is limited [87, 88]. The basic architecture of the ProtBert model along with the selective finetuned layers is given in Figure 3.1

Algorithm 1 Data Preprocessing

```
trainDf = pandas dataframe of training set;
for i ← 0 to len(trainDf) do
    if len(trainDf [i] ['EC']) == 1 then
        EC_num.append(trainDf [i] ['EC']); seq.append(seq[i]);
    if len(trainDf [i] ['EC']) > 1 then
        EC_num.append(trainDf [i] ['EC'] [0]);
end
[EC1, EC2, EC3, EC4] = EC_num.split('.');
seq2 = str(EC1) + seq;
seq3 = str(EC1) + str(EC2) + seq;
seq4 = str(EC1) + str(EC2) + str(EC3) + seq;
[EC1_, EC2_, EC3_, EC4_] = labelEncoder([EC1, EC2, EC3, EC4])
[EC1x, EC2x, EC3x, EC4x] = OneHotEncoder([EC1_, EC2_, EC3_, EC4_]);
[seqx, seqx2, seqx3, seqx4] = tokenize([seq, seq2, seq3, seq4])
tokenizer.addSpecialTokens(str([EC1,EC2,EC3,EC4)])
```

Result: EC1x, EC2x, EC3x and EC4X are the output labels while seqx, seqx2, seqx3 and seqx4 are the input sequences

Data Preprocessing

The data has to be preprocessed in a series of steps in order to make it suitable to be injected into the algorithm. The downloaded data [71] consisted of a series of protein sequences along with their corresponding EC numbers and GO terms. The sequences were filtered to separate out the proteins that were enzymes and had a corresponding EC number which was used as the target for

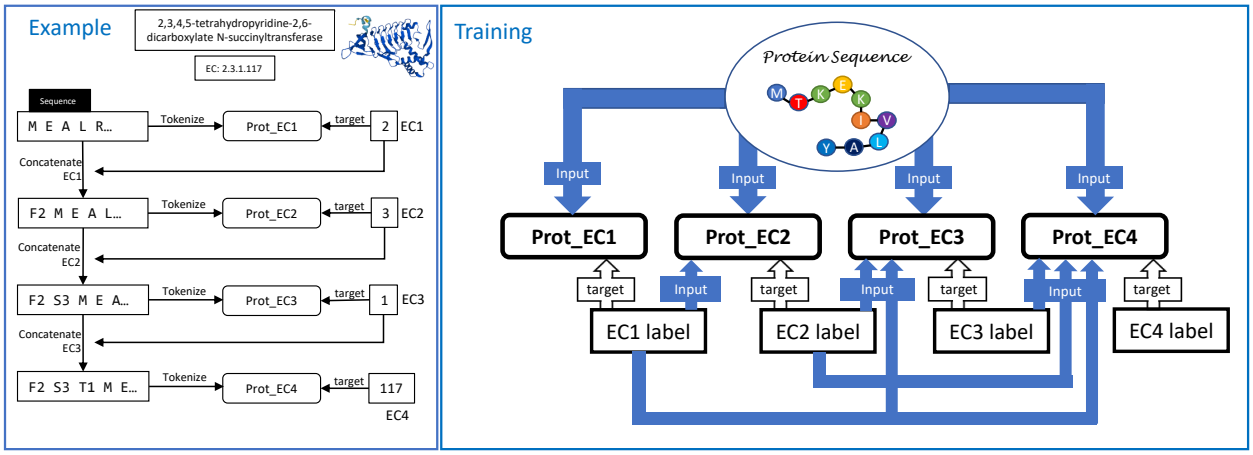

Figure 3.2: Right: Data Flow block diagram during Training Stage of Prot_EC. Detailed architecture of each Prot_ECX module is given in Figure 3.1. Each Prot_ECX block is responsible for calculating one stage of the EC number. The EC number of the previous stage is also concatenated with the sequence before feeding it to the next Prot_ECX block. Left: Example dataflow showing details about how the data is fed to the blocks during the training stage.

the classifier models. The EC numbers for each sequence were separated based on the hierarchical stages and converted into one hot vector to make it suitable for the model. Next, the sequences were tokenized which involved assigning an integer number to each of the amino acids present in the data. Special tokens were added to the Tokenizer for the EC number labels so that they could be concatenated with the sequences and inputted into the model when necessary. The final tokenized dataset was made up of four arrays: The input_ids, which holds the tokenized sequences, the token_type_ids, which tells the algorithm the token type of each of the elements in the input_ids array, the attention mask, indicating the input segments to be masked during training and finally, the labels, which is an array with each element representing the one hot encoded version of the corresponding target label.

Training

After data preprocessing, comes the training step which involves teaching the algorithm the overall patterns within the enzyme sequences and the global and local relationships between the amino acids which determine which type of chemical reactions they catalyze as given in the EC numbers. A block diagram illustrating the training step is given in Figure 3.2. The model consists of 4 ProtBert-like blocks (Prot_ECX), where each of which specializes in one stage of the EC numbers. Each of the blocks was modified by adding a classification header at the end where the number of nodes in the header was made to match the number of unique EC numbers present in each stage. Instead of training the blocks from scratch, the weights were initialized with pretrained values from the Huggingface repository where the model was trained on 2,122 million proteins from the BFD100 dataset which helps the model understand the general structure of proteins before they are trained on enzyme sequences.

$$Loss = -\frac{1}{N}\sum_{i}^{N}\sum_{j}^{M} y_{ij} log(\hat{y}_{ij}) \tag{3.1}$$

Each ProtBert block was then trained (finetuned) on the enzyme sequences for 10 epochs with a gradient accumulated step of 32 on a single Nvidia RTX 2080Ti GPU taking around 40 hours for the training process to complete. This is much less compared to our main benchmark model (Proteinfer [71]) which requires 8 NVIDIA P100 GPUs running for 60 hours. No dropout was used as the model did not show signs of overfitting with mean pooler used as the pooler type. A multilabel cross-entropy loss function was used to calculate the training loss at each step which is given in equation 3.1. Here, N stands for the total number of training samples, M represents the number of classes for a particular EC block, y is the true label, and $\hat{y}$ is the predicted label. During training, the positional and non-positional embeddings, and the attention layers were frozen, while

the normalization layers, the pooler layers, and the classification headers were allowed to be fine-tuned. This ensured stability as the attention layers were already pretrained to hold the structural information of proteins. The initial learning rate was selected to be 5e-4. However, this does not have any significant impact on the results as the learning rate was dynamically adjusted in each step of the training process.

The internal architecture of each Prot_ECX block is given in detail in Figure 3.1. The input sequences are first fed to the tokenizer which tokenizes each amino acid into latent representation which is understood by the transformer algorithm. Next, the Positional and non-positional embeddings are generated and all the embeddings are summed together and inputted into the Attention layer which consists of multi-head attention blocks followed by the normalization layer. This outputs the amino acid embeddings which are then pooled together into the protein embeddings. Finally, the flow ends with a classification layer that predicts the Enzyme commission number.

Prot_EC, ProtBert, and typical BERT style models embed the input using three vectors; token embeddings, positional embeddings, and segment embeddings. Token embeddings are used to create a numeric representation of the sequence via taking each token (in this case each amino acid) and replacing it with a learnable vector. The order in which tokens appear in biological or natural language sequences is another important piece of information that should be included in the embedding. Positional embedding captures this order by using a learnable vector conditioned on the position of the token and adding it to the vector embedding. As a result of this addition, embedding vectors for tokens that are close together in the sequence are pushed in similar directions in the embedding space. The last embedding vector, i.e. segment embedding, is a remnant of the next sentence prediction function of BERT and not in use in ProtBERT-BFD. In this work, token and segment embeddings are referred to as non-positional embeddings. All the embeddings are then added together and pushed into the attention layers.

Table 3.3: Accuracy at different layer combinations which was used to determine which embedding layers to finetune during the training process.

EC_num	Layer finetune combinations				
	[0,1,0]	[1,1,0]	[1,0,0]	[0,0,0]	[0,1,1]
EC1	97.33	**97.92**	97.58	97.79	97.31
EC2	95.55	**96.35**	96.11	95.30	95.63
EC3	95.38	**96.34**	95.92	95.11	95.61
EC4	89.20	89.68	**89.74**	87.42	88.55

To determine which layers to freeze during the finetuning process, five different combinations of freezing positional, non-positional, and pooler layers were created and the model was trained using each combination to find out the one giving the best accuracy. The experimental results are provided in Table 3.3. Here, the numbers in the square brackets illustrate which layers are frozen during the finetuning process. So, [1,1,0] represents the run where the positional and non-positional embeddings are frozen while the pooler layer is finetuned. On the other hand, the attention layers are always frozen as it would require very large computational resources and a very big dataset to train them. Whereas, the normalization layer and the classification layer are always finetuned as they are initialized from scratch and are task-specific.

Table 3.3 shows that finetuning the pooler layer while keeping the positional and non-positional embeddings frozen provides the best result. For each combination, the model was trained for 5 epochs on the random train dataset while the random development split was used to generate the Accuracy results. So the [1,1,0] combination was used for the final training process.

The first block named Prot_EC1 took the sequences as inputs with the first hierarchical stage of the EC numbers acting as the corresponding training target. On the other hand, the later blocks starting from Prot_EC2 took all the earlier stages of the EC numbers along with the sequence as inputs as illustrated in Figure 3.2. This was done by concatenating the previous stage EC numbers with the

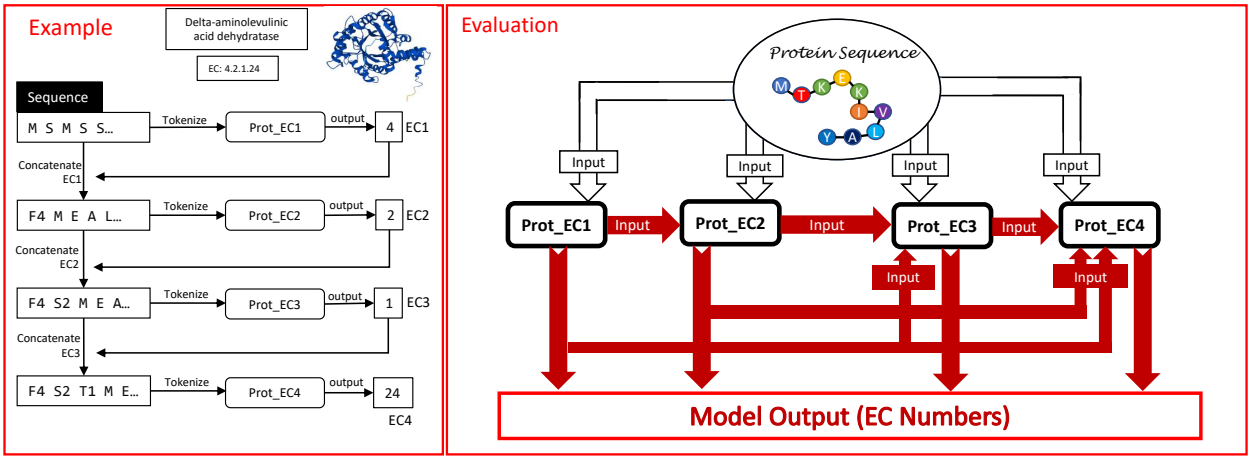

Figure 3.3: Right: Data Flow block diagram during Evaluation Stage. The filled arrows mean predicted output from a previous Prot_ECX block while the non-filled ones stand for input sequences. The prediction of one block is concatenated to the next EC block sequence. Left: An example of the data flow during the evaluation stage.

protein sequence before tokenizing them into embeddings. Special tokens were used to tokenize the EC numbers of each stage. All previous EC stages had to be inputted into the next stage as the EC numbers are hierarchical and the previous stage values are also required to determine the label of the current stage. An example of the data flow during the training process is also given in Figure 3.2 to illustrate the whole process.

Evaluation

The evaluation process of the model is provided in Figure 3.3. It starts by feeding the protein sequences as inputs to the Prot_EC1 module. This outputs the predicted EC1 label which is then concatenated to the sequence input and inserted into the Prot_EC2 block after tokenization. Next, the second block, Prot_EC2 uses this information to predict the second EC number. In the case of the third and fourth Prot_EC blocks, all the previous predicted EC numbers are propagated to the model in order to make the prediction. For example, for Prot_EC4, the predicted output of

Table 3.4: Performance comparison between Prot_EC and ProteInfer models for the Random and Clustered Split datasets.

Models	Parameters	Random split				Clustered split			
		EC1	EC2	EC3	EC4	EC1	EC2	EC3	EC4
ProteInfer	Accuracy(%)	97.69	97.32	97.10	95.61	89.93	88.02	86.73	82.23
	Precision	0.9857	0.9879	0.9902	0.9888	0.9680	0.9704	0.9770	0.9770
	Recall	0.9768	0.9715	0.9680	0.9504	0.8992	0.8778	0.8614	0.8094
	F1_score	0.9813	0.9796	0.9790	0.9692	0.9323	0.9218	0.9156	0.8853
	MCC	0.9644	0.9618	0.9617	0.9492	0.8772	0.8620	0.8569	0.8267
ProteInfer_EN	Accuracy(%)	97.71	97.43	97.23	95.75	89.42	87.53	86.51	82.48
	Precision	0.9918	**0.9927**	**0.9934**	**0.9905**	**0.9829**	**0.9828**	**0.9845**	**0.9829**
	Recall	0.9771	0.9733	0.9708	0.9544	0.8941	0.8738	0.8623	0.8205
	F1_score	0.9844	0.9829	0.9820	**0.9721**	0.9364	0.9251	0.9194	0.8944
	MCC	0.9705	0.9680	0.9671	0.9539	0.8869	0.8701	0.8647	0.8407
Prot_EC	Accuracy(%)	**99.32**	**99.11**	**99.03**	**97.16**	**97.88**	**96.79**	**96.64**	**91.20**
	Precision	**0.9932**	0.9910	0.9902	0.9718	0.9788	0.9678	0.9664	0.9133
	Recall	**0.9931**	**0.9910**	**0.9901**	**0.9717**	**0.9787**	**0.9677**	**0.9663**	**0.9132**
	F1_score	**0.9931**	**0.9910**	**0.9901**	0.9717	**0.9787**	**0.9677**	**0.9663**	**0.9132**
	MCC	**0.9913**	**0.9894**	**0.9862**	**0.9709**	**0.9728**	**0.9618**	**0.9521**	**0.9107**

Prot_EC1, Prot_EC2, and Prot_EC3 are concatenated with the protein sequences and inputted into the model to get the fourth EC number. Finally, the predicted outputs of all four Prot_EC models are gathered and stitched together to represent the full EC number prediction corresponding to the given test sequence. An example of the data flow during the evaluation process is also given in Figure 3.3 to illustrate the whole process.

Results

This section presents the results and analysis of the presented algorithm Prot_EC, along with comparisons with other models and methods.

Performance Comparison

The overall test accuracies along with the Precision, Recall, F1 scores, and MCC scores of Prot_EC, Proteinfer, and Proteinfer_EN on the different EC numbers on both the random split and clustered split datasets are provided in Table 3.4. The Prot_EC is the model proposed in this study whereas Proteinfer is a CNN-based protein annotation tool developed in [71] and Proteinfer_EN refers to an ensemble model consisting of multiple Proteinfer base models. On average, the accuracy scores of the models trained on the random split are greater than that of the clustered split. This is expected as it is more challenging with the training split and testing split samples derived from different clusters making them dissimilar in structure. Other trends in the data show that the accuracies fall for successive EC numbers with EC1 performing the best and EC4 showing the worst results. This happens as the number of classes increases for successive EC numbers with EC1 only having 7 classes whereas, EC4 has 402 classes. Also contributing to this trend is the fact that the EC numbers are hierarchical so any errors in the previous EC numbers are propagated to the later ones making the prediction of later values more difficult.

When comparing the different models, it is seen that Proteinfer has the lowest accuracy scores for all the EC numbers in RN split and only EC4 in CS split while the ensemble model Proteinfer_EN has the worst scores for the first three EC numbers in CS split. On the other hand, Prot_EC is seen to outperform the other benchmark models in all the EC numbers in both the dataset splits with the difference margins being much greater for the clustered split compared to the random. This

Table 3.5: Accuracy and F1 score of Proteinfer and Prot_EC models with Train and Test split of different similarity index. Here, CDHit was used to make the training and test sets of various similarities. The results show Prot_EC achieves better accuracy and F1_scores in all the different similarity values.

Models	Similarity	Accuracy				F1_score			
		EC1	EC2	EC3	EC4	EC1	EC2	EC3	EC4
ProteInfer_EN	0.9	95.63	95.09	94.68	91.77	0.9704	0.9674	0.9654	0.9459
	0.8	93.99	93.23	92.66	88.74	0.9593	0.9549	0.9521	0.9261
	0.7	91.68	90.67	89.87	84.64	0.9433	0.9374	0.9334	0.8985
	0.6	87.79	86.24	85.00	77.50	0.9160	0.9066	0.8999	0.8487
	0.5	80.79	78.31	76.23	64.60	0.8647	0.8495	0.8380	0.7490
	0.4	70.53	66.64	63.51	47.05	0.7823	0.7562	0.7360	0.6018
Prot_EC	0.9	98.77	98.44	98.20	94.83	0.9876	0.9843	0.9819	0.9481
	0.8	98.32	97.89	97.60	93.25	0.9830	0.9787	0.9759	0.9324
	0.7	97.77	97.16	96.75	91.17	0.9776	0.9715	0.9675	0.9116
	0.6	96.84	95.84	95.26	87.26	0.9682	0.9582	0.9524	0.8725
	0.5	95.10	93.42	92.63	80.64	0.9509	0.9342	0.9262	0.8062
	0.4	92.24	89.99	87.98	69.92	0.9223	0.9000	0.8799	0.6992

shows that Prot_EC is better at understanding the detailed patterns within the enzyme sequences and the relationships between amino acids. Also, Transformers being primarily sequence-based algorithms gives them an edge over other types of deep learning models when it comes to working on very long sequences.

The F1_scores and Matthews Correlation Coefficient (MCC) scores follow a similar trend to the accuracy values with the random split results generally being higher for a model compared to that for the clustered split. The ProteInfer model shows the worst F1_scores in general, followed by the ensemble Proteinfer model, while Prot_EC gives a better score for most of the EC numbers on both the splits. The only exception is the fourth EC number for the random split, where the

Table 3.6: Performance metrics of different models on the DEEPre New dataset with five-fold cross-validation

Models	EC1					EC2				
	Accuracy	Kappa Score	Precision	Recall	F1_score	Accuracy	Kappa Score	Precision	Recall	F1_score
DEEPre	94.44	0.926	0.947	0.912	0.928	92.95	0.914	0.785	0.756	0.766
EzyPred	90.31	0.870	0.905	0.870	0.885	87.85	0.841	0.790	0.709	0.737
SVM-PsePSSM	83.34	0.769	0.913	0.753	0.812	84.99	0.795	**0.831**	0.677	0.732
NN-Raw	88.07	0.840	0.830	0.807	0.817	89.30	0.861	0.661	0.615	0.629
Prot_EC	**95.75**	**0.942**	**0.954**	**0.942**	**0.948**	**93.15**	**0.919**	0.821	**0.822**	**0.818**

ProteInfer_EN has a slight advantage over Prot_EC in terms of the F1_score. However, this is not seen for the clustered split, where Prot_EC shows better F1_scores for all the EC numbers in both the dataset splits. Also, the trend of Prot_EC having better margins when it comes to the clustered split compared to the random split is seen for MCC and the F1_scores as well and confirms that Prot_EC shows better results compared to the benchmark models in terms of performance.

When we look at the precision and recall scores, it is seen that the benchmark models have a much higher precision score compared to the recall scores whereas, in the case of Prot_EC, the values are nearly equal for all the EC numbers across both the dataset splits. So, most of the predicted results of the benchmark models were true labels but it also did not pick up many of the actual positives in the datasets. The recall values could be buffed up by increasing the detection threshold of Proteinfer, but this again would decrease the precision value, so eventually, there would be minimal impact on the F1_scores. Moreover, the threshold value acts as an extra hyperparameter, and the need to tune it puts an extra burden on the end user decreasing the user-friendliness of the tool. Also, the threshold parameter depends on the test dataset and its optimum value for each case is unknown to a user with an unlabeled test set. Prot_EC, on the other hand, does not require any extra threshold tuning to balance out the precision and recall values.

The test the performance comparison of Prot_EC and Proteinfer on non-homologous protein sequences, several different test sets were created with the help of CDHit [89, 90]. To prepare this, the training set was kept the same as in the random split dataset while the test set was curtailed to include only those sequences that are similar to the training set sequences by no more than a certain threshold. Six test sets were created with thresholds ranging from 90% to 40%. Here, a dataset made with a threshold of 40% similarity means that there are no sequences between the training and test sets that are more than 40% similar to each other. Table 3.5 shows the accuracy and F1 score of Prot_EC and Proteinfer on the various test sets created. The results show similar trends in both accuracy and F1 score where prot_EC is able to hold out its performance much better compared to Proteinfer as the maximum allowed similarity is decreased from 90% to 40% with a large gap in performance between the two models occurring at the 40% threshold mark. This shows that Prot_EC is successful in predicting EC numbers even with very dissimilar training and test sets which indicates its ability to determine deep patterns within the protein sequences.

Next, the performance of Prot_EC was compared with some other popular deeplearning EC prediction models. Table 3.6 shows the Accuracy, Cohen's Kappa score, Precision, Recall, and F1 scores on the New dataset created in DEEPre [50]. It has 22,168 protein sequences that were collected from UniProt SwissProt (released on 9/7/2016) database followed by a 40% similarity threshold sweep with CDHit [89] to only keep the low-homology enzyme sequences. A five-fold cross-validation method was used to evaluate the model performance. The results show Prot_EC is able to outperform the other models in most of the metrics. However, the margin of performance increase is lower compared to other datasets due to the smaller size of this dataset. Comparison for EC3 and EC4 classifications could not be made as only EC1 and EC2 results were available for these models.

Table 3.7 shows the cross-dataset evaluation of Prot_EC compared to the other models on the primary EC number (EC1). Here, the models were trained on the DEEPre New dataset but were

Table 3.7: Evaluation of primary EC number(EC1) predictions of different models on the Cofactor dataset.

Models	Accuracy	Kappa Score	Presicion	Recall	F1_score
DEEPre	85.52	0.797	0.869	0.867	0.865
EFICAz	80.46	0.757	0.841	0.729	0.780
EzyPred	70.76	0.644	0.804	0.643	0.710
SVM-Prot	71.14	0.313	0.656	0.331	0.434
Prot_EC	**89.39**	**0.860**	**0.907**	**0.905**	**0.906**

tested on the cofactor dataset which was an independent, non-homologous, and non-overlapping dataset collected from a separate source(PDB). Details of how the dataset was created could be found in DEEPre [50]. The results show Prot_EC outperforming the other models with a better margin as compared to Table 3.6. So, Prot_EC is better at predicting EC numbers where the train and test sets are more dissimilar from each other indicating that it's better at identifying patterns within the protein sequences.

Finally, Table 3.8 shows the comparison of Prot_EC with other models on the ECPred [45] Held out dataset. The training set was extracted from the UniProt SwissProt (release:2017_3) database while the test set consists of Enzymes that were annotated in later versions of Uniprot. This made the test dataset very challenging as it contained very little similarity with the training set sequences. The results show Prot_EC performing significantly better than DEEPre and ECPred as well as the other models. This further proves that Prot_EC is better at predicting EC numbers when the training and test sets sequences are more dissimilar from one another. The substrate level EC number (EC4) comparison has not been made here as the test set contained too few EC4 labels to evaluate reliable performance metrics.

Table 3.8: Comparison of Prot_EC with other models on the ECPred Held out dataset.

EC_num	Models	Precision	Recall	F1_score
EC1	ProtFun	0.15	0.10	0.12
	EzyPred	0.16	0.13	0.15
	EFICAz	0.69	0.30	0.42
	DEEPre	0.67	0.40	0.50
	ECPred-wne	0.34	0.48	0.40
	ECPred	0.54	0.43	0.48
	Prot_EC	**0.89**	**0.87**	**0.88**
EC2	EzyPred	0.13	0.10	0.11
	EFICAz	0.33	0.07	0.11
	DEEPre	0.07	0.25	0.11
	ECPred	0.35	0.20	0.26
	Prot_EC	**0.45**	**0.55**	**0.49**
EC3	DEEPre	0.14	0.03	0.05
	ECPred	0.31	0.17	0.22
	Prot_EC	**0.39**	**0.43**	**0.41**

Confusion Matrix

Figure 3.4 shows the confusion matrices of the first three EC_num classifiers on both the random and clustered splits. It shows the number of samples in each of the classes and how they were predicted by our model. The diagonal elements represent the number of test samples in each class that was correctly classified whereas, the off-diagonal elements were the wrongly predicted samples. The concentration on the diagonal elements is much more prominent compared to the rest for both the random and clustered splits confirming the high accuracy of the model. Also, the numbers on the diagonals for CT split are bigger than that of RN split even though the accuracy of RN is greater. This happens because the total size of the test set of CT is much larger than that of RN.

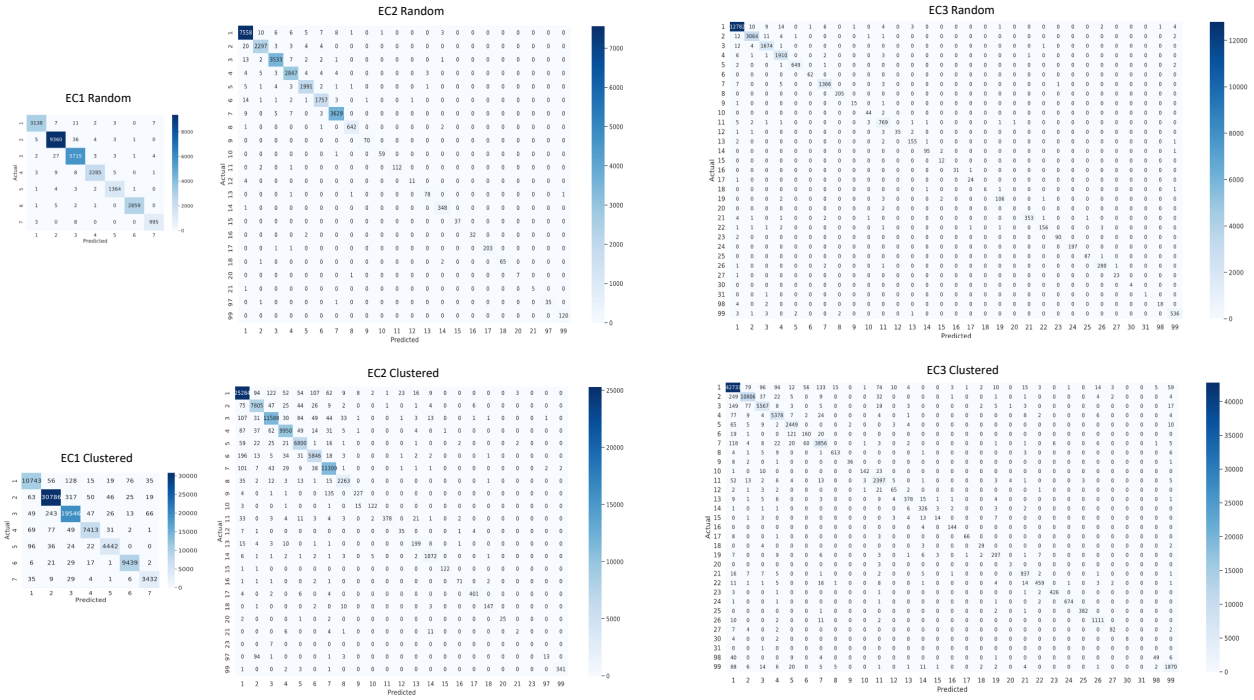

Figure 3.4: Confusion Matrices for EC1, EC2 & EC3 classifiers on both dataset splits.

The matrices show similar trends for the misclassified samples in both splits. For example, in EC1_random, several samples belonging to class 3 were misclassified as class 2 and vice versa, and the same trend is also seen for the clustered slit EC1 matrix. These similarities in trends are also present for the second and third EC number classifiers. This suggests that many of the classes are similar to each other and are difficult to differentiate during inference.

ROC curves and AUC

Next, the ROC curves for the Prot_EC model trained on the random and clustered splits are shown in Figures 3.5, 3.6, and 3.7. The curves show how well the model is able to separate the positive and negative labels at different threshold values. In an ideal case, the curve would take the form of an inverted L with the entire area falling under the curve and this denotes a 100% separability

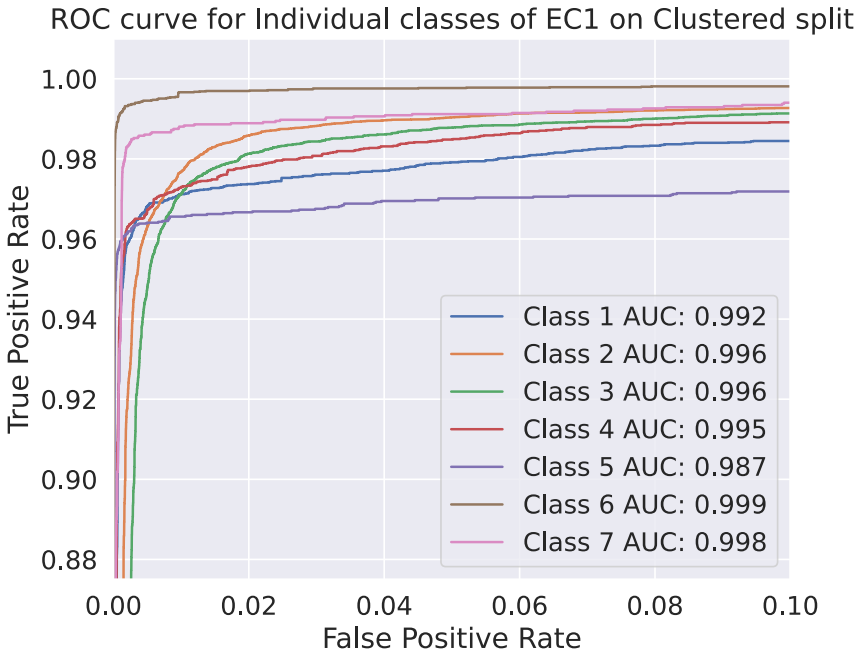

Figure 3.5: ROC curves of various classifiers in Prot_EC for the random and clustered datasets.

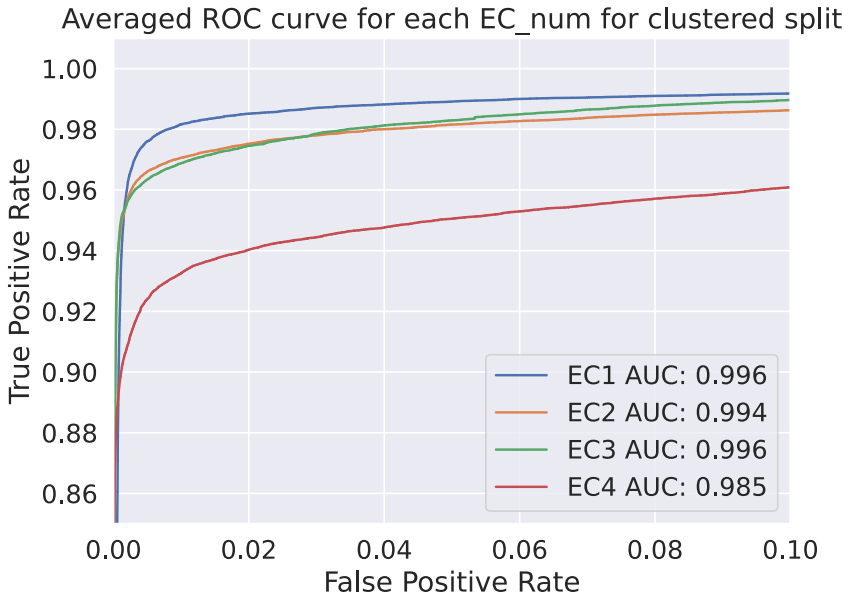

Figure 3.6: ROC curves of various classifiers in Prot_EC for the random and clustered datasets.

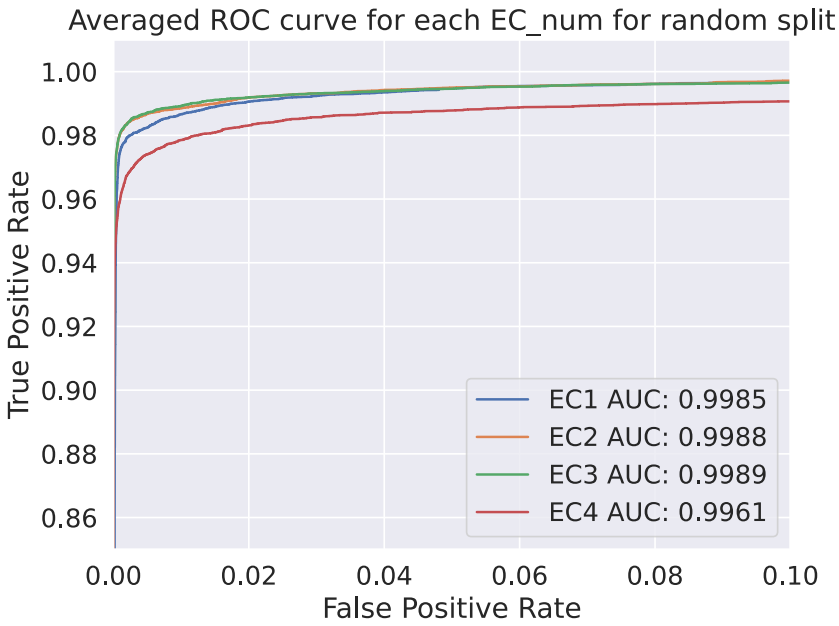

Figure 3.7: ROC curves of various classifiers in Prot_EC for the random and clustered datasets.

between the positive and negative samples. Figure 3.5 shows the individual class separability of the first EC number for the clustered split along with the area under the curve (AUC) for each class. This shows that it is easier to separate some classes compared to others. Another factor contributing to this is the size of the training sample for each class. It is seen that classes that have more samples in the training set tend to do better in the ROC curve compared to the ones with lower training data. For example, 36% of the training samples belong to class 2 and it shows a very prominent ROC curve whereas, only 5% belong to class 5 and is reflected by its worse shape and lower AUC score.

Figure 4.6 and Figure 4.7 show the ROC curves and AUC scores for the different EC number classifiers in the case of the clustered split and the random split datasets respectively. For the random split, the first three EC_num classifiers show very high AUC values along with sharp ROC curves while the classifier for the fourth EC number has a slightly lower score. A similar trend is

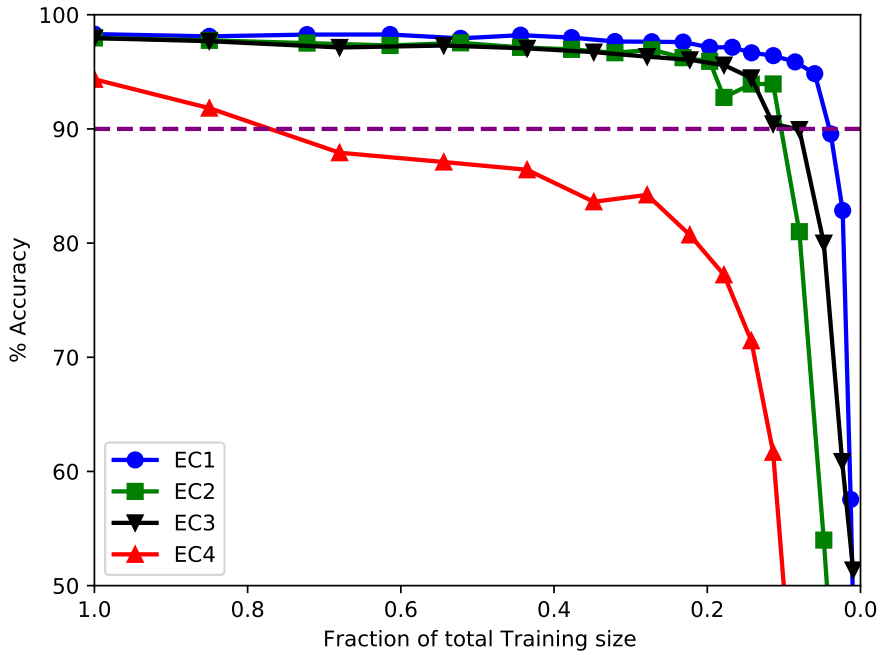

Figure 3.8: Test Accuracies of Prot_EC at different Training set sizes derived from the random dataset.

also seen for the clustered split with the EC1 classifier showing very high AUC scores followed by the second and third EC number classifiers with EC4 having the lowest score. However, in the clustered split, EC4 is trailing behind the others with a larger margin compared to the random split. This could be explained by the fact that the clustered split dataset has a lower training size than that of the random split and this combined with a large number of classes in EC4 caused some of the classes to have a very low number of training data eventually adding to the difficulty in differentiating between these classes.

STS Analysis

The shrinking training set (STS) study analyzes the impact of reducing the size of the training set on the test accuracy of the model. Figure 3.8 shows the test accuracies of the EC_num models

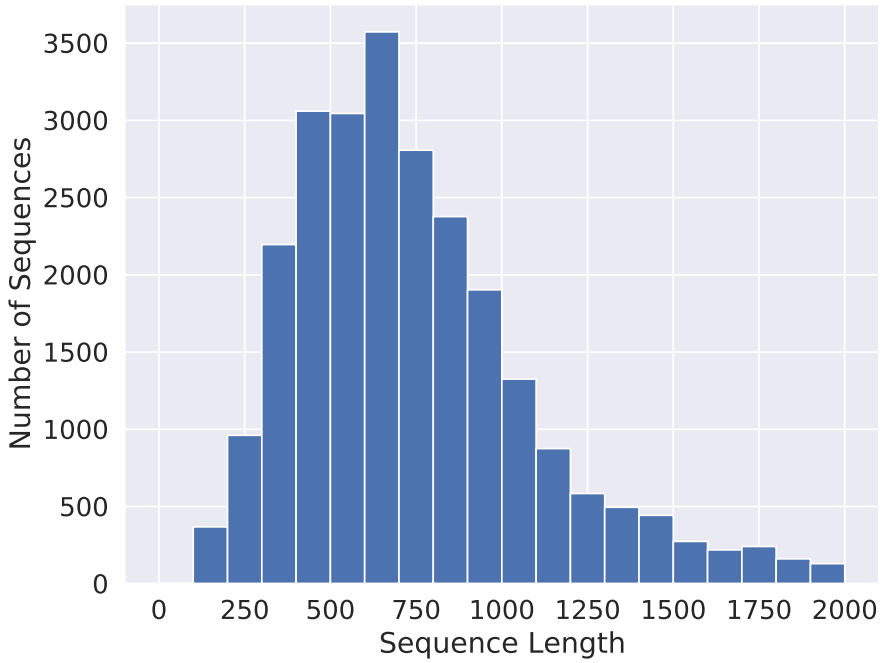

Figure 3.9: Distribution of sequence Length in the Random split test set.

with different training sizes on the random split. The shrinking training sets are created by splitting the training data in a stratified manner into two parts, where one part is kept and the other one is discarded. The model is then trained on the modified training data for 3 epochs and the splitting step is repeated to get the next training set. The results show the algorithm is very resilient to shrinking training set size. The first three EC_num classifiers are able to hold comparable accuracy scores with only 10% of the total training size while the fourth one maintains a value above 85% with the training set reduced to half its original number. The lower performance of EC4 compared to the other three EC numbers could again be contributed to its large number of classes. This causes some of the classes to end up with a very limited number of samples diminishing the model's ability to correctly classify them.

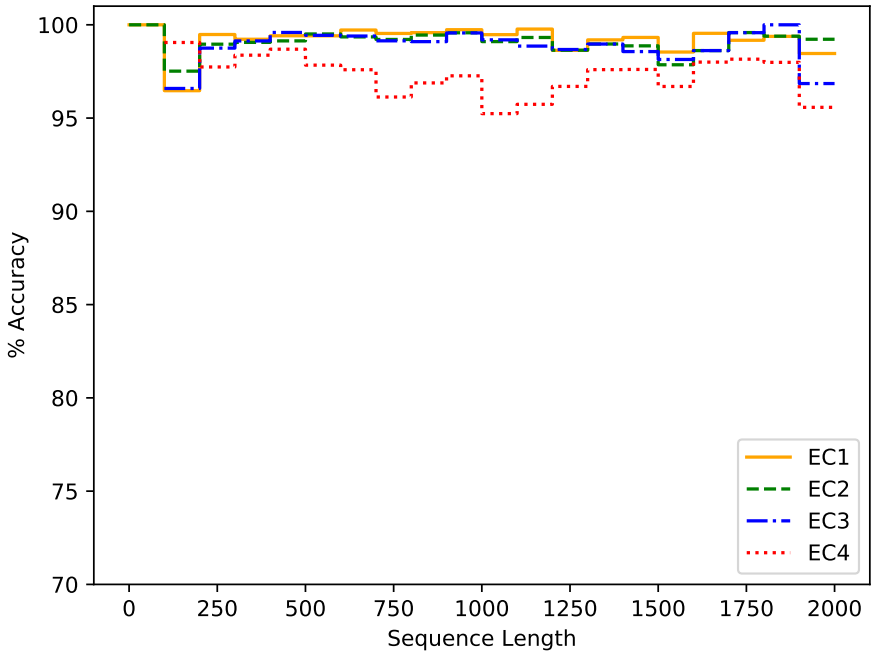

Figure 3.10: Average test accuracy of Prot_EC for the random split at different sequence lengths.

Sequence Length Statistics

Figure 3.9 shows the distribution of sequence lengths of enzymes for the random split test set up to 2000. Similar distribution could also be seen for the training and development sets and also for the whole dataset in general. There were a few samples that go beyond 2000, but their frequency keeps getting smaller as the sequence length increases so they are not included in the histogram. The frequency of samples seems to peak at around 750 units and the value diminished at a near-exponential rate making the bulk of the samples fall below a sequence length of 1500.

Next, the average test accuracy for the random split is plotted against sequence length in Figure 3.10 to show that the model's performance is not dependent on the size of the enzymes. The average accuracy is calculated by summing over all the correctly predicted samples in a range spanning every 100 units and dividing it by the total frequency of samples in that range. The

graph shows steady accuracies up to a length of 1000, after which it showed slightly degraded performance. This occurs as the model was trained with sequences that were truncated to 1000 elements due to limitations on hardware availability in hand. However, the model does not have any limitations in terms of sequence size and could accommodate the training of sequences of any length. As a result, users who have access to more powerful GPUs could retrain the model on non-truncated sequences to get ever better performance.

Project Discussion and Conclusion

This section contains discussions relating to various issues and technical information regarding the study. During the training phase of the model, the training loss and evaluation loss were monitored in every epoch as the model was trained. Both losses followed similar trends and became saturated at around 10 epochs. The successive eval accuracies remained constant and did not decrease indicating that the model had enough countermeasures to stop overfitting so, the number of training epochs in this model is not a sensitive hyperparameter that needs to be carefully tuned making the whole process more robust.

There are a small number of Enzymes in the dataset that were associated with more than one set of EC numbers. However, as this number was very small (2.54%), the first set of EC numbers was extracted and used as the corresponding label to train and evaluate our model. The Proteinfer benchmark model on the other hand outputted multiple sets of EC numbers on some occasions. In these cases, the model is taken to have made the correct prediction even if one of its outputted EC sets matched any of the sets in the true labels. This makes sure that the accuracies of the benchmark models are not underestimated in any case.

The model was trained on sequences that were truncated to a length of 1000 due to the limitations

in the available computational hardware. However, the model is able to operate with sequences of any length without any modifications to its architecture and could therefore be trained to obtain even better accuracy and performance if better computational hardware is available.

A transformer-based deep learning model named Prot_EC is proposed in this work which shows better performance in terms of accuracy and F1_scores compared to other full sequence deep learning models [71], especially for the clustered split. The model provides automatic hyperparameter tuning with balanced precision and recall values making it easy to use. Prot_EC is also able to retain its performance even with a fraction of its total training size and its accuracy is independent of sequence length making it suitable for enzymes of any length.

The results show that Prot_EC is able to perform very well with low homology train and test splits compared to other models. This part could be repurposed and used to predict other annotations in the protein universe with minimal modifications to the overall model structure. This makes it suitable to be expanded to predict multiple annotations using a single large model adding to the overall usage of the algorithm and is a potential and exciting area of future work.

CHAPTER 4: KITE-DDI: A KNOWLEDGE GRAPH INTEGRATED TRANSFORMER MODEL FOR ACCURATELY PREDICTING DRUG-DRUG INTERACTION EVENTS FROM DRUG SMILES AND BIOMEDICAL KNOWLEDGE GRAPH

It is a common practice in modern medicine to prescribe multiple medications simultaneously to treat diseases. However, these medications could have adverse reactions between them, known as Drug-Drug Interactions (DDI), which have the potential to cause significant bodily injury and could even be fatal. Hence, it is essential to identify all the DDI events before prescribing multiple drugs to a patient. Most contemporary research for predicting DDI events relies on either information from Biomedical Knowledge graphs (KG) or drug SMILES, with very few managing to merge data from both to make predictions. While others use heuristic algorithms to extract features from SMILES and KGs, which are then fed into a Deep Learning framework to generate output. In this study[1], we implemented a KG-integrated Transformer architecture to generate an end-to-end fully automated Machine Learning pipeline for predicting DDI events with high accuracy. The algorithm takes full-scale molecular SMILES sequences of a pair of drugs and a biomedical KG as input and predicts the interaction between the two drugs with high precision. The results show superior performance in two different benchmark datasets compared to existing state-of-the-art models especially when the test and training sets contain distinct sets of drug molecules. This demonstrates the strong generalization of the proposed model, indicating its potential for DDI event prediction for newly developed drugs. The model does not depend on heuristic models for generating embeddings and has a minimal number of hyperparameters, making it easy to use while demonstrating outstanding performance in low-data scenarios.

[1]Part of the materials in this chapter is submitted in IEEE Access

Related Works

Polypharmacy, the routine prescription of several medications concurrently, has become a prevalent practice in contemporary medicine [91]. This technique of treating patients with multiple medications has become more common with the rapid increase of the number of approved drugs commercially available for consumption, especially to elderly individuals or patients suffering from long-lasting medical ailments like diabetes, Cancer, chronic heart conditions, etc. When multiple drugs are taken simultaneously in clinical practice, it can make the treatment process more complicated and potentially lead to unexpected interactions between the drugs, which is known as Drug-Drug Interaction (DDI). In some cases, these interactions can even be fetal [92].

As a result, a physician needs to examine the DDI events between the newly prescribed drugs and all other drugs currently taken by the patient to make sure that no adverse reactions are occurring between them. This, in turn, makes it necessary to maintain comprehensive DDI databases of all approved drugs currently in circulation and also to investigate the interactions between newly developed drugs with all other already approved drugs. The current method of determining new DDI events is via wet lab testing by domain experts. This is a very time-consuming and resource-intensive process and one of the major steps in a drug approval pipeline, resulting in the elongation of the overall system and also making new drugs more expensive to the general public.

To mitigate this, there have been a lot of studies focused on computational methods in order to construct algorithmic models for predicting DDI events. These computational methods have shown comparable performance to traditional wet assays based in vivo and in vitro trials while requiring much less time and resources. Recent efforts towards DDI research can be categorized into four main groups: literature extraction-based, matrix factorization-based, ensemble learning-based, and network-based.

The literature extraction-based studies are concerned with implementing natural language processing (NLP) techniques and machine learning models to extract drug-drug interactions from the biomedical literature. Given the large number of research articles and publications that are continually being published, there is a significant amount of drug knowledge that could be extracted from these materials [93, 94, 95]. Hence, numerous deep learning-based NLP algorithms have been developed that could extract and identify drug-drug interactions (DDI) from textual data. Recent examples of such study include Sun et al. [96], who devised a deep convolutional neural network based model called DCNN, which utilized multiple layers and small convolutions, to extract drug-drug interactions (DDIs). In a subsequent study [97], the same team developed a Recurrent Hybrid Convolutional Neural Network model known as RHCNN, which incorporated both canonical convolution and dilated convolution to predict DDI events with good accuracy. Nevertheless, conventional CNNs are unable to address the issue of lengthy sentences as they solely focus on neighboring words and disregard long-term dependencies and deep patterns within the textual data. In an attempt to solve this issue, Kavuluru et al. [98] put forward a character-level Recurrent Neural Network model known as char-RNN as a means to tackle the long-term word dependency in the DDI extraction task. Although these approaches have shown promise, the task often necessitates meticulous human annotations, which can be quite time-consuming and resource-intensive.

The second group of studies concerning DDI research uses matrix representation and adjacency matrix to analyze the relationship between the interacting drugs in a DDI event. Much of the recent research in this area has used a technique known as Neural Collaborative Filtering which was first introduced by Xiangnan et al [99]. Here, they have improved the simple linear inner product operation which is used in case of traditional matrix factorization (MF) systems and replaced it with a neural network-based collaborative filtering system which can successfully extract and analyze the complex structure present in the network data. Some examples of recent scholarly works in this field include Yu et al. [100], who have approached the identification of potential side

effects of drugs as a matrix reconstruction task, employing a linear neural network layer. Other related works consist of a study by Shi et al. [101] who have introduced a model called TMFUF, which is based on triple matrix factorization to generate relationships between the different drugs in the dataset, several novel matrix factorization-based models were developed which is based on manifold learning and neural networks [102]. In addition, Zhu et al. [103] developed a machine learning based network to learn representations of drug dependency and introduced a supervised learning technique called probabilistic dependent matrix tri-factorization (PDMTF) to predict adverse drug-drug interactions (ADDI). Furthermore, Yu et al. [104] proposed a novel algorithm called DDINMF, leveraging the principles of semi-nonnegative matrix factorization. Lastly, Zhang et al. [105] put forward the manifold regularized matrix factorization method to predict DDI events with high accuracy.

The third group of research in this area consists of ensemble learning-based methods which combine multiple different algorithms called sub-models to generate the final output. The different sub-models are responsible for understanding various relationships between the input data, enabling it to generate better results when all the sub-models are put together compared to their individual performance. Some notable studies in this area include a semi-supervised deep learning architecture proposed by Deepika et al. [106], who have used network representation learning and meta-learning on four different drug datasets to make DDI predictions using an ensemble architecture, other studies include Zhang et al. [107], who introduced another ensemble learning based model called SFLLN where they have applied linear neighborhood regularization techniques for predicting DDI events, Zhu et al. [108], who implemented unified multi-attribute discriminative representation learning (MADRL) to device an ensemble model for Adverse DDI prediction which can extract both intrinsic and extrinsic features and connections between pair of drug molecules. Finally, Schwarz et al. [109] proposed a Siamese multimodal neural network based model called AttentionDDI which uses the self-attention mechanism to combine various drug features like tar-

gets, pathways, gene expression profiles, etc into a latent vector that is then used to predict DDI events.

The fourth and last research area includes network-based approaches where, the algorithms extract information from graphs and networks describing drug data. Many contemporary studies employ a cheminformatics library known as RDKit [110] to convert drug SMILES (Simplified Molecular Input Line Entry System) into molecular graphs that represent the chemical structure of the drug compounds using heuristic algorithms. RDkit could also be used to extract features from drug compounds with the help of domain expertise and available literature. Notable contemporary research in this area includes the work by Hu et al. [111] who focused on the development of different molecular learning techniques using pre-trained GNNs. They aimed to extract the chemical structure embedding of drug molecules. Chen et al. [112] utilized a pretrained message-passing neural network to create structural representations of drugs. The input data for this network consisted solely of the number and chirality of atoms, as well as the type and orientation of bonds. Nyamabo et al. [113] developed a gated message-passing neural network (GMPNN-CS) to analyze the molecular graph representations of drugs. This network is capable of learning chemical substructures of varying sizes and shapes. The main objective of their research was to predict potential drug-drug interactions between pairs of drugs. Other works include GNN-MolGNet [114], which is a highly effective molecular graph model that has been extensively pre-trained at both the node and graph levels. This comprehensive training allows it to extract valuable chemical insights and generate easily understandable representations. Qian et al. [115] used feature similarity and feature selection techniques to construct a gradient boosting-based classifier in the rich biomedical network made up of several entities to expedite the process and get reliable prediction performance. Their goal was to enhance efficiency and ensure accurate prediction results. Next, the LR-GNN [116] paper introduced a propagation rule that captures the node embeddings of each GCN encoding layer to construct link representations (LRs). Further development was made with the

proposed GCNMK [117] model which used various mechanisms associated with different types of DDI. Here, two graph convolution kernels are created by expanding and reducing the associated DDI network to make accurate predictions.

Many of these models used a type of data known as the Knowledge Graph (KG) which is a powerful tool that allows for the representation of entity relationships. By integrating data from various sources, it creates a comprehensive biomedical information repository. Graph embedding methods, such as TransE [118] and ComplEx [119] could be utilized to analyze the information present with these graph data. These methods aim to learn dense vectors for both relations and nodes in a low-dimensional space. The resulting vectors are then utilized in downstream biomedical tasks [120, 121].

Many of the previous methods solely focus on predicting drug interactions without thoroughly examining the significance of these interactions or analyzing the type of interactions that are occurring in a specific situation and the impact they have on human metabolism. This brought about a rise in studies concentrating more on the type of interactions and their impact while predicting DDI events. Some of these include the work by Ryu et al. [122] who presented drug interactions as 86 metabolic events, which were described using human-readable sentences. The authors put forward a deep learning model called DeepDDI, which utilizes drug chemical structural information to accurately predict each DDI event. Furthermore, Deng et al. [102] put forward a multimodal deep learning framework called DDIMDL. This system effectively integrates various drug features to accurately predict 65 DDI-related events. Lin et al. [123] examined a dataset containing 100 drug-drug interaction (DDI) events sourced from the DrugBank database.

However, most of these studies use a method of splitting training and test set DDI events called the transductive setting [124]. Here, the training and test sets consist of common drug compounds so the algorithm could predict DDI events by analyzing relationships and patterns within

drug compounds with known action mechanisms. However, it is important to consider the inductive setting, also known as cold-start scenarios which most previous studies have overlooked [102, 113, 123, 125]. In this setting, all available drugs are first divided into test and training sets. Later, the DDI events that exist between drug pairs in each split are gathered to make up the datapoints in the training and test datasets. This was identified in the MSEDDI model [126], where a multi-classification DDI event prediction system was proposed which used the more challenging inductive dataset split technique to evaluate their results. It uses a multi-structural deep learning framework to combine information from multiple sources to achieve state-of-the-art performance for DDI prediction in the more challenging inductive setting and hence has been used as the primary benchmark method for performance comparison with the proposed model in this work. However, the MSEDDI model used various heuristic models that are based on domain expert knowledge to extract features from the SMILE notations of drug compounds to make DDI predictions. This makes it dependent on other models and cannot be implemented to generate an end-to-end pipeline that could input raw drug SMILES and output DDI classifications.

In this work, we have implemented an end-to-end fully machine learning based model that only takes raw drug SMILEs and a biological knowledge graph as input and predicts DDI events with high accuracy. The model is evaluated on two different datasets and shows better performance compared to state-of-the-art models, especially in the more challenging inductive setting. This indicates that the proposed architecture is better at extracting the latent chemical features buried in the drug SMILEs and can also understand the deep structure and patterns responsible within the biomedical knowledge graphs that is required for making highly accurate predictions in the inductive setting. The key contributions and novelty of this work are outlined below:

- Superior performance and accuracy compared to other related state-of-the-art models especially on predictions in the inductive dataset split setting showing good generalization.

- It is an end-to-end machine learning model with no heuristic components that rely on domain expert knowledge.
- The proposed model is lightweight, computationally inexpensive, and easy to use as it is an end-to-end pipeline requiring very few hyperparameter optimizations.
- The proposed algorithm only requires 2 inputs (KG and SMILES) as opposed to the main benchmark model which requires five (KG, SMILES, MPNN, AFP, WEAVE) [126]. These other inputs need to be generated using separate full-scale algorithms raising the dependency of the method on other models.
- The proposed architecture shows better performance at low data settings compared to the main benchmark method.

The model architecture and details about the dataset including preprocessing steps and evaluation criteria are given in the Methodology section. The experimental results, ablation study, and detailed analysis of the results are outlined in the Results section. Finally, further discussion, overall outcome, and future research direction are given in the Project Conclusion section.

Methodology

Dataset

Two different datasets have been prepared to evaluate the performance of KITE-DDI and compare it with other similar benchmark methods. The first dataset (Dataset 1) is prepared following the same process as Deng et al [102]. It consists of 572 drugs and 37,264 DDI events which were all sourced from the DrugBank repository [127]. Each DDI event corresponds to a drug pair and represents an increase or decrease in the interaction between the two drugs on the metabolism level

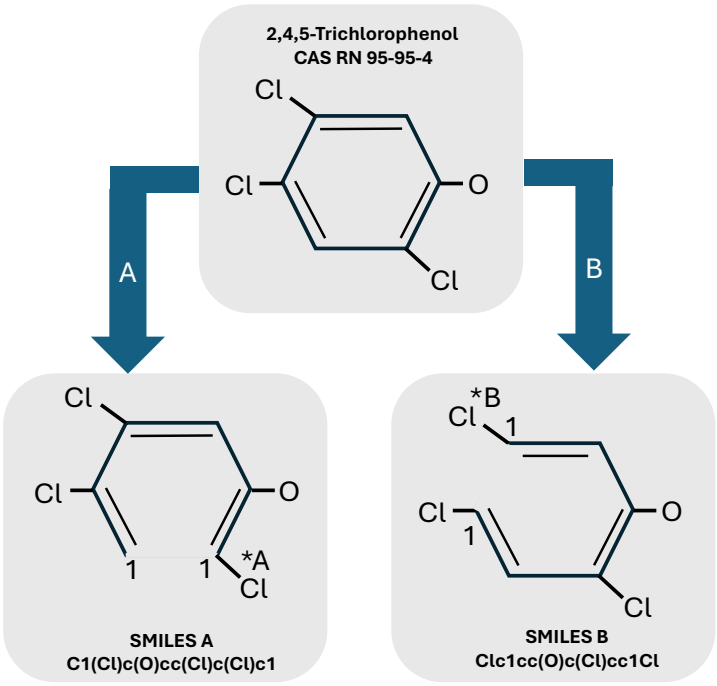

Figure 4.1: Multiple equally valid SMILES representation of a single compound.

of the human body totalling to 65 events. The second dataset (Dataset 2) is created analogously to the paper by Lin et al [123]. Here, 1258 drugs were collected from the DragBank repository [127] which consisted of 325,539 pairwise DDI event datapoints in total and comprised of 100 unique DDI classes.

Each data point in the dataset consists of a pair of drugs along with their Simplified Molecular Input Line Entry System (SMILES) notation which is a representation of the three-dimensional chemical structure of the drug molecule with the help of a string of symbols. The SMILE notation for representing chemical compounds is a well-accepted and practiced method for inputting drug compounds into computational algorithms and machine learning models [128]. However, there are certain limitations to the SMILE notation due to its conversion of three-dimensional chemical compounds into a linear two-dimensional format. One of the major ones related to this study

Table 4.1: Number of Datapoints in each split of Dataset 1 and 2

Splits	**Dataset 1**	**Dataset 2**
Training set	20404	156713
5-fold Cross-validation	5101	39187
U1 Split	10541	110828
U2 Split	1080	15679
Unique Drugs	572	1254

involves the representation of a single molecule by multiple equally valid but different SMILES notations. This happens in cyclic structures where different notations may arise depending on the starting position of the cyclic compound. This phenomenon is illustrated in Figure 4.1, which shows two different SMILE notations for the same cyclic molecule, “2,4,5-Trichlorophenol CAS RN 95-95-4". To resolve this problem, a randomizer augmentation is attached to the dataset module which feeds the data into the model. This randomizer generates a different but valid SMILES representation each time a drug is inserted into the model which teaches it about the existence and structure of the multiple representations of the same molecule.

Apart from the drug SMILES, the other input of the proposed algorithm is a Biomedical Knowledge graph called the “Drug Repurposing Knowledge Graph” (DRKG). This graph consists of information relating to genes, compounds, biological pathways, diseases, proteins, side effects, symptoms, etc. It consists of about a hundred thousand nodes divided into 13 entry types and around 6 million edges belonging to 107 types. Previous research suggests that the biomedical knowledge graphs contain vital information about DDI events which could be leveraged to achieve better performance in predicting drug interactions.

Both Datasets 1 and 2 are randomly divided into five equal parts to create the fivefold cross-validation. In this process, one fold is fixed as the validation set while the other four folds are

Table 4.2: Percentage composition of the most abundant DDI events in each Dataset and their description

Events	**Dataset 1 (%)**	**Dataset 2 (%)**
Drug A metabolism decreases when combined with Drug B	25.76	30.93
Risk or severity of adverse effects increases	25.30	10.89
Increase in serum concentration	15.80	5.12
Decrease in serum concentration	4.94	1.36
Therapeutic efficacy of Drug A decreases	3.48	3.98
Excretion decreases resulting in decrease of serum level	0.33	11.30
Drug A metabolism increases when combined with Drug B	1.92	8.16
Increase in the risk or severity of QTc prolongation	0.27	5.35

combined to create the training set. This step is repeated five times, changing the fold which is set as the validation, and the accuracy metrics averaged to evaluate the final results. These five-fold cross-validation results are used to measure the performance of the model in the transductive setting. To compute and compare the inductive performance of the model, two different dataset splits called U1 and U2 are also generated. In the case of U1, only one drug in each DDI pair in the test set is present in the training set while in the case of U2, both the drugs in each DDI pair in the test set are absent in the training set. As a result, the U1 dataset is more challenging compared to the fivefold cross-validation, while the U2 data split is the most challenging. The composition of training and testing DDI events in each dataset split is given in Table 4.1.

In addition to these two datasets, an additional unlabeled dataset has also been prepared to pretrain

the model in an unsupervised manner. The dataset consists of 17,741 drug SMILES sourced from the DurgBank [127] online repository and the ChEMBL database [129].

The composition of the most abundant DDI events in the two datasets is given in Table 4.2. It shows that these top four events account for about 75% and 66% in Dataset 1 and Dataset 2, respectively creating a long tail distribution with the least abundant events contributing to a small portion of the overall data leading to a skewed class representation.

Transformer in Drug Discovery

The transformer architecture was first introduced in the paper "Attention is all you need" [2]. It is a sequence-to-sequence model, meaning it takes a sequence as an input and outputs another. It started out as a natural language processing (NLP) algorithm but soon expanded to other tasks like computer vision, bioinformatics, and drug discovery. One of the major limitations of the transformer architecture is that it requires a large amount of data to be trained effectively and achieve optimal performance. One of the major components of the proposed algorithm in this work involves a transformer block but the architecture has been modified along with the training regime to adopt it for low data setting.

The transformer module used in this work is of the encoder-only variant. It consists of three encoders that take the tokenized input sequence and convert it into latent vectors. All the encoder modules consist of trainable weights which generate various features of the input sequence, that are then summed up to be fed into the encoder layers. The token embeddings contain information about the type of token that each sequence base consists of. The segment embeddings divide the input sequence into two segments for the pair of drug SMILES in each data point. These two embeddings are then combined to create the non-positional embedding vector. The final encoder is responsible for generating the positional embeddings, which also consist of learnable weights

containing information about the position of each token in the input sequence.

The encoder module is responsible for generating the representation of the input sequence in the form of a latent vector. It consists of attention heads which are the principal computational blocks in the transformer architecture followed by a normalization layer and a position-wise feedforward network before ending with another normalization layer. These layers make sure that the information flow does not become too large or too small which could potentially lead to the vanishing or exploding gradient problem. There is also a residual connection that bypasses the attention and feedforward network in each layer to make sure that the input information also reaches the later layers in the module. Multiple encoder layers are stacked on top of one another to create the overall architecture of the encoder module.

Self-Attention

The multiheaded attention layer from the transformer architecture could be used elsewhere with appropriate modifications. Self-attention is an effective and computationally efficient way of extracting long and short term patterns within a one-dimensional linear vector. This can be viewed as a fully connected layer where the weights are dynamically generated according to the pairwise input relationships. In this case, the input vector is copied three times and fed into the multiheaded attention mechanism as query, keys, and values. The output vector generated is the same shape as the input vector but contains information about the structure, dependencies, and patterns within the input. In the proposed model, the self-attention module is used to extract interaction features from the concatenated knowledge graph embeddings of the two drug compounds in the DDI event.

Table 4.3 shows the comparison of the complexity, sequential operations, and maximum path length between self-attention and three frequently employed layer types. The key benefits of the self-attention layer are outlined below [13]:

Table 4.3: Computational complexity of Self-Attention compared to other algorithms

Layer Type	**Complexity**	**Sequential Operations**	**Maximum Path length**
Self-Attention	O(n2 . d)	O(1)	O(1)
Recurrent	O(n . d2)	O(n)	O(n)
Convolutional	O(k . n . d2)	O(1)	O(logk (n))
Self-Attention (restricted)	O(r . n . d)	O(1)	O(n/r)

- The maximum path length of this model is similar to that of fully connected layers, which makes it ideal for computing long-range dependencies. In contrast to fully connected layers, this approach demonstrates greater efficiency in terms of parameters and excels in handling inputs of varying lengths.

- Convolutional layers possess a restricted receptive field, often necessitating a deep network stack to attain a comprehensive global pattern extraction. On the other hand, the self-attention's constant maximum path length enables it to effectively capture long-range dependencies across various layers.

- The consistent number of sequential operations and the maximum path length contribute to the parallelizability and effectiveness of self-attention in long-range modeling, surpassing that of recurrent layers.

It could be observed from Table 3 that the self-attention layer understands interactions between all tokens through a consistent number of sequential operations in contrast to a recurrent layer which requires O(n) sequential operations. When the sequence length is shorter than the representation dimensionality, self-attention layers tend to be faster than recurrent layers. This is particularly true for sentence representations used in advanced machine translation models, like word-piece and

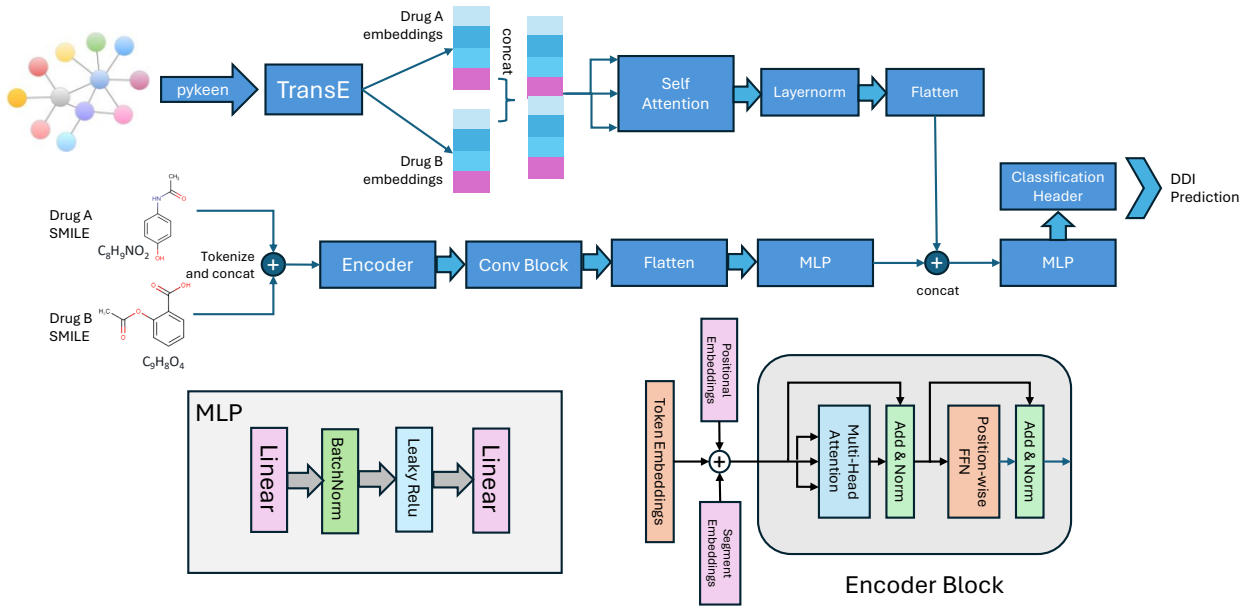

Figure 4.2: Overall architecture of the proposed KITE-DDI model. The internal layers of the MLP and encoder blocks are also illustrated in the figure.

byte-pair representations. In order to optimize computational performance for tasks that deal with lengthy sequences, it is possible to restrict self-attention to a specific neighborhood around each output position in the input sequence. This approach would result in an increase in the highest possible path length to O(n/r).

Model

The overall architecture of the proposed KITE-DDI model is illustrated in Figure 5.1. The model has two inputs; the first one is from a biomedical Knowledge Graph (KG) and the second is from a pair of Drug SMILEs which are involved in the DDI event. The Drug Repurposing Knowledge Graph (DRKG) is used as the biomedical KG, which is then fed into the Pykeen deep learning framework [130] to extract the entity features using the TransE [118] algorithm. These embeddings

for the two drug compounds are then concatenated into a one-dimensional vector of length 800 followed by a self-attention layer which extracts the information relating to the interaction between the two drug embeddings.

On the other side, the SMILE representation of the two Drug compounds in the DDI event is randomized, tokenized, and concatenated into a single one-dimensional sequence which is then padded or concatenated to a fixed predefined dimension of 500 tokens. The sequence then goes into the encoder module, made up of 6 encoder layers with 8 multiheaded attention blocks. A model dimensionality of 256 is picked which seems to maximize efficiency while keeping the computational complexity relatively low. The length of the feed-forward network is also kept equal to the model dimensionality at 256 to ensure stability. The output from the encoder layers are latent vectors of each token in the input sequence that are gathered into a multidimensional vector of shape 500 by 256. This latent vector then goes through a convolutional block to extract the features and shrink the dimension.

The structure of the convolutional module is illustrated in Figure 4.3. A single repeating block consists of a convolutional layer followed by a batch normalization layer, Relu activation, another convolutional layer, and a final batch normalization layer at the end. This block is then repeated 8 times with residual connections between each block. The extracted feature from this convolutional module is then flattened and fed into a Multilevel Perceptron (MLP) module consisting of a fully connected linear layer, batch normalization, leaky RELU activation function and another linear layer at the end. The output from the MLP is then concatenated with the KG embeddings and fed into another MLP module before outputting the output class predictions determined by a Softmax layer.

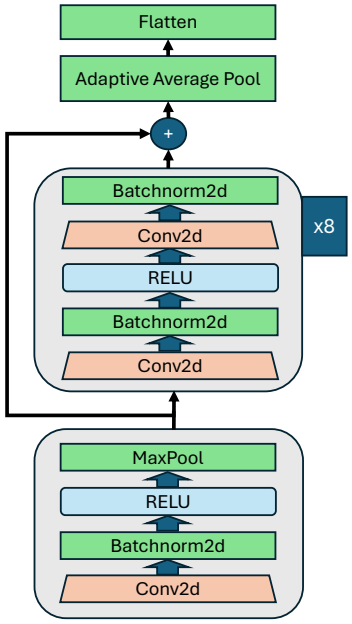

Figure 4.3: Block diagram of the convolutional module used in the proposed model.

Training

The first step of the training involves pretraining the transformer encoder so that it understands the detailed structure of the incoming SMILES representation of the drug compounds. This is done with the help of the pretraining dataset consisting of 17,741 drug SMILES. For the first step of the process, the drug SMILES are converted into pairwise DDI datapoints. The first drug in a datapoint is selected chronologically from the dataset and it is paired up with another randomly selected SMILE. The two drug SMILES are then tokenized and concatenated to form a single sequence. A vocabulary dictionary is created from the pretrained SMILES for the tokenization process which is later reused in the finetuning step as well. The input sequence is truncated if it exceeds the maximum allowed sequence length of 500 tokens and padded if it is shorter to make all of them the same length.

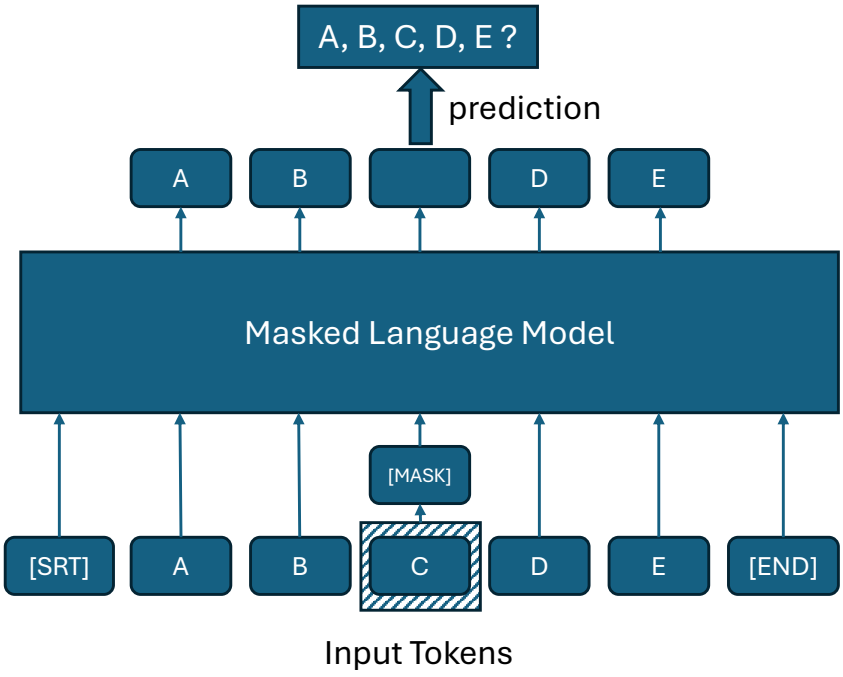

Figure 4.4: Masked Language Modelling Technique used for pretraining the Transformer encoder blocks.

A masked language modelling (MLM) technique is used to pretrain the transformer module in an unsupervised manner. Here, 15% of the tokens in the input sequence are randomly masked using a special masking token and the task of the training algorithm is to correctly predict the hidden bases. The illustration of the overall MLM process is given in Figure 4.4. A cross-entropy loss function given in Equation 4.1 is used to calculate the loss in each step of the training process and the Adam optimizer is used with a learning rate of 1e-5. The model is pretrained for a total of 700 epochs with a batch size of 8.

$$Loss = -\frac{1}{N}\sum_{i}^{N}\sum_{j}^{M} y_{ij} log(\hat{y}_{ij}) \tag{4.1}$$

After the pretraining is done, the learned weights for the encoder layer and the positional and non-

Table 4.4: Performance Comparison of proposed model with similar SOTA methods

Dataset	Model	U2 Split				U1 Split			
	Metrics	**Accuracy**	**F1 Score**	**AUPR**	**AUC**	**Accuracy**	**F1 Score**	**AUPR**	**AUC**
Dataset 1	**MSEDDI**	44.51%	0.1691	0.3999	0.9543	65.17%	0.4771	**0.6810**	**0.9823**
	DeepDDI	36.02%	0.1373	0.2781	0.9059	57.74%	0.3416	0.5594	0.9575
	Lee's method	40.97%	0.2022	0.3184	0.8302	64.05%	0.5039	0.6244	0.9247
	DDIMDL	40.75%	0.1590	0.3635	0.9512	64.15%	0.4460	0.6558	0.9799
	MSD-SA-DDI	43.78%	0.2326	0.3810	0.8675	64.59%	0.5471	0.6390	0.9435
	KITE-DDI	**51.29%**	**0.4698**	**0.4648**	**0.9649**	**67.21%**	**0.6628**	0.6727	0.9729
Dataset 2	**MSEDDI**	63.09%	0.3111	0.6596	0.9863	76.97%	0.6486	0.8315	0.9947
	DeepDDI	36.11%	0.1868	0.2820	0.9264	58.83%	0.4709	0.5851	0.9746
	Lee's Method	48.67%	0.3082	0.4349	0.9093	69.17%	0.5934	0.7119	0.9687
	DDIMDL	46.99%	0.3032	0.4386	0.9685	67.20%	0.5817	0.7086	0.9885
	MSD-SA-DDI	47.94%	0.2937	0.4450	0.9686	66.64%	0.5919	0.6820	0.9862
	KITE-DDI	**66.96%**	**0.6537**	**0.7197**	**0.9886**	**79.09%**	**0.7834**	**0.8528**	**0.9948**

positional embeddings are transferred to the main model which is then finetuned on the application datasets. The other modules apart from the transformer block are randomly initialized. The Cross-entropy loss function is used for the overall training along with an Adam optimizer with a learning rate of 5e-5 and a weight decay of 1e-5. The model is trained up to 100 epochs with a batch size of 32 on a single Nivida RTX 2070 Ti GPU requiring around 4 GB of memory and takes approximately 5 hours to train. The model is then evaluated on Dataset 1 and 2 on both the transductive and inductive settings to investigate the generalization and performance of the model compared to other recent state-of-the-art methods.

Result

This section presents the performance of the proposed algorithm in DDI event prediction along with comparison with other state of the art models and methods. The performance of the proposed method, KITE-DDI, compared to other state of the art models in four different evaluation metrics are given in Table 4.4. Accuracy, F1 score, AUPR, and AUC have been used as the different performance metrics. Here, the accuracy is given by the number of correctly labeled DDI events for a pair of drugs divided by the total number of samples. The F1 score is a combination of the precision and recall values for the predictions samples and is given in Equation 4.2. The AUPR denotes the area under the precision-recall curve. It is normalized to a maximum value of unity where a larger value corresponds to better performance. Lastly, the AUC computes the area under an ROC curve and, like the AUPR, ranges between 0 and 1.

$$F1\,score = \frac{TP}{TP + \frac{1}{2}(FP + FN)} \tag{4.2}$$

The results show that the proposed model is able to outperform the other methods in most metrics in both the Datasets. Here, U1 split refers to the inductive test split where one the drugs in each DDI pair is not present in the training set and has not been seen before by the algorithm. On the other hand, the U2 split consists of DDI pairs where none of the two drugs are present in the training set. The U1 and U2 splits create a more challenging task, where the algorithm must understand the latent processes in which the drug pairs interact to make DDI predictions.

For Dataset 1, KITE-DDI outperformed MSEDDI, which is the principal benchmark method in this study, by 6.78% and 2.04% in accuracy in the U2 and U1 split tasks, respectively. It also managed to show better performance in terms of accuracy, F1 score, AUPR and AUC compared to the other algorithms in the U2 split and in accuracy and F1 score for the U1 split. For the AUPR

and AUC values in the U1 split, the MSEDDI algorithm exhibited marginally better performance compared to KITE-DDI. This occurs as the positive and negative threshold hyperparameter values are set automatically for the proposed algorithm resulting in a smaller gap between them.

The results obtained from running experiments on Dataset 2 show similar trends with KITE-DDI outperforming the other methods in the different performance metrices for both the U1 and U2 tasks. The values for most of the metrices are greater in general for Dataset 2 compared to Dataset 1. This happens as the first Dataset is smaller than the second, resulting in a larger training set size, making the former a more challenging test case.

Also, it could be seen that the gap between the performance scores between KITE-DDI and the benchmark models are greater for the U2 task compared to the U1 task. This shows that the proposed model is able to perform better compared to the other algorithms when there is significant difference between the contents of the training and the test set proving that KITE-DDI is better at generalizing in more challenging circumstances. Another pattern that emerges from these results is the better outperformance of KITE-DDI compared to MSEDDI in the first Dataset than Dataset 2 indicating that the proposed model performs better with limited training data compared to the previous algorithms.

Next, the ablation study for the different versions of the proposed algorithm and the contribution of the different modules in the model are shown in Table 4.5. Experiments with six different variants of the proposed model are carried out and the results examined, where architectural modifications are progressively made to reflect the contribution of each of these modification on the overall performance of the model. The different evaluation metrics that are used for comparison in these experiments include accuracy, macro F1 score, weighted F1 score, Mathews correlation coefficient (MCC) score, weighted precision, macro precision, weighted recall and macro recall. The results from the principal benchmark model, MSEDDI are also included in this table for comparison. The

Table 4.5: Ablation study comparing the performance of different developed models with the primary benchmark method, MSEDDI

Models	Dataset split	Accuracy	F1 Score (weighted)	F1 Score (macro)	MCC	Precision (Weighted)	Precision (macro)	Recall (weighted)	Recall (macro)
BERT_base	Evaluation	68.61%	0.6778	0.4092	0.6184	0.6799	0.4690	0.6861	0.3900
	U2	25.28%	0.2338	0.0428	0.0564	0.2429	0.0419	0.2528	0.0595
	U1	46.29%	0.4477	0.2575	0.3365	0.4590	0.3079	0.4629	0.2603
BERT_small	Evaluation	66.01%	0.6588	0.3715	0.5928	0.6637	0.3917	0.6601	0.3778
	U2	23.15%	0.2388	0.0292	0.0701	0.3008	0.0446	0.2315	0.0285
	U1	44.33%	0.4441	0.2220	0.3297	0.4667	0.2299	0.4433	0.2588
BERT_large	Evaluation	71.58%	0.7097	0.4541	0.6560	0.7154	0.5471	0.7157	0.4307
	U2	26.30	0.2478	0.0329	0.0811	0.2771	0.0506	0.2630	0.0310
	U1	47.55	0.4660	0.2566	0.3560	0.4734	0.2925	0.4755	0.2726
TRFM_conv	Evaluation	77.02%	0.7636	0.5447	0.7221	0.7716	0.6515	0.7702	0.5062
	U2	30.09%	0.2650	0.0615	0.0904	0.2888	0.0900	0.3009	0.0684
	U1	51.09%	0.4915	0.2904	0.3935	0.5177	0.3467	0.5110	0.2985
TRFM_conv (linear Encoder)	Evaluation	82.59%	0.8238	0.6455	0.7904	0.8274	0.6852	0.8259	0.6486
	U2	30.83%	0.2823	0.0465	0.1246	0.2832	0.0533	0.3083	0.0490
	U1	50.58%	0.4936	0.3135	0.3938	0.5074	0.3345	0.5058	0.3572
TRFM_pretrained	Evaluation	87.43%	0.8731	0.7044	0.8488	0.8761	**0.7354**	0.8743	**0.6956**
	U2	41.11%	0.3803	**0.1199**	0.2437	0.3919	0.1016	0.4111	**0.1167**
	U1	58.92%	0.5784	0.4409	0.4939	0.5920	0.4741	0.5892	0.4403
MSEDDI	Evaluation	87.47%	0.8728	0.6287	0.8489	0.8737	0.6330	0.8747	0.6151
	U2	44.33%	0.4087	0.0844	0.3009	0.4119	**0.1027**	0.4433	0.0845
	U1	65.11%	0.6368	0.4128	0.5688	0.6501	0.5018	0.6511	0.4049
KITE-DDI	Evaluation	**89.01%**	**0.8883**	**0.6744**	**0.8678**	**0.8896**	0.7134	**0.8900**	0.6562
	U2	**51.30%**	**0.4698**	0.0657	**0.3679**	**0.4737**	0.0860	**0.5130**	0.0614
	U1	**67.21%**	**0.6628**	**0.5118**	**0.5970**	**0.6701**	**0.5580**	**0.6721**	**0.5331**

description of each of the ablation variants are given below:

- **BERT_base:** This variant is based on the basic BERT model which consists of just an encoder with 8 encoder layers each of which consists of 6 attention heads. The dimension of the model is 256 with a feedforward network dimension of 1024. The drug pairs in each datapoint are tokenized and inserted into the encoder via the non-positional and positional encoders. The first element in the input sequence is set as the CLS token and its corresponding output goes into a classification header to predict the DDI labels.

- **BERT_small:** This variant is a miniaturized version of the BERT_base model with the same number of encoders and attention heads but with a feed forward network dimension of 256 instead of 1024.

- **BERT_large:** This model consists of 12 encoder layers stacked one after the other with each consisting of 12 multiheaded attention blocks. The dimension of the model is 768 and the feed forward network has a length of 3072.

- **TRFM_conv:** In this variant, a convolutional module is inserted after the encoder layers to collect and process the information coming from the latent vector of the output sequences. It is made up of convolutional layers, batch normalization, RELU activation and pooling layers with skip connections to aid in the propagation of input data. The output from the conv layer is then fed into a classification header to generate the DDI predicitons.

- **TRFM_conv (linear_Encoder)**: This model is a modification of the previous one with a linear positional encoder used instead of the sinusoidal one which is the conventional transformer positional encoding technique.

- **TRFM_pretrained:** In this model, the transformer encoder architecture is pretrained on an unlabeled dataset of drug SMILEs before finetuning it on the DDI Datasets.

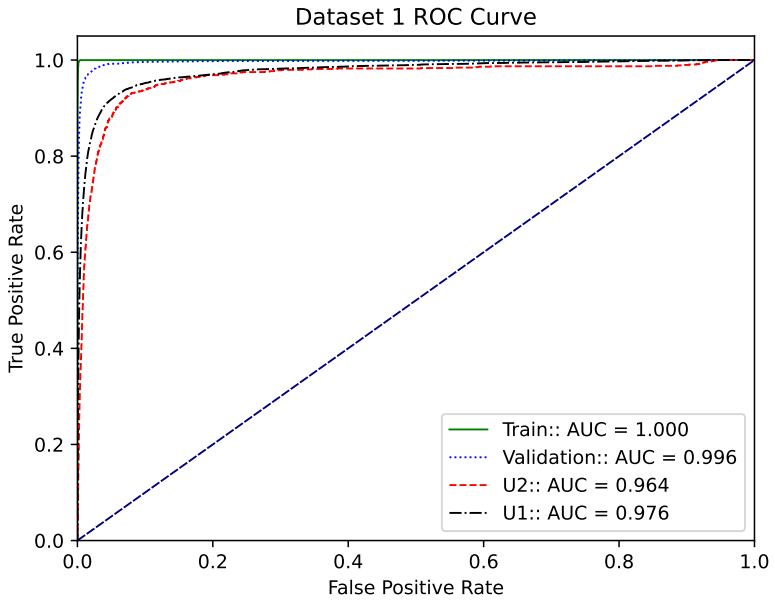

Figure 4.5: Averaged ROC curve of Dataset 1 for the train, validation, U1 and U2 splits.

- **KITE-DDI:** This is the final model where the DRKG drug embeddings are integrated into the model to increase the generalizability of the model on inductive tasks.

In order to compare the contribution of the different variants, their performance using various evaluation matrices are given in Table 4.5 for the validation, U1 and U2 test sets. The results show gradual increase of the performance of the model as successive modules are added into it. The first three models are modified versions of the BERT [78] architecture which consists of a classification output token to make the output prediction. The bert_large_reg model has been the most successful in terms of most metrics among these three.

The next model, Trfm_conv, consists of a convolutional module after the encoder block to process information from the output sequences. This modification demonstrates a notable enhancement in the model's performance, while also being more efficient and demanding fewer computational

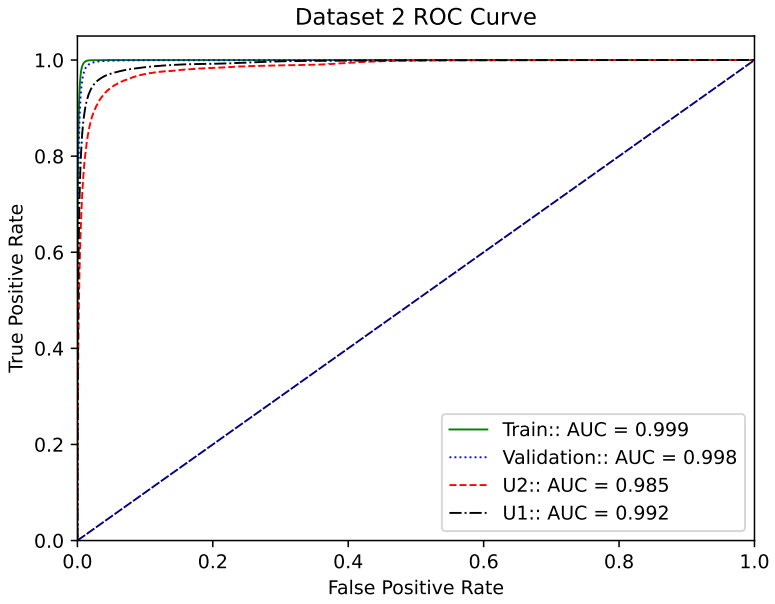

Figure 4.6: Averaged ROC curve of Dataset 2 for the train, validation, U1 and U2 splits.

resources than the Bert_large_reg model. Next, the sinusoidal encoder is replaced by a linear one to streamline the encoding process and finally, the model is pretrained on an unlabeled dataset of drug SMILES to enhance the accuracy and performance of the model even further. Lastly, the drug embeddings extracted from a biomedical knowledge graph are incorporated into the transformer architecture with the help of self-attention modules to develop the final KITE-DDI model.

The Receiver Operating Characteristic (ROC) curve of the implemented model for Datasets 1 and 2 are given in Figures 4.5, 4.6 and 3.7. The ROC curve plots False positive rate vs True positive rate to show how well the algorithm performs at differentiating between the positive and negative samples. In the most ideal situation, the ROC curve should take the shape of an inverted 'L' which maximizes the area under the curve. Figure 4.5 and Figure 4.6 show the ROC curves for the Training, validation, U1 and U2 split results of the Kite-DDI model for Dataset 1 and 2 respectively. A unit gradient line is also drawn to represent the output for a perfectly random system. The results

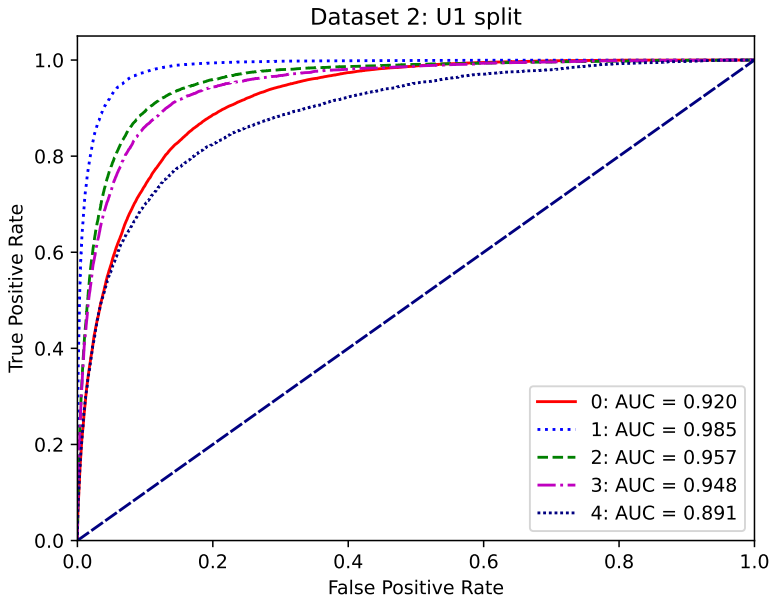

Figure 4.7: Individual ROC curve for the 5 most populous classes in Dataset 2 for the U1 split.

show that the algorithm is doing very well in differentiating between the positive and negative samples in both Dataset 1 and 2. The area under the curve which is represented by the parameter, AUC, is also above 0.95 for both the U1 and U2 inductive splits for Datasets 1 and 2.

Lastly, the Figure 4.7 shows the individual ROC curves for the top 5 most populous classes on Dataset 2. It shows that the proposed algorithm is better at discerning some classes compared to others. This happens as some of the DDI events have longer, unique and more prominent markers to identify them compared to others which are more difficult to differentiate. However, even the more difficult classes have AUC of around 0.9 or more which shows that the algorithm is still able to differentiate between them with adequate accuracy.

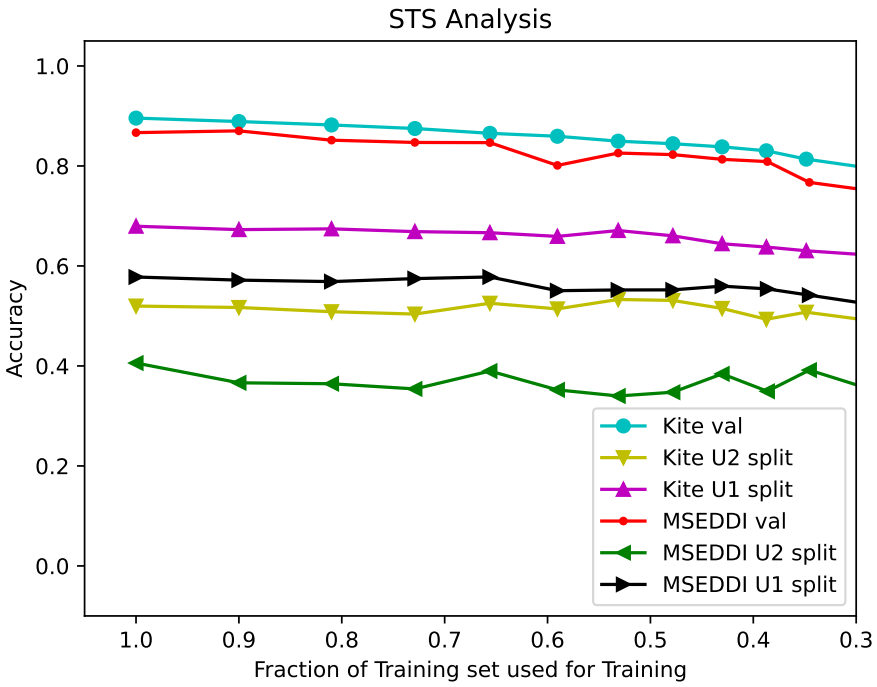

Figure 4.8: STS analysis of KITE-DDI model on Dataset 1 for the validation, U1 and U2 splits including comparison with the MSEDDI model.

STS Analysis

In order to test the performance of the model at low data scenarios, a Shrinking Training Set (STS) analysis is carried out on the proposed algorithm and comparisons have been made with the main benchmark model and the results illustrated in Figure 4.8. In each step of the process, the dataset used to train the model is shrunk by 10% and the new accuracy values are computed. The size of the validation and the U1 and U2 splits are kept the same during the process. Dataset 1 is used as the starting point and all classes which have less than 5 samples in the training set are discarded to make sure that all the datasets have the same number of classes. A stratified split is then done on the training set to split it into two segments of 90% and 10%. The bigger segment is kept and the smaller one discarded. This process is repeated until the training set size falls to around 7% of its initial size.

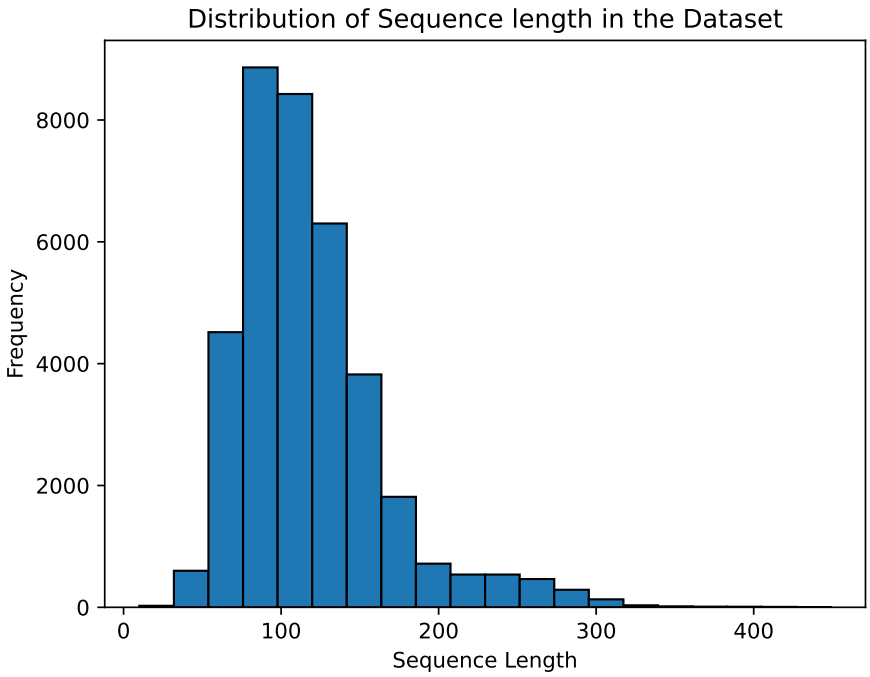

Figure 4.9: Histogram showing the frequency distribution of SMILES with different lengths in Dataset 1.

The Validation, U1 and U2 accuracies are plotted in Figure 4.8 for different training set sizes along with the corresponding values for the MSEDDI model. The results show that the proposed architecture maintains its accuracy consistently even at very small training set sizes and manages to keep its performance advantage over the primary benchmark model, MSEDDI, in all the different training set sizes. Also, there are limited fluctuations in accuracy while varying the training set which shows the stability and robustness of the proposed architecture.

Sequence Length Analysis

This section analyzes the variability of the model performance and its dependencies on the input sequence size. As a sequence model based on the transformer architecture, KITE-DDI is able to

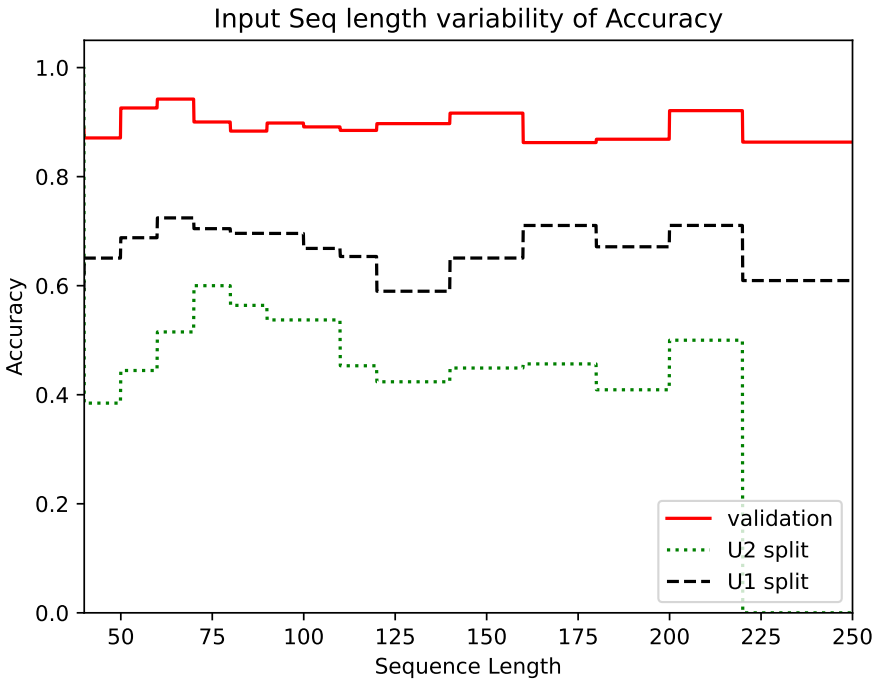

Figure 4.10: Variability of accuracy with input sequence length for the KITE-DDI model on Dataset 1 for the validation, U1 and U2 splits.

take input sequences of any sizes. The distribution of input sequence length in the dataset is given in Figure 4.9. Most of the drug compounds have a SMILE length from 55 to 200 tokens with a peak of 100 bases and a narrow tail at the end. The accuracy of the algorithm at different sequence length is plotted in Figure 4.10. Here, the datapoints are distributed in different bins based on their token lengths and the mean accuracy for each bin is represented by the graph. It shows that the proposed algorithm can hold consistent accuracy with minimal fluctuations for different input lengths and its performance is not dependent on the length of the input sequences. Maintaining similar accuracy even with longer input sequences indicates that the model can understand and learn the long-term patterns and dependencies within the input and can generalize well to a wide range of varying sequence lengths. However, the accuracy of the U2 split does drop to zero after an input length of 225. This happens as the U2 dataset is quite small and it only contains two

samples that are longer than 225 tokens in the dataset leading to the result being irrelevant beyond that point.

Project Conclusion

In this work, we have implemented a transformer based lightweight end-to-end deep learning system, KITE-DDI, for predicting DDI events with higher precision and accuracy compared to other similar state-of-the-art methods. The algorithm integrates the information from Biomedical Knowledge graphs with SMILES representations of drug compounds with the help of a transformer encoder, a convolutional module, and a self-attention block to make the predictions. The model's performance was evaluated on two prominent benchmark datasets using several metrics. The results showed that KITE-DDI performed significantly better compared to other contemporary methods specially on inductive datasets, while at the same time not dependant on external heuristic algorithms for feature extraction. The model is also easy to use, requiring very few hyperparameters to train effectively and shows consistent performance even at very low training set sizes. Furthermore, the system is able to automatically maintain balance between precision and recall without having to tune any additional hyperparameter and its accuracy is independent of the input sequence length making it robust and generalizable.

Although there have been significant work and scientific studies conducted in the field of DDI predictions in recent times, there are still many areas of improvement in the domain. Some of these potential avenues of exploration and investigation includes expanding DDI prediction models to other non small molecule drugs like RNA based medications. Additional work is also required in collecting more samples for rarer DDI events to create a less skewed dataset which could be used to train more efficient DDI prediction models. Hence, there are considerable work that must be done in this field, encompassing a multitude of possible avenues for exploration and investigation.

CHAPTER 5: PROT_GO: A TRANSFORMER BASED FUSION MODEL FOR ACCURATELY PREDICTING GENE ONTOLOGY (GO) TERMS FROM FULL SCALE PROTEIN SEQUENCES

Recent developments in next-generation sequencing technology have led to the creation of extensive, open-source protein databases consisting of hundreds of millions of sequences. To render these sequences applicable in biomedical applications, they must be meticulously annotated by wet lab testing or extracting them from existing literature. Over the last few years, researchers have developed numerous automatic annotation systems, particularly deep learning models based on machine learning and artificial intelligence, to address this issue. In this work[1], we propose a transformer-based fusion model capable of predicting Gene Ontology (GO) terms from full-scale protein sequences, achieving state-of-the-art accuracy compared to other contemporary machine learning annotation systems. The approach performs particularly well on clustered split datasets, which comprise training and testing samples originating from distinct distributions that are structurally diverse. This demonstrates that the model is able to understand both short and long term dependencies within the enzyme's structure and can precisely identify the motifs associated with the various GO terms. Furthermore, the technique is light weight and less computationally expensive compared to the benchmark methods, while at the same time not unaffected by sequence length, rendering it appropriate for diverse applications with varying sequence lengths.

[1]Part of the materials in this chapter is submitted in ACM Transactions on Intelligent Systems and Technology, Special Issue on Transformers

Project Introduction

The swift advancement of sequencing technologies has accelerated the growth of extensive protein sequence databases and made it both economically viable and practical to build large protein repositories both in the private domain and under open-source licenses. The substantial augmentation in the accessibility of protein sequences led to the establishment of protein databases such as Uniprot, with hundreds of millions of data points. Each day, about one hundred thousand proteins are added to the ever-expanding global public protein databases [37]. To maximize the utility of these protein databases, they must be accurately annotated by the scientific community. Domain experts and curators exert considerable effort to manually annotate these protein sequences to address this issue; nonetheless, this procedure is exceedingly slow and resource intensive resulting in successful annotation of only approximately 0.03% of proteins available in public databases [37].

The demand for expedited annotation of extensive protein datasets has led to the emergence of many automated and semi-automatic approaches. Various annotation systems have been developed over the years, employing distinct approaches to label proteins. One approach involved comparing the sequence of the target protein to other previously identified proteins to identify commonalities. The target received identical annotation to the known protein if substantial sequence segments between them exhibited alignment. BLAST [38] was among the most effective tools employing this methodology for annotating protein sequences.

A subsequent prevalent approach for protein annotation employed functional features to characterize target proteins. A protein is ascribed a certain function if it possesses the pattern linked to that function. Subsequent iterations of these models began employing profile Hidden Markov models (HMMs), enhancing their robustness and capacity to identify soft matches [39, 40]. These models calculated the likelihood of a new sequence possessing a specific function if the likelihood exceeded a set threshold. They are rapid and effective, resulting in numerous notable protein

databases, such as Pfam and InterPro, utilizing them to annotate their extensive protein collections [41, 42]. Nonetheless, these models, while rapid and precise, possess certain limits.

Conventionally, they calculated the likelihood of each functional motif separately, resulting in reduced efficiency and accuracy when different functional groups shared a common characteristic. Furthermore, the signatures employed to identify a certain function in a protein sequence frequently necessitate human curator involvement and are not entirely automated, rendering the entire process quite tedious and laborious. These constraints the need for additional research in the area to develop more efficient annotation systems to manage the increasing volume of proteins that are continuously added to databases everyday.

Machine learning algorithms, particularly a variant known as deep learning characterized by models with multiple hidden layers, have garnered significant interest recently due to their success across various applications. Initial deep learning models focused on computer vision and machine translation applications, but they rapidly diversified into additional domains such as medical imaging and bioinformatics.

The functioning of a deep learning model typically comprises of two phases. The initial phase involves training, during which labeled and semi-labeled data is used to ascertain the model's weights, succeeded by the inference phase, where the acquired weights are employed to classify an unknown sample. The later layers of models, which engage in more generalized functions, may be utilized across analogous tasks. This approach is referred to as transfer learning, wherein, rather than training a model from scratch for a new task, the model's weights can be initialized from a comparable pretrained application. Subsequent to the transferring process, the model weights undergo training on the fresh data in a process referred to as fine-tuning. This method yields improved accuracy with reduced training data, requiring less training time and lower computational resources [43].

In light of the success of deep learning models across diverse domains and applications, numerous studies have been conducted to implement them in protein functional prediction and classification tasks [44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55]. Additionally, deep learning models have been applied to protein structure prediction [56, 57, 58, 59, 60] and protein design [61, 62, 63, 64, 65].

A significant advancement in protein annotation commenced when researchers began inputting substantial volumes of unstructured raw data into extensive deep-learning models. This exempted the models from human biases, facilitated the identification of previously unrecognized patterns, and mitigated bottlenecks resulting from the time consuming curation process. Recent advancements in this domain encompass DeepEC [66], an ensemble Convolutional Neural Network (CNN) model that annotates protein sequences with Enzyme Commission (EC) numbers.

However, it can only annotate sequences shorter than one thousand units and necessitates a minimum of ten data points from a single class for optimal functionality. Other notable advancements in the area include CNN-based ensemble models for annotating unaligned protein domain sequences [67], the application of a transformer-based model to enhance both accuracy and computational efficiency [68], the training of various models to comprehend protein sequence structures [69], and the development of a transformer-based model for predicting protein functionality [70]. But these studies do not address whole protein sequences, which are frequently significant for bioinformatics applications. This issue has been addressed in Proteinfer [71], which use both a singular and an ensemble CNN architecture to annotate whole protein sequences with GO terms and EC numbers.

We have previously developed a ProtBert [69] based model named ProtEC [36] to accurately annotate EC numbers with high accuracy. This work further builds upon that to propose a classification based fusion model to predict GO terms from full scale protein sequences that achieves state-of-the-art performance while at the same time addressing some of the major shortcomings of the previous models.

The primary novelties of our study are outlined below:

- Achieving state-of-the-art accuracy on GO term annotation of full-scale protein sequences.
- Employing a singular model rather than an ensemble of models to enhance system efficiency and usability, while at the same time making it more light weight.
- The training time, GPU memory, and other computational resources needed to utilize the model are considerably lower compared to other similar methods.
- The proposed model features fully automated hyperparameter optimization, facilitating training without the need for a development or validation dataset.
- A superior comprehension of the protein sequence structure, which is demonstrated by the minimal accuracy disparity between the random and clustered dataset splits.
- Negligible dependency on the input sequence length, exhibiting robustness and the ability to be deployed in applications with significant input sequence length variability.

The project is structured into four sections. The first section presents an overview of the study accompanied by appropriate literature reviews. The second section delineates the model architecture comprehensively and explains the training and evaluation techniques. The third section presents the experimental results of the proposed model and the performance comparison with other benchmark methods. Finally, the last section finishes the work and offers additional discussions on potential future research directions.

Model

This section first describes the input and output of the model, the dataset being used in this study, and the steps taken to preprocess the dataset. Next, the architecture of the proposed model is described in detail along with the training and evaluations steps and the experimental setup.

GO Terms

GO terms are keywords used to annotate protein sequences which contains various information like the gene that was used to transcribe the protein molecule, the type of organism which primarily uses the protein, the functionality of the protein, the biological processes and pathways connected to it, etc. The annotation of the protein sequence with GO terms is of utmost importance to the biological community as it helps to identify the characteristics and properties of the protein along with its usage and functions within an organism. The GO terms could be divided into three categories based on the type of information they contain:

- **Molecular Function:** Activities at the molecular level executed by gene products. Molecular function delineate processes that transpire at the molecular level, such as "catalysis" or "transportation." It denote activities rather than the entities (molecules or complexes) executing the actions, and do not delineate the location, timing, or context of the action. Molecular functions typically relate to activities executed by individual gene products (such as proteins or RNA), while certain activities are carried out by molecular complexes consisting of numerous gene products. Broad functional terms include catalytic activity and transporter activity, whereas narrower functional words encompass adenylate cyclase activity and Toll-like receptor binding.
- **Cellular Component:** This type of GO term describes the location occupied by a macro-

molecular apparatus relative to cellular compartments and structures. The gene ontology delineates the locations of gene products in two manners: the first method is by cellular anatomical entities, where a gene product performs a molecular function. Cellular anatomical entities encompass cellular components like the plasma membrane and cytoskeleton, along with membrane-bound compartments such as the mitochondrion. The other method is with the help of stable macromolecular complexes to which they belong, for instance, the clathrin complex.

- **Biological Process:** The extensive procedures, or 'biological programming,' executed through various molecular activities. Instances of general biological process terminology include DNA repair and signal transduction. Examples of narrower terminology include pyrimidine nucleobase biosynthesis process and glucose transmembrane transport.

Dataset

The protein sequences and their corresponding GO terms that has been used in this study has been extracted from UniProt [37], which is a large public repository of protein sequences. The UniProt archive consists of two different databases; the first is called Swiss-Prot, which consists of about half a million protein sequences that has been annotated manually by domain experts. The other database is known as TrEMBL and consists of around 250 million unreviewed sequences. The dataset used in this study has been prepared from the sequences taken from the SwissProt database as it contains reliable labels and could be used to train and test our model.

Two different dataset splits are used to train and evaluate our model. The first is the random split, where the training, development and testing sets are created by randomly splitting the whole dataset in a 8:1:1 ratio. The second dataset split is called the clustered split, where a tool called Uniref50 [76] is used to divide the dataset into sections where the sequences in each splits are

Table 5.1: Number of sequences of each aspect in the random dataset splits

Split	Proteins	Biological Processes	Molecular Functions	Cellular components
Train	438,522	346,677	369,909	321,980
Development	55,453	43,734	46,785	40,765
Test	54,289	42,830	45,662	39,929

Table 5.2: Number of sequences of each aspect in the clustered dataset splits

Split	Proteins	Biological Processes	Molecular Functions	Cellular components
Train	182,965	144,292	154,150	134,200
Development	180,309	143,130	152,156	131,778
Test	183,475	144,605	155,593	135,345

different in structure and has the least possible common segments between them. This makes the clustered split dataset much more challenging compared to the former. The number of datapoints in the random and clustered split datasets are given in Table 5.1 and Table 5.2 respectively.

Next, during the preprocessing steps, the sequences in the dataset that does not have any GO term annotations associated with them are filtered out. After that, the list of GO terms present in the dataset are collected and divided based on their aspects, which are biological processes, cellular components and molecular functions. Each dataset is next split into three parts where each contains only GO terms confined to one aspect. Only the top 100 most populous GO terms from each aspect are used for training and evaluation as many of them only appear in the dataset very rarely. The number of protein sequences in each aspect for the training, development and testing sets are also given in Table 5.1 and Table 5.2 for the random and clustered datasets respectively.

After separating out the datasets based on the three aspects, the GO terms are one-hot encoded where, a value of 1 is assigned to the terms that are present for a particular protein sequence and 0 if they are absent. Next, the protein sequences are tokenized and inputted into the algorithm while the one-hot encoded GO terms serves as the truth labels or targets.

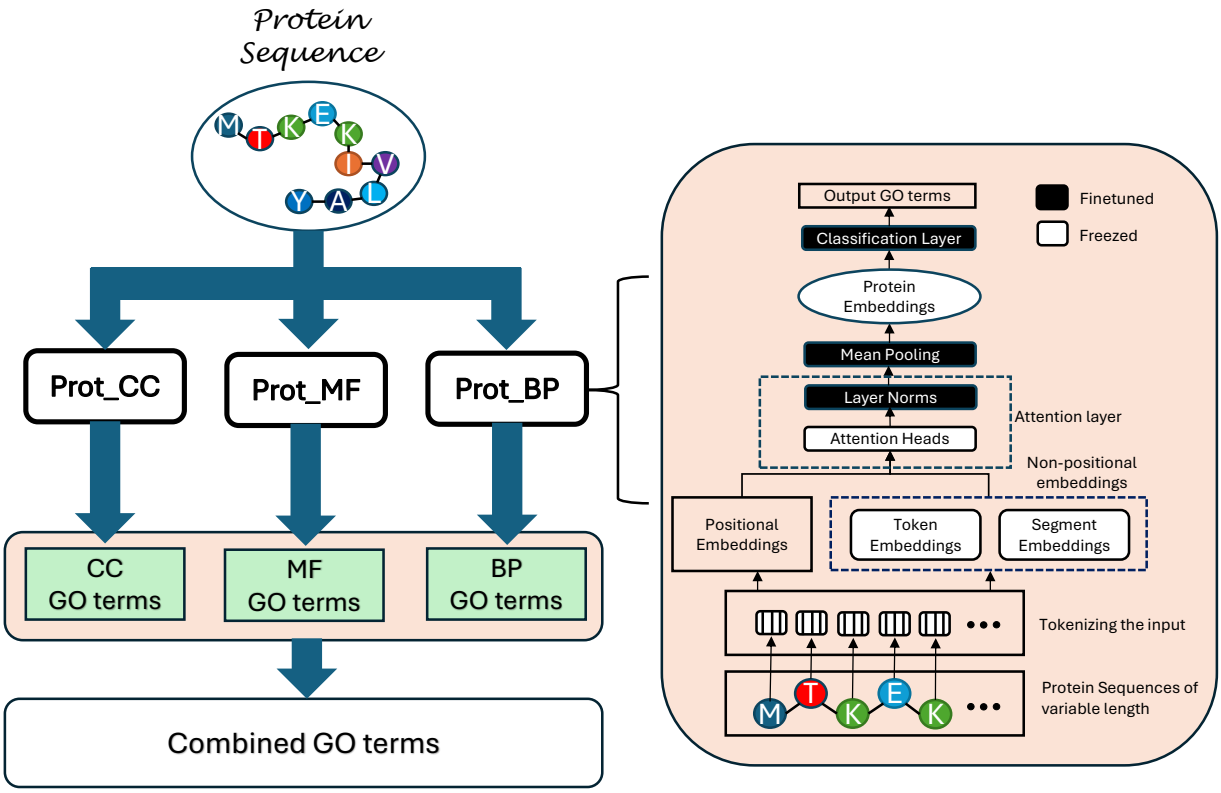

Figure 5.1: Block diagram illustrating the detailed architecture of the proposed model of ProtGo.

Model Architecture

A block diagram illustrating the detailed architecture of the proposed model is given in Figure 5.1. Three transformer based modules are used in the model, where each specializes on one aspect of the GO terms. The protein sequence is first tokenized, where each amino acid in the sequence is

represented by an unique alphabet. The tokenizer takes the input sequence and converts it into a string of vectors which could be now inserted into the transformer architecture.

The protein sequence is copied and parallelly inserted into all three of the transformer blocks; Prot_CC, Prot_MF, and Prot_BP. Each of these modules predict the GO terms related to one of the aspects. Next, the predictions from all three blocks are combined together to create the total output of the model.

The internal architecture of each of the prot_X block is also given in Figure 5.1. After the input protein sequences are tokenized, they are fed into three encoders namely, the positional encoder, the token encoder and the segment encoder. Each of the encoder is responsible for generating one aspect of the input tokens. The token encoder and the segment encoder are together referred to as non-positional encoders.

Internally, each of the encoders contain learnable weights which could be factored with the inputs to generate the output embeddings. The positional enbeddings contains information about the position of each token in the sequence, while the token embeddings describes the type of token in each position. Lastly, the segment embeddings mostly remains dormant in this case as the protein sequences are not divided into multiple segments. After all the three types of embeddings are generated, they are summed together and fed into the transformer encoder.

The encoder module of the transformer is made up of multiple encoder layers stacked on top of each other. The proposed model consists of 30 encoder layers, where each consists of a multihead attention module followed by layer normalization and mean pooling. Each encoder layer also consists of a skip connection or residual network that facilitates the propagation of early layer information to the later layers of the model. A model dimensionality of 1024 is used along with 12 heads for the attention layers. After the transformer encoder extracts the embeddings from the input sequence, they are fed into a classification layer which predicts the GO terms that the input

sequence are associated with. Lastly, the labels from all three submodules are collected and pooled together to form the final model output.

Training and Evaluation

The first step in the training process involves pretraining the Prot_X blocks on the BFD100 dataset which consists of 2,122 million protein sequences of variable lengths [69]. The pretrained variant of ProtBert was obtained from the HuggingFace website [86]. During pretraining, segments of the sequence, determined by the masking probability, were concealed, and the model was instructed to predict the masked tokens. This enables the model to comprehend and assimilate intricate patterns within the data, as well as the interrelations among various amino acids in the protein sequence. The ProtBert module comprised of 30 hidden layers, each containing 1024 nodes, with an intermediate hidden layer size of 4096. The model includes 16 attention heads and contained a total of 420 million parameters. It was pretrained in [69] utilizing 128 nodes and 1024 TPUs for 800,000 steps on sequences limited to a maximum length of 512, followed by an additional 200,000 steps on sequences with a maximum length of 2,000. A masking probability of 15% was implemented in conjunction with a Lamb optimizer. A learning rate of 0.002 and a weight decay rate of 0.01 were chosen.

To adapt the ProtBert module for our purposes, we acquired the pretrained weights and fine-tuned them on enzyme sequences. One major shortcoming of this is that fine-tuning substantial transformer models such as ProtBert demands significant data and computational resources. To address this, we have executed selective fine-tuning, wherein certain layers of the model are adjusted while others retain their pretrained values. This strategy enhances accuracy in large models with limited data availability[87, 88]. The fundamental architecture of the ProtBert model, along with the selectively fine-tuned layers, is illustrated in Figure 5.1. An NLL loss function is used to pretrain the

modules in an unsupervised manner. The layers which were frozen to get the most efficiency in case of the ProtBert modules were referenced from [36].

Next, the pretrained Prot_Bert modules are selectively finetuned on the processed random and clustered datasets. The training set from each type of split is used to train the model, while the testing set has been reserved for the evaluation and comparison with benchmarks methods. The cross entropy loss function has been used during the finetuning process and is given in equation 5.1. Here, N is the total number of samples in the dataset while M is the total number of classes, y represents the true label and $\hat{y}$ stands for the predicted labels. An Adam optimizer has been used to train the model on a total of 10 epochs with an initial learning rate of 5e-4. The HuggingFace [86] trainer has been used to train the model which automatically adjusts and optimizes most of the hyperparameters along with a gradient accumulation step of 32. The model was trained on a single Nvidia RTX2080Ti GPU which required around 3.5GB of memory and took approximately 80 hours to be completely trained.

After the training step, the model was evaluated on the held out testing set for both the random and clustered split datasets. The results obtained in the experiment was compared with the benchmark models and are presented in detail in the next section.

$$Loss = -\frac{1}{N}\sum_{i}^{N}\sum_{j}^{M} y_{ij} log(\hat{y}_{ij}) \tag{5.1}$$

Results

This section presents the evaluation statistics and performance comparison of the proposed ProtGO model with previous state-of-the-art benchmark models. Other analysis results including the Receiver Operating Characteristic (ROC) curve and sequence length analysis of the proposed model

Table 5.3: Evaluation results of the proposed model on the Random Split dataset on various performance metrics compared to the benchmark methods.

Aspect	Model	Accuracy	F1 score	Precision	Recall
Biological Processes	ProtGO	**86.06%**	**0.9251**	**0.9725**	0.8821
	Proteinfer	80.21%	0.8902	0.8447	0.9409
	Proteinfer_EN	83.29%	0.9088	0.8652	**0.9570**
Molecular Function	ProtGO	**94.60%**	**0.9722**	**0.9882**	0.9568
	Proteinfer	88.94%	0.9415	0.9166	0.9677
	Proteinfer_EN	91.67%	0.9565	0.9344	**0.9797**
Cellular Component	ProtGO	**78.30%**	**0.8783**	**0.9469**	0.8189
	Proteinfer	69.36%	0.8191	0.7386	0.9191
	Proteinfer_EN	75.40%	0.8597	0.7928	**0.9390**

have also been outlined and discussed in this section.

Several evaluation metrics have been used to comprehensively compare the results of the models on both the random and clustered dataset splits. Table 5.4 shows the performance of the proposed model on all three GO aspects, namely, Biological Processes, Molecular Function, and Cellular Component on the random split dataset using the overall Accuracy, F1 score, Precision, and Recall evaluation metrics. The results show that the proposed model, ProtGO has been able to perform significantly better compared to the main benchmark methods Proteinfer and ProteinferEN [71]. Here, ProteinferEN refers to the ensembled version of the Proteinfer model, where several base models have been trained and later put together to generate the final output.

ProtGO outperformed ProteinferEN by around 3% in terms of Accuracy in all three GO terms aspects while showing better F1 scores as well. The Precision scores of ProtGO are also significantly better than the other methods. However, in terms of the Recall scores, ProteinferEN shows better results out of the three compared models. This delineates that the ProteinferEN model is better at

Table 5.4: Evaluation results of the proposed model on the Clustered Split dataset on various performance metrics compared to the benchmark methods.

Aspect	Model	Accuracy	F1 score	Precision	Recall
Biological Processes	**ProtGO**	**82.16%**	**0.9021**	**0.9532**	0.8561
	Proteinfer	72.48%	0.8424	0.8940	0.7965
	Proteinfer_EN	75.56%	0.8650	0.8618	**0.8683**
Molecular Function	**ProtGO**	**91.51%**	**0.9556**	**0.9785**	**0.9338**
	Proteinfer	81.51%	0.8992	0.9241	0.8756
	Proteinfer_EN	83.88%	0.9159	0.9012	0.9312
Cellular Component	**ProtGO**	**73.28%**	**0.8458**	**0.9215**	0.7817
	Proteinfer	63.06%	0.7782	0.8014	0.7563
	Proteinfer_EN	66.22%	0.8047	0.7832	**0.8277**

avoiding false negative samples compared to ProtGO and vice versa. But the trade off between the false positive and false negative samples could be controlled using a minimum decision threshold hyperparameter, which has been set to the default value for all the algorithms. Hence, a better indicator of performance is the F1 score, which combines the values of Precision and Recall to produce a single metric. In terms of the F1 score, the ProtGO module has been able to consistently outperformed the benchmark methods in both the datasets.

Table 5.4 shows the performance comparison of the proposed model along with the benchmark methods on the clustered split dataset. The results are similar to the random split, with ProtGO performing better in terms of Accuracy and F1 score in all three GO aspects. However, the gap between the performance between ProtGO and the benchmark methods is more in case of the clustered split compared to the random split dataset. This shows that ProtGO is a more robust model and have a strong understanding of the structure and deep motif patterns of the protein structure. Hence, it performs similarly for the random and clustered test dataset while the benchmark mod-

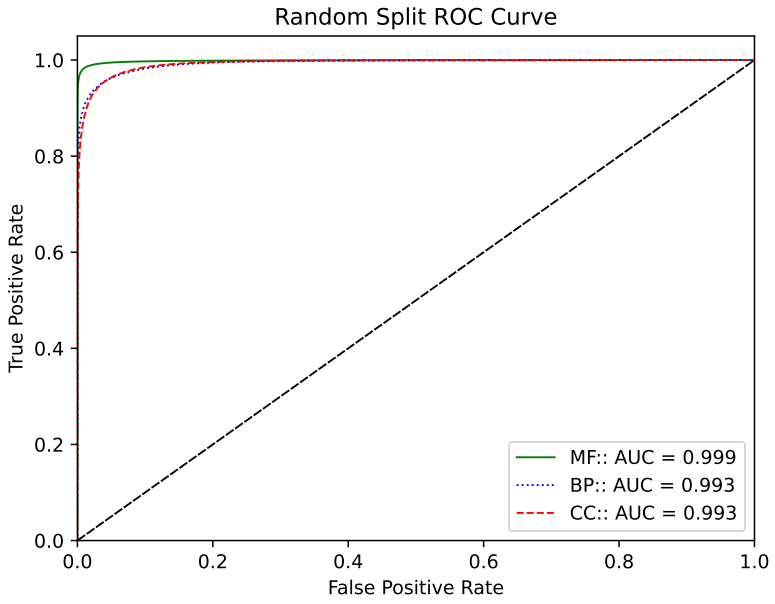

Figure 5.2: Averaged ROC curve of the ProtGO model for the random split dataset.

els show significant difference in Accuracy for the two datasets. There is a gap of around 7% in Accuracy between the proposed model and the benchmark methods for the clustered split dataset which is significantly greater than that seen for the random dataset split. A similar trend could also be seen in the F1 score, Precision and Recall scores for the two splits.

ROC Curves

Next, the ROC curve for the random split dataset is shown in Figure 5.2 for the ProtGO model. The ROC curve shows the ability of a binary classifier model to differentiate between the positive and negative samples. In order to transform the ROC curve for multi-class classifications as in this study, the class wise ROC curves have been generated by considering all other classes as negative samples, followed by micro averaging all the classes to get the overall ROC curve. A straight line

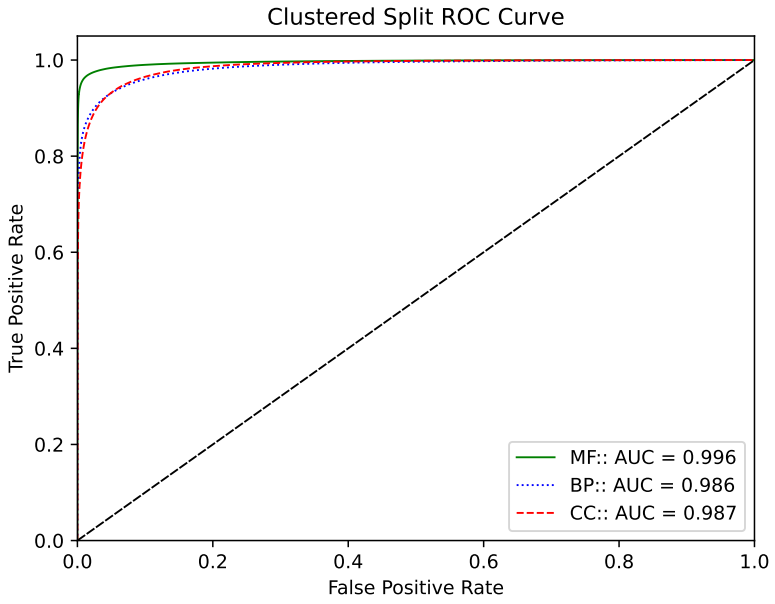

Figure 5.3: Averaged ROC curve of the ProtGO model for the Clustered split dataset.

ROC curves

along the origin with a gradient of 1 represents a random classification and is shown by a black line in the figures which could to be used as a reference. while, an inverted L shaped curve touching the origin and the top two corners would represent a perfect separation between the positive and negative samples. The ROC curve for the clutered split dataset is given in Figure 5.3

The ROC plots for the ProtGO model on both the random and clustered dataset splits show near perfect separation of the negative and positive samples with a large area under the curves. The model for the Molecular Function (MF) GO aspect performs the best in both the datasets with the other two closely following it. The AUC values which stands for "Area under the curve" is also above 99% for the random split dataset while that for the clustered split is above 98%. These results indicate that the proposed model is excellent at differentiating between the various classes with great ease.

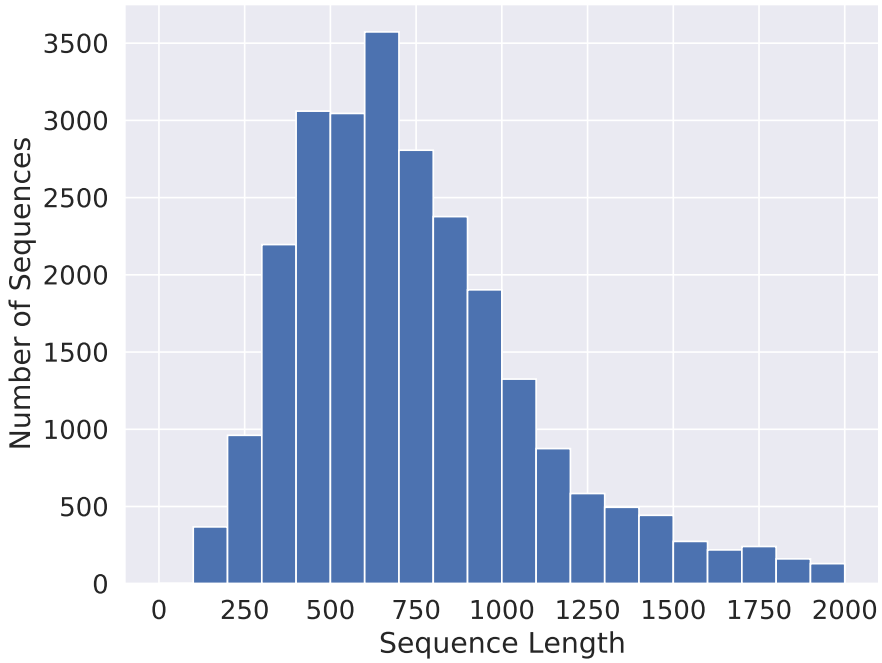

Figure 5.4: Left: Frequency distribution of the input protein sequence lengths in the dataset.

Sequence Length Analysis

Sequence Length Analysis

In the Sequence Length Analysis (SLA) experiment, the variability of the test Accuracy on the length of the input sequence is studied along with the distribution of input sequences in the dataset. Figure 5.4 shows a histogram to represent the abundance of input sequences of different lengths in the dataset, while Figure 5.4 outlines the test Accuracy of the ProtGO model for the various input sequence lengths in the random split dataset for all three GO aspects. The frequency distribution plot peaks at around 750 tokens while most of the sequences are confined between a range of 300 to 1200 amino acids. It could be seen from Figure 5.5 that the Accuracy of the proposed model holds approximately steady till 1000 tokens while experiencing a gradual downward shift after that. This happens as the input tokens are truncated at 1000 tokens in this study for the ease of computation whereas the input sequence length could be easily increased without having to

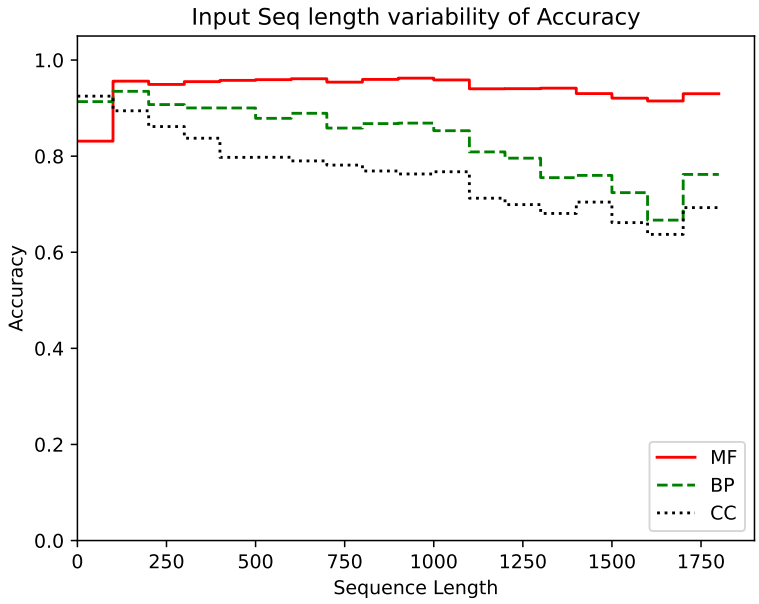

Figure 5.5: Variability of ProtGO Accuracy for the different GO aspects on the input sequence length.

make any algorithmic changes to increase Accuracy for longer sequences. The Accuracy of the Molecular Function GO aspect does not dip even for very long sequences which shows that these GO terms are relatively easier to predict compared to the others. Whereas, the Accuracy of the other two GO aspects fall to around 70% at very long sequence lengths.

Project Conclusion

In this study, a lightweight, easy to use, transformer-based fusion machine learning model have been proposed which could predict GO terms with high Accuracy from full scale protein sequences. The performance of the model have been tested on two datasets named Random and Clustered split for all three GO aspects. The results show state-of-the-art performance on several different evaluation metrics compared to other recent related methods. The ProtGO model have

been able to outperform the benchmark methods used in this study in terms of Accuracy and F1 scores by significant margins in both the random and clustered split datasets. Moreover, the results show that the margin of improvement for the ProtGO model from the benchmark methods in terms of both Accuracy and F1 scores have been significantly greater for the more challenging clustered dataset compared to the random split dataset. This indicates that the proposed architecture is better at deciphering and interpreting deep patterns and structures within the protein sequences which leads to the performance gap being greater for the more challenging dataset.

The ROC curve for the ProtGO model have also been generated for both the datasets and it shows the models ability to successfully differentiate between the positive and negative samples very effectively. This is also demonstrated by the AUC score which is above 99% and 98% for the random and clustered split datasets respectively. Lastly, the sequence length analysis study for the ProtGO model is also conducted where, the model demonstrating limited variability of test Accuracy as the sequence length of the input data is varied. The Accuracy is more stable upto a sequence length of a 1000 amino acids as the input sequences are truncated at that value. This indicates that the Accuracy of the model could be further increased at longer sequence lengths by raising the input sequence truncation threshold hyperparameter.

Despite several scientific studies completed recently on protein annotation utilizing deep learning models, there is a deficiency of research focused on models specifically built for GO term annotation, particularly in scenarios characterized by limited data availability. Additional efforts are necessary to gather supplementary datasets pertaining to GO term annotations to enhance the efficacy of deep learning models and improve the overall Precision of GO term predictions. Furthermore, several protein annotation models that predict distinct functionalities could be integrated into a unified framework, thereby optimizing shared computations in the analysis of protein structure. These are reserved for future endeavors.

CHAPTER 6: CONCLUSION

This chapter presents a comprehensive summary of the dissertation and delineates the concluding observations. It also examines the outcomes of several investigations that demonstrate the usefulness of the implemented procedures, as well as potential future research directions and prospective endeavors.

Machine learning and artificial intelligence have been progressing rapidly in recent times. LLM models like ChatGPT and Gemini are at the forefront of the revolution. The fundamental structure of these LLM models is the transformer architecture, an encoder-decoder framework that utilizes the attention mechanism. The transformer framework originated as a natural language processing framework but rapidly diversified into several applications, including computer vision and audio analysis.

Despite their high accuracy, models utilizing the transformer architecture necessitate substantial data for effective training. This presents a challenge in numerous application domains such as biomedical research, bioinformatics, and drug discovery, where the accumulation of extensive datasets is impractical due to confidentiality and privacy concerns. This work implements several innovative training regimes and architectural improvements to enhance the transformer model's suitability for low-data applications.

The suggested methodologies encompass selective transfer learning systems, knowledge graph-integrated transformer fusion models, multi-modal input systems, hierarchical modular architectures, and distributive models, among others. The effectiveness of the proposed ideas have been investigated with the help of three projects, where the modifications have been implemented and various experiments conducted to compare the results with the conventional transformer models. The summaries and final observations for each project are provided below:

1. **ProtEC:** In this project, we have introduced a transformer-based deep learning system capable of annotating full-scale protein sequences with EC numbers, achieving superior accuracy relative to the previous SOTA algorithm, Proteinfer, which we have used as our primary benchmark. The model uses a hierarchical serialized modular architecture consisting of four transformer ProtBert modules to predict the four EC numbers with high accuracy. Selective transfer learning have been utilized to train the models with limited application data with the help of unsupervised pretraining from unannotated protein sequences.

   The input of the model is full-scale protein sequences sourced from the UniProt protein database. The test the accuracy of the model and compare it with other SOTA models, two different datasets have been used which has been referred to as random and clustered splits. The clustered split dataset is a more challenging test case where the training and test subsets are created by UniRef50 which is an established tool for separating sequences with least possible overlaps. The results show ProtEC performing significantly better than the primary benchmark models specially in the clustered split dataset which shows that the algorithm is able to understand the semantic patterns and deep motifs within the input sequences.

   The key innovations of the work consists of using a novel selective transfer learning training method as well as a modular serial hierarchical architecture. Other innovations include attaining SOTA performance, utilizing a singular model rather than an ensemble of models to enhance system efficiency and user-friendliness, requiring significantly reduced time and computational resources for training in comparison to alternative models, featuring fully automated hyperparameter adjustments, facilitating training without the need for a development or validation dataset, and enhanced comprehension of intricate patterns within protein sequences as demonstrated by the minimal disparity in accuracy between the random and clustered dataset splits along with the model's capability to function exceptionally well with limited data.

2. **KITE_DDI:** This project presents a comprehensive machine learning system that utilizes raw drug SMILES and a biological knowledge graph as inputs to accurately predict DDI events. The model is assessed on two distinct datasets and demonstrates superior performance relative to state-of-the-art models, particularly in the more demanding inductive context. This suggests that the proposed architecture excels in extracting latent chemical features from drug SMILES and comprehending the intricate structures and patterns within biomedical knowledge graphs necessary for achieving highly accurate predictions in an inductive context.

   The model architecture consists of a knowledge graph integrated transformer encoder structure which combines the information from graphs as well as drug SMILES to predict DDI events with high accuracy. Other model highlights include a convolutional network instead of a fully connected layer to extract information from the encoder output and to attain vector dimensionality reduction, along with a self attention block to combine and merge information from multiple channels.

   The key contributions and innovations of this study include achieving exemplary performance and precision relative to other advanced models, particularly in predictions within the inductive dataset split configuration, demonstrating effective generalization, implementing a comprehensive machine learning model devoid of heuristic elements that depend on domain expertise. The model being lightweight, cost-effective in computing, and user-friendly, functioning as an end-to-end pipeline that necessitates minimal hyperparameter adjustment, necessitating just two inputs (KG and SMILES), in contrast to the primary benchmark model, which demands five (KG, SMILES, MPNN, AFP, WEAVE), and demonstrating superior performance under low data conditions relative to the primary benchmark approach.

   The results also show that the model is more stable and robust and generalizes better compared to other related methods which relied on the conventional transformer architecture.

3. **ProtGO:** This is the final project in this dissertation work that has been used to demonstrate the efficacy of the proposed low data application innovations. Here, we have implemented a distributed modular transformer based classification fusion architecture that takes full-scale protein sequences as input and predicts the GO terms associated with that protein. The implemented method is able to predict all three GO aspects, namely Biological processes, Cellular components, and Molecular functions with higher accuracy compared to other state-of-the-art methods. A selective pretraining paradigm similar to that used in the first project is utilized here as well. Three ProtBert modules are pretrained in an unsupervised manner on unannotated protein sequences before they are finetuned on the application dataset.

   The implemented model is tested out on two separate dataset splits called the random and clustered splits. In case of the random split, the training, development and testing subsets are divided in a random manner, whereas, in clusted split dataset, the subsets are separated using the UniRef50 clustering tool that can partition sequences based on minimum possible overlaps within them. The results show that the ProtGO model performs significantly better compared to the primary benchmark methods specially on the more challenging clustered dataset demonstrating the ability of the model to genralize well. The results also indicate the implemented model is excellent at separating between the different classes based on the experimental ROC curves and shows very limited dependancy on the input sequence lengths.

   The key innovations of this study includes attaining cutting-edge precision in GO term annotation of comprehensive protein sequences, utilizing a single model instead of an ensemble to improve system efficiency and usability, while also reducing computational complexity, the training duration, GPU memory, and other computational resources required to employ the model in comparison to alternative methods. The proposed model incorporates fully automated hyperparameter optimization, enabling training without requiring a development or validation dataset making it more user-friendly. The model also demostrates an enhanced un-

derstanding of the protein sequence structure, which is evidenced by the negligible accuracy difference between the random and clustered dataset partitions. Lastly, it displays minimal reliance on input sequence length, demonstrating resilience and suitability for applications with considerable variety in input sequence lengths.

All three application projects implemented in this dissertation study demonstrate superior performance compared to their corresponding conventional transformer based methods which proves the effectiveness of the proposed training regime and architectural modifications and innovations. Although, the proposed ideas were demonstrated on some specific application cases, they could easily be applied in other circumstances with similar input data types and output objectives.

With the mainstream research direction of Machine Learning and Artificial Intelligence studies residing in the exploration of expanding models to increase performance, it is becoming increasingly difficult to implement modern models in application fields which involves low data availability. This makes it of the utmost importance to conduct research directed towards the innovation of novel ways to transform ML architectures for low data applications. This has been the primary objective of this dissertation study and the results show that the proposed methods have been able to successfully achieve this in multiple real world test cases.

LIST OF REFERENCES

[1] J. Gerstmayr, P. Manzl, and M. Pieber, "Multibody models generated from natural language," *Multibody System Dynamics*, pp. 1–23, 01 2024.

[2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf

[3] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," in *Advances in Neural Information Processing Systems*, Z. Ghahramani, M. Welling, C. Cortes, N. Lawrence, and K. Weinberger, Eds., vol. 27. Curran Associates, Inc., 2014. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2014/file/a14ac55a4f27472c5d894ec1c3c743d2-Paper.pdf

[4] X. Qiu, T. Sun, Y. Xu, Y. Shao, N. Dai, and X. Huang, "Pre-trained models for natural language processing: A survey," *Science China technological sciences*, vol. 63, no. 10, pp. 1872–1897, 2020.

[5] N. Parmar, A. Vaswani, J. Uszkoreit, L. Kaiser, N. Shazeer, A. Ku, and D. Tran, "Image transformer," in *International conference on machine learning*. PMLR, 2018, pp. 4055–4064.

[6] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *European conference on computer vision*. Springer, 2020, pp. 213–229.

[7] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," in *International Conference on Learning Representations*, 2021. [Online]. Available: https://openreview.net/forum?id=YicbFdNTTy

[8] L. Dong, S. Xu, and B. Xu, "Speech-transformer: A no-recurrence sequence-to-sequence model for speech recognition," in *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2018, pp. 5884–5888.

[9] A. Gulati, J. Qin, C.-C. Chiu, N. Parmar, Y. Zhang, J. Yu, W. Han, S. Wang, Z. Zhang, Y. Wu, and R. Pang, "Conformer: Convolution-augmented transformer for speech recognition," in *Interspeech 2020*. ISCA: ISCA, Oct. 2020.

[10] X. Chen, Y. Wu, Z. Wang, S. Liu, and J. Li, "Developing real-time streaming transformer transducer for speech recognition on large-scale dataset," in *ICASSP 2021 - 2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2021, pp. 5904–5908.

[11] P. Schwaller, T. Laino, T. Gaudin, P. Bolgar, C. A. Hunter, C. Bekas, and A. A. Lee, "Molecular transformer: A model for uncertainty-calibrated chemical reaction prediction," *ACS Central Science*, vol. 5, no. 9, pp. 1572–1583, 2019, pMID: 31572784. [Online]. Available: https://doi.org/10.1021/acscentsci.9b00576

[12] A. Rives, J. Meier, T. Sercu, S. Goyal, Z. Lin, J. Liu, D. Guo, M. Ott, C. L. Zitnick, J. Ma, and R. Fergus, "Biological structure and function emerge from scaling unsupervised learning to 250 million protein sequences," *Proceedings of the National Academy of Sciences*, vol. 118, no. 15, p. e2016239118, 2021. [Online]. Available: https://www.pnas.org/doi/abs/10.1073/pnas.2016239118

[13] T. Lin, Y. Wang, X. Liu, and X. Qiu, "A survey of transformers," *AI Open*, vol. 3, pp. 111–132, 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2666651022000146

[14] R. Jozefowicz, O. Vinyals, M. Schuster, N. Shazeer, and Y. Wu, "Exploring the limits of language modeling," 2016. [Online]. Available: https://arxiv.org/abs/1602.02410

[15] M.-T. Luong, H. Pham, and C. D. Manning, "Effective approaches to attention-based neural machine translation," 2015. [Online]. Available: https://arxiv.org/abs/1508.04025

[16] Y. Wu, M. Schuster, Z. Chen, Q. V. Le, M. Norouzi, W. Macherey, M. Krikun, Y. Cao, Q. Gao, K. Macherey, J. Klingner, A. Shah, M. Johnson, X. Liu, L. Kaiser, S. Gouws, Y. Kato, T. Kudo, H. Kazawa, K. Stevens, G. Kurian, N. Patil, W. Wang, C. Young, J. R. Smith, J. Riesa, A. Rudnick, O. Vinyals, G. S. Corrado, M. Hughes, and J. Dean, "Google's neural machine translation system: Bridging the gap between human and machine translation," *ArXiv*, vol. abs/1609.08144, 2016. [Online]. Available: https://api.semanticscholar.org/CorpusID:3603249

[17] O. Kuchaiev and B. Ginsburg, "Factorization tricks for lstm networks," 2018. [Online]. Available: https://arxiv.org/abs/1703.10722

[18] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," 2017. [Online]. Available: https://arxiv.org/abs/1701.06538

[19] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," *CoRR*, vol. abs/1409.0473, 2014. [Online]. Available: https://api.semanticscholar.org/CorpusID:11212020

[20] Y. Kim, C. Denton, L. Hoang, and A. M. Rush, "Structured attention networks," in *International Conference on Learning Representations*, 2017. [Online]. Available: https://openreview.net/forum?id=HkE0Nvqlg

[21] A. Parikh, O. Täckström, D. Das, and J. Uszkoreit, "A decomposable attention model for natural language inference," in *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, J. Su, K. Duh, and X. Carreras, Eds. Austin, Texas: Association for Computational Linguistics, Nov. 2016, pp. 2249–2255. [Online]. Available: https://aclanthology.org/D16-1244

[22] L. u. Kaiser and S. Bengio, "Can active memory replace attention?" in *Advances in Neural Information Processing Systems*, D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, and R. Garnett, Eds., vol. 29. Curran Associates, Inc., 2016. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2016/file/fb8feff253bb6c834deb61ec76baa893-Paper.pdf

[23] N. Kalchbrenner, L. Espeholt, K. Simonyan, A. van den Oord, A. Graves, and K. Kavukcuoglu, "Neural machine translation in linear time," 2017. [Online]. Available: https://arxiv.org/abs/1610.10099

[24] J. Gehring, M. Auli, D. Grangier, D. Yarats, and Y. N. Dauphin, "Convolutional sequence to sequence learning," in *Proceedings of the 34th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, D. Precup and Y. W. Teh, Eds., vol. 70. PMLR, 06–11 Aug 2017, pp. 1243–1252. [Online]. Available: https://proceedings.mlr.press/v70/gehring17a.html

[25] F. Informatik, Y. Bengio, P. Frasconi, and J. Schmidhuber, "Gradient flow in recurrent nets: the difficulty of learning long-term dependencies," *A Field Guide to Dynamical Recurrent Neural Networks*, 03 2003.

[26] J. Cheng, L. Dong, and M. Lapata, "Long short-term memory-networks for machine reading," *ArXiv*, vol. abs/1601.06733, 2016. [Online]. Available: https://api.semanticscholar.org/CorpusID:6506243

[27] R. Paulus, C. Xiong, and R. Socher, "A deep reinforced model for abstractive summarization," 2017. [Online]. Available: https://arxiv.org/abs/1705.04304

[28] Z. Lin, M. Feng, C. N. dos Santos, M. Yu, B. Xiang, B. Zhou, and Y. Bengio, "A structured self-attentive sentence embedding," *ArXiv*, vol. abs/1703.03130, 2017. [Online]. Available: https://api.semanticscholar.org/CorpusID:15280949

[29] S. Sukhbaatar, a. szlam, J. Weston, and R. Fergus, "End-to-end memory networks," in *Advances in Neural Information Processing Systems*, C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett, Eds., vol. 28. Curran Associates, Inc., 2015. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2015/file/8fb21ee7a2207526da55a679f0332de2-Paper.pdf

[30] Łukasz Kaiser and I. Sutskever, "Neural gpus learn algorithms," 2016. [Online]. Available: https://arxiv.org/abs/1511.08228

[31] K. Cho, B. van Merrienboer, Çaglar Gülçehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using rnn encoder–decoder for statistical machine translation," in *Conference on Empirical Methods in Natural Language Processing*, 2014. [Online]. Available: https://api.semanticscholar.org/CorpusID:5590763

[32] A. Graves, "Generating sequences with recurrent neural networks," *arXiv preprint arXiv:1308*, vol. 0850, 2013.

[33] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1800–1807, 2016. [Online]. Available: https://api.semanticscholar.org/CorpusID:2375110

[34] P. W. Battaglia, J. B. Hamrick, V. Bapst, A. Sanchez-Gonzalez, V. Zambaldi, M. Malinowski, A. Tacchetti, D. Raposo, A. Santoro, R. Faulkner, C. Gulcehre, F. Song, A. Ballard, J. Gilmer, G. Dahl, A. Vaswani, K. Allen, C. Nash, V. Langston, C. Dyer, N. Heess, D. Wierstra, P. Kohli, M. Botvinick, O. Vinyals, Y. Li, and R. Pascanu, "Relational inductive biases, deep learning, and graph networks," 2018. [Online]. Available: https://arxiv.org/abs/1806.01261

[35] R. Child, S. Gray, A. Radford, and I. Sutskever, "Generating long sequences with sparse transformers," 2019. [Online]. Available: https://arxiv.org/abs/1904.10509

[36] A. Tamir, M. Salem, and J.-S. Yuan, "Protec: A transformer based deep learning system for accurate annotation of enzyme commission numbers," *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, vol. 20, no. 6, pp. 3691–3702, 2023.

[37] T. U. Consortium, "UniProt: a worldwide hub of protein knowledge," *Nucleic Acids Research*, vol. 47, no. D1, pp. D506–D515, 11 2018. [Online]. Available: https://doi.org/10.1093/nar/gky1049

[38] S. F. Altschul, W. Gish, W. Miller, E. W. Myers, and D. J. Lipman, "Basic local alignment search tool," *J. Mol. Biol.*, vol. 215, no. 3, pp. 403–410, Oct. 1990.

[39] A. Krogh, M. Brown, I. S. Mian, K. Sjölander, and D. Haussler, "Hidden markov models in computational biology. applications to protein modeling," *J. Mol. Biol.*, vol. 235, no. 5, pp. 1501–1531, Feb. 1994.

[40] S. R. Eddy, "Profile hidden Markov models." *Bioinformatics*, vol. 14, no. 9, pp. 755–763, 10 1998. [Online]. Available: https://doi.org/10.1093/bioinformatics/14.9.755

[41] M. Blum, H.-Y. Chang, S. Chuguransky, T. Grego, S. Kandasaamy, A. Mitchell, G. Nuka, T. Paysan-Lafosse, M. Qureshi, S. Raj, L. Richardson, G. A. Salazar, L. Williams, P. Bork, A. Bridge, J. Gough, D. H. Haft, I. Letunic, A. Marchler-Bauer, H. Mi, D. A. Natale, M. Necci, C. A. Orengo, A. P. Pandurangan, C. Rivoire, C. J. A. Sigrist, I. Sillitoe, N. Thanki, P. D. Thomas, S. C. E. Tosatto, C. H. Wu, A. Bateman, and R. D. Finn, "The InterPro protein families and domains database: 20 years on," *Nucleic Acids Res.*, vol. 49, no. D1, pp. D344–D354, Jan. 2021.

[42] S. El-Gebali, J. Mistry, A. Bateman, S. R. Eddy, A. Luciani, S. C. Potter, M. Qureshi, L. J. Richardson, G. A. Salazar, A. Smart, E. L. Sonnhammer, L. Hirsh, L. Paladin, D. Piovesan, S. C. Tosatto, and R. D. Finn, "The Pfam protein families database in 2019," *Nucleic Acids Research*, vol. 47, no. D1, pp. D427–D432, 10 2018. [Online]. Available: https://doi.org/10.1093/nar/gky995

[43] K. Weiss, T. M. Khoshgoftaar, and D. Wang, "A survey of transfer learning," *Journal of Big Data*, vol. 3, no. 1, p. 9, May 2016. [Online]. Available: https://doi.org/10.1186/s40537-016-0043-6

[44] M. Kulmanov, M. A. Khan, and R. Hoehndorf, "DeepGO: predicting protein functions from sequence and interactions using a deep ontology-aware classifier," *Bioinformatics*, vol. 34, no. 4, pp. 660–668, 10 2017. [Online]. Available: https://doi.org/10.1093/bioinformatics/btx624

[45] A. Dalkiran, A. S. Rifaioglu, M. J. Martin, R. Cetin-Atalay, V. Atalay, and T. Doğan, "Ecpred: a tool for the prediction of the enzymatic functions of protein sequences based

on the ec nomenclature," *BMC Bioinformatics*, vol. 19, no. 1, p. 334, Sep 2018. [Online]. Available: https://doi.org/10.1186/s12859-018-2368-y

[46] R. Cao, C. Freitas, L. Chan, M. Sun, H. Jiang, and Z. Chen, "ProLanGO: Protein function prediction using neural machine translation based on a recurrent neural network," *Molecules*, vol. 22, no. 10, Oct. 2017.

[47] J. J. Almagro Armenteros, C. K. Sønderby, S. K. Sønderby, H. Nielsen, and O. Winther, "DeepLoc: prediction of protein subcellular localization using deep learning," *Bioinformatics*, vol. 33, no. 21, pp. 3387–3395, 07 2017. [Online]. Available: https://doi.org/10.1093/bioinformatics/btx431

[48] A. S. Schwartz, G. J. Hannum, Z. R. Dwiel, M. E. Smoot, A. R. Grant, J. M. Knight, S. A. Becker, J. R. Eads, M. C. LaFave, H. Eavani, Y. Liu, A. K. Bansal, and T. H. Richardson, "Deep semantic protein representation for annotation, discovery, and engineering," *bioRxiv*, 2018. [Online]. Available: https://www.biorxiv.org/content/early/2018/07/10/365965

[49] A. Sureyya Rifaioglu, T. Doğan, M. Jesus Martin, R. Cetin-Atalay, and V. Atalay, "Deepred: Automated protein function prediction with multi-task feed-forward deep neural networks," *Scientific Reports*, vol. 9, no. 1, p. 7344, May 2019. [Online]. Available: https://doi.org/10.1038/s41598-019-43708-3

[50] Y. Li, S. Wang, R. Umarov, B. Xie, M. Fan, L. Li, and X. Gao, "DEEPre: sequence-based enzyme EC number prediction by deep learning," *Bioinformatics*, vol. 34, no. 5, pp. 760–769, 10 2017. [Online]. Available: https://doi.org/10.1093/bioinformatics/btx680

[51] J. Hou, B. Adhikari, and J. Cheng, "DeepSF: deep convolutional neural network for mapping protein sequences to folds," *Bioinformatics*, vol. 34, no. 8, pp. 1295–1303, 12 2017. [Online]. Available: https://doi.org/10.1093/bioinformatics/btx780

[52] S. A. Memon, K. A. Khan, and H. Naveed, "HECNet: a hierarchical approach to enzyme function classification using a Siamese Triplet Network," *Bioinformatics*, vol. 36, no. 17, pp. 4583–4589, 05 2020. [Online]. Available: https://doi.org/10.1093/bioinformatics/btaa536

[53] K. A. Khan, S. A. Memon, and H. Naveed, "A hierarchical deep learning based approach for multi-functional enzyme classification," *Protein Sci.*, vol. 30, no. 9, pp. 1935–1945, Sep. 2021.

[54] R. Concu and M. N. D. S. Cordeiro, "Alignment-free method to predict enzyme classes and subclasses," *International Journal of Molecular Sciences*, vol. 20, no. 21, 2019. [Online]. Available: https://www.mdpi.com/1422-0067/20/21/5389

[55] Z. Zou, S. Tian, X. Gao, and Y. Li, "mldeepre: Multi-functional enzyme function prediction with hierarchical multi-label deep learning," *Frontiers in Genetics*, vol. 9, 2019. [Online]. Available: https://www.frontiersin.org/articles/10.3389/fgene.2018.00714

[56] M. AlQuraishi, "End-to-end differentiable learning of protein structure," *Cell Systems*, vol. 8, no. 4, pp. 292–301.e3, 2019. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2405471219300766

[57] A. W. Senior, R. Evans, J. Jumper, J. Kirkpatrick, L. Sifre, T. Green, C. Qin, A. Žídek, A. W. R. Nelson, A. Bridgland, H. Penedones, S. Petersen, K. Simonyan, S. Crossan, P. Kohli, D. T. Jones, D. Silver, K. Kavukcuoglu, and D. Hassabis, "Improved protein structure prediction using potentials from deep learning," *Nature*, vol. 577, no. 7792, pp. 706–710, Jan 2020. [Online]. Available: https://doi.org/10.1038/s41586-019-1923-7

[58] R. M. Rao, J. Liu, R. Verkuil, J. Meier, J. Canny, P. Abbeel, T. Sercu, and A. Rives, "Msa transformer," in *Proceedings of the 38th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, M. Meila and

T. Zhang, Eds., vol. 139. PMLR, 18–24 Jul 2021, pp. 8844–8856. [Online]. Available: https://proceedings.mlr.press/v139/rao21a.html

[59] Y. Du, J. Meier, J. Ma, R. Fergus, and A. Rives, "Energy-based models for atomic-resolution protein conformations," *CoRR*, vol. abs/2004.13167, 2020. [Online]. Available: https://arxiv.org/abs/2004.13167

[60] J. Yang, I. Anishchenko, H. Park, Z. Peng, S. Ovchinnikov, and D. Baker, "Improved protein structure prediction using predicted interresidue orientations," *Proceedings of the National Academy of Sciences*, vol. 117, no. 3, pp. 1496–1503, 2020. [Online]. Available: https://www.pnas.org/doi/abs/10.1073/pnas.1914677117

[61] S. Biswas, G. Khimulya, E. C. Alley, K. M. Esvelt, and G. M. Church, "Low-n protein engineering with data-efficient deep learning," *Nature Methods*, vol. 18, no. 4, pp. 389–396, Apr 2021. [Online]. Available: https://doi.org/10.1038/s41592-021-01100-y

[62] A. Madani, B. McCann, N. Naik, N. S. Keskar, N. Anand, R. R. Eguchi, P.-S. Huang, and R. Socher, "Progen: Language modeling for protein generation," *bioRxiv*, 2020. [Online]. Available: https://www.biorxiv.org/content/early/2020/03/13/2020.03.07.982272

[63] I. Anishchenko, S. J. Pellock, T. M. Chidyausiku, T. A. Ramelot, S. Ovchinnikov, J. Hao, K. Bafna, C. Norn, A. Kang, A. K. Bera, F. DiMaio, L. Carter, C. M. Chow, G. T. Montelione, and D. Baker, "De novo protein design by deep network hallucination," *Nature*, vol. 600, no. 7889, pp. 547–552, Dec 2021. [Online]. Available: https://doi.org/10.1038/s41586-021-04184-w

[64] K. K. Yang, Z. Wu, and F. H. Arnold, "Machine-learning-guided directed evolution for protein engineering," *Nature Methods*, vol. 16, no. 8, pp. 687–694, Aug 2019. [Online]. Available: https://doi.org/10.1038/s41592-019-0496-6

[65] S. Mazurenko, Z. Prokop, and J. Damborsky, "Machine learning in enzyme engineering," *ACS Catalysis*, vol. 10, no. 2, pp. 1210–1223, Jan 2020. [Online]. Available: https://doi.org/10.1021/acscatal.9b04321

[66] J. Y. Ryu, H. U. Kim, and S. Y. Lee, "Deep learning enables high-quality and high-throughput prediction of enzyme commission numbers," *Proceedings of the National Academy of Sciences*, vol. 116, no. 28, pp. 13 996–14 001, 2019. [Online]. Available: https://www.pnas.org/doi/abs/10.1073/pnas.1821905116

[67] M. L. Bileschi, D. Belanger, D. H. Bryant, T. Sanderson, B. Carter, D. Sculley, A. Bateman, M. A. DePristo, and L. J. Colwell, "Using deep learning to annotate the protein universe," *Nature Biotechnology*, vol. 40, no. 6, pp. 932–937, Jun 2022. [Online]. Available: https://doi.org/10.1038/s41587-021-01179-w

[68] D. Dohan, A. Gane, M. L. Bileschi, D. Belanger, and L. Colwell, "Improving protein function annotation via unsupervised pre-training: Robustness, efficiency, and insights," in *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery Data Mining*, ser. KDD '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 2782–2791. [Online]. Available: https://doi.org/10.1145/3447548.3467163

[69] A. Elnaggar, M. Heinzinger, C. Dallago, G. Rehawi, Y. Wang, L. Jones, T. Gibbs, T. Feher, C. Angerer, M. Steinegger, D. Bhowmik, and B. Rost, "ProtTrans: Toward understanding the language of life through self-supervised learning," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 10, pp. 7112–7127, Oct. 2022.

[70] N. Brandes, D. Ofer, Y. Peleg, N. Rappoport, and M. Linial, "ProteinBERT: a universal deep-learning model of protein sequence and function," *Bioinformatics*, vol. 38, no. 8, pp. 2102–2110, 02 2022. [Online]. Available: https://doi.org/10.1093/bioinformatics/btac020

[71] T. Sanderson, M. L. Bileschi, D. Belanger, and L. J. Colwell, "Proteinfer, deep neural networks for protein functional inference," *eLife*, vol. 12, p. e80942, feb 2023. [Online]. Available: https://doi.org/10.7554/eLife.80942

[72] Enzyme Commission Number. [Online]. Available: https://www.abbexa.com/enzyme-commission-number

[73] T. U. Consortium, "UniProt: the universal protein knowledgebase in 2021," *Nucleic Acids Research*, vol. 49, no. D1, pp. D480–D489, 11 2020. [Online]. Available: https://doi.org/10.1093/nar/gkaa1100

[74] A. MacDougall, V. Volynkin, R. Saidi, D. Poggioli, H. Zellner, E. Hatton-Ellis, V. Joshi, C. O'Donovan, S. Orchard, A. H. Auchincloss, D. Baratin, J. Bolleman, E. Coudert, E. de Castro, C. Hulo, P. Masson, I. Pedruzzi, C. Rivoire, C. Arighi, Q. Wang, C. Chen, H. Huang, J. Garavelli, C. R. Vinayaka, L.-S. Yeh, D. A. Natale, K. Laiho, M.-J. Martin, A. Renaux, K. Pichler, and T. U. Consortium, "UniRule: a unified rule resource for automatic annotation in the UniProt Knowledgebase," *Bioinformatics*, vol. 36, no. 17, pp. 4643–4648, 05 2020. [Online]. Available: https://doi.org/10.1093/bioinformatics/btaa485

[75] D. H. Haft, J. D. Selengut, and O. White, "The TIGRFAMs database of protein families," *Nucleic Acids Res.*, vol. 31, no. 1, pp. 371–373, Jan. 2003.

[76] B. E. Suzek, Y. Wang, H. Huang, P. B. McGarvey, C. H. Wu, and UniProt Consortium, "UniRef clusters: a comprehensive and scalable alternative for improving sequence similarity searches," *Bioinformatics*, vol. 31, no. 6, pp. 926–932, Mar. 2015.

[77] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soricut, "Albert: A lite bert for self-supervised learning of language representations," in *International Conference on Learning Representations*, 2020. [Online]. Available: https://openreview.net/forum?id=H1eA7AEtvS

[78] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*, J. Burstein, C. Doran, and T. Solorio, Eds. Association for Computational Linguistics, 2019, pp. 4171–4186. [Online]. Available: https://doi.org/10.18653/v1/n19-1423

[79] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "Distilbert, a distilled version of BERT: smaller, faster, cheaper and lighter," *CoRR*, vol. abs/1910.01108, 2019. [Online]. Available: http://arxiv.org/abs/1910.01108

[80] K. Clark, M.-T. Luong, Q. V. Le, and C. D. Manning, "Electra: Pre-training text encoders as discriminators rather than generators," *arXiv preprint arXiv:2003.10555*, 2020.

[81] N. S. Keskar, B. McCann, L. R. Varshney, C. Xiong, and R. Socher, "Ctrl: A conditional transformer language model for controllable generation," *arXiv preprint arXiv:1909.05858*, 2019.

[82] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever *et al.*, "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.

[83] Z. Dai, Z. Yang, Y. Yang, J. G. Carbonell, Q. V. Le, and R. Salakhutdinov, "Transformer-xl: Attentive language models beyond a fixed-length context," *CoRR*, vol. abs/1901.02860, 2019. [Online]. Available: http://arxiv.org/abs/1901.02860

[84] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, "Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," *arXiv preprint arXiv:1910.13461*, 2019.

[85] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, P. J. Liu *et al.*, "Exploring the limits of transfer learning with a unified text-to-text transformer." *J. Mach. Learn. Res.*, vol. 21, no. 140, pp. 1–67, 2020.

[86] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz *et al.*, "Huggingface's transformers: State-of-the-art natural language processing," *arXiv preprint arXiv:1910.03771*, 2019.

[87] K. Lu, A. Grover, P. Abbeel, and I. Mordatch, "Pretrained transformers as universal computation engines," *arXiv preprint arXiv:2103.05247*, 2021.

[88] M. Salem, A. Keshavarzi Arshadi, and J. S. Yuan, "Ampdeep: hemolytic activity prediction of antimicrobial peptides using transfer learning," *BMC Bioinformatics*, vol. 23, no. 1, p. 389, Sep 2022. [Online]. Available: https://doi.org/10.1186/s12859-022-04952-z

[89] W. Li and A. Godzik, "Cd-hit: a fast program for clustering and comparing large sets of protein or nucleotide sequences," *Bioinformatics*, vol. 22, no. 13, pp. 1658–1659, 05 2006. [Online]. Available: https://doi.org/10.1093/bioinformatics/btl158

[90] L. Fu, B. Niu, Z. Zhu, S. Wu, and W. Li, "CD-HIT: accelerated for clustering the next-generation sequencing data," *Bioinformatics*, vol. 28, no. 23, pp. 3150–3152, Dec. 2012.

[91] D. M. Qato, J. Wilder, L. P. Schumm, V. Gillet, and G. C. Alexander, "Changes in Prescription and Over-the-Counter Medication and Dietary Supplement Use Among Older Adults in the United States, 2005 vs 2011," *JAMA Internal Medicine*, vol. 176, no. 4, pp. 473–482, 04 2016. [Online]. Available: https://doi.org/10.1001/jamainternmed.2015.8581

[92] P. Zhang, H.-Y. Wu, C.-W. Chiang, L. Wang, S. Binkheder, X. Wang, D. Zeng, S. K. Quinney, and L. Li, "Translational biomedical informatics and pharmacometrics approaches in the drug interactions research," *CPT: Pharmacometrics & Systems*

Pharmacology, vol. 7, no. 2, pp. 90–102, 2018. [Online]. Available: https://ascpt.onlinelibrary.wiley.com/doi/abs/10.1002/psp4.12267

[93] S. Liu, K. Chen, Q. Chen, and B. Tang, "Dependency-based convolutional neural network for drug-drug interaction extraction," in *2016 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)*. IEEE, Dec. 2016.

[94] Y. Shen, K. Yuan, Y. Li, B. Tang, M. Yang, N. Du, and K. Lei, "Drug2Vec: Knowledge-aware feature-driven method for drug representation learning," in *2018 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)*. IEEE, Dec. 2018.

[95] Y. Zhang, W. Zheng, H. Lin, J. Wang, Z. Yang, and M. Dumontier, "Drug–drug interaction extraction via hierarchical RNNs on sequence and shortest dependency paths," *Bioinformatics*, vol. 34, no. 5, pp. 828–835, 10 2017. [Online]. Available: https://doi.org/10.1093/bioinformatics/btx659

[96] X. Sun, L. Ma, X. Du, J. Feng, and K. Dong, "Deep convolution neural networks for drug-drug interaction extraction," in *2018 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)*. IEEE, Dec. 2018.

[97] X. Sun, K. Dong, L. Ma, R. Sutcliffe, F. He, S. Chen, and J. Feng, "Drug-drug interaction extraction via recurrent hybrid convolutional neural networks with an improved focal loss," *Entropy*, vol. 21, no. 1, 2019. [Online]. Available: https://www.mdpi.com/1099-4300/21/1/37

[98] R. Kavuluru, A. Rios, and T. Tran, "Extracting drug-drug interactions with word and character-level recurrent neural networks," in *2017 IEEE International Conference on Healthcare Informatics (ICHI)*. IEEE, Aug. 2017.

[99] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T.-S. Chua, "Neural collaborative filtering," in *Proceedings of the 26th International Conference on World Wide Web*. Republic and Canton of Geneva, Switzerland: International World Wide Web Conferences Steering Committee, Apr. 2017.

[100] L. Yu, M. Cheng, W. Qiu, X. Xiao, and W. Lin, "idse-he: Hybrid embedding graph neural network for drug side effects prediction," *Journal of Biomedical Informatics*, vol. 131, p. 104098, 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1532046422001149

[101] J.-Y. Shi, K. Gao, X.-Q. Shang, and S.-M. Yiu, "LCM-DS: A novel approach of predicting drug-drug interactions for new drugs via Dempster-Shafer theory of evidence," in *2016 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)*. IEEE, Dec. 2016.

[102] Y. Deng, X. Xu, Y. Qiu, J. Xia, W. Zhang, and S. Liu, "A multimodal deep learning framework for predicting drug–drug interaction events," *Bioinformatics*, vol. 36, no. 15, pp. 4316–4322, 05 2020. [Online]. Available: https://doi.org/10.1093/bioinformatics/btaa501

[103] J. Zhu, Y. Liu, Y. Zhang, and D. Li, "Attribute supervised probabilistic dependent matrix tri-factorization model for the prediction of adverse drug-drug interaction," *IEEE Journal of Biomedical and Health Informatics*, vol. 25, no. 7, pp. 2820–2832, 2021.

[104] H. Yu, K.-T. Mao, J.-Y. Shi, H. Huang, Z. Chen, K. Dong, and S.-M. Yiu, "Predicting and understanding comprehensive drug-drug interactions via semi-nonnegative matrix factorization," *BMC Syst. Biol.*, vol. 12, no. S1, Apr. 2018.

[105] W. Zhang, Y. Chen, D. Li, and X. Yue, "Manifold regularized matrix factorization for drug-drug interaction prediction," *Journal of Biomedical Informatics*, vol. 88,

pp. 90–97, 2018. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1532046418302144

[106] S. Deepika and T. Geetha, "A meta-learning framework using representation learning to predict drug-drug interaction," *Journal of Biomedical Informatics*, vol. 84, pp. 136–147, 2018. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1532046418301217

[107] W. Zhang, K. Jing, F. Huang, Y. Chen, B. Li, J. Li, and J. Gong, "Sflln: A sparse feature learning ensemble method with linear neighborhood regularization for predicting drug–drug interactions," *Information Sciences*, vol. 497, pp. 189–201, 2019. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0020025519304116

[108] J. Zhu, Y. Liu, Y. Zhang, Z. Chen, and X. Wu, "Multi-attribute discriminative representation learning for prediction of adverse drug-drug interaction," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 12, pp. 10 129–10 144, 2022.

[109] K. Schwarz, A. Allam, N. A. Perez Gonzalez, and M. Krauthammer, "AttentionDDI: Siamese attention-based deep learning method for drug-drug interaction predictions," *BMC Bioinformatics*, vol. 22, no. 1, p. 412, Aug. 2021.

[110] "RDKit: Open-source cheminformatics," http://www.rdkit.org, [Online; accessed 11-April-2013].

[111] W. Hu*, B. Liu*, J. Gomes, M. Zitnik, P. Liang, V. Pande, and J. Leskovec, "Strategies for pre-training graph neural networks," in *International Conference on Learning Representations*, 2020. [Online]. Available: https://openreview.net/forum?id=HJlWWJSFDH

[112] Y. Chen, T. Ma, X. Yang, J. Wang, B. Song, and X. Zeng, "MUFFIN: multi-scale feature fusion for drug–drug interaction prediction," *Bioinformatics*, vol. 37, no. 17, pp. 2651–2658, 03 2021. [Online]. Available: https://doi.org/10.1093/bioinformatics/btab169

[113] A. K. Nyamabo, H. Yu, Z. Liu, and J.-Y. Shi, "Drug–drug interaction prediction with learnable size-adaptive molecular substructures," *Briefings in Bioinformatics*, vol. 23, no. 1, p. bbab441, 10 2021. [Online]. Available: https://doi.org/10.1093/bib/bbab441

[114] P. Li, J. Wang, Y. Qiao, H. Chen, Y. Yu, X. Yao, P. Gao, G. Xie, and S. Song, "An effective self-supervised framework for learning expressive molecular global representations to drug discovery," *Briefings in Bioinformatics*, vol. 22, no. 6, p. bbab109, 05 2021. [Online]. Available: https://doi.org/10.1093/bib/bbab109

[115] S. Qian, S. Liang, and H. Yu, "Leveraging genetic interactions for adverse drug-drug interaction prediction," *PLOS Computational Biology*, vol. 15, no. 5, pp. 1–17, 05 2019. [Online]. Available: https://doi.org/10.1371/journal.pcbi.1007068

[116] C. Kang, H. Zhang, Z. Liu, S. Huang, and Y. Yin, "LR-GNN: a graph neural network based on link representation for predicting molecular associations," *Briefings in Bioinformatics*, vol. 23, no. 1, p. bbab513, 12 2021. [Online]. Available: https://doi.org/10.1093/bib/bbab513

[117] F. Wang, X. Lei, B. Liao, and F.-X. Wu, "Predicting drug–drug interactions by graph convolutional network with multi-kernel," *Briefings in Bioinformatics*, vol. 23, no. 1, p. bbab511, 12 2021. [Online]. Available: https://doi.org/10.1093/bib/bbab511

[118] A. Bordes, N. Usunier, A. Garcia-Duran, J. Weston, and O. Yakhnenko, "Translating embeddings for modeling multi-relational data," in *Advances in Neural Information Processing Systems*, C. Burges, L. Bottou, M. Welling, Z. Ghahramani, and K. Weinberger, Eds., vol. 26. Curran Associates, Inc., 2013.

[119] T. Trouillon, J. Welbl, S. Riedel, E. Gaussier, and G. Bouchard, "Complex embeddings for simple link prediction," in *Proceedings of The 33rd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, M. F. Balcan and K. Q. Weinberger, Eds., vol. 48. New York, New York, USA: PMLR, 20–22 Jun 2016, pp. 2071–2080. [Online]. Available: https://proceedings.mlr.press/v48/trouillon16.html

[120] X. Yue, Z. Wang, J. Huang, S. Parthasarathy, S. Moosavinasab, Y. Huang, S. M. Lin, W. Zhang, P. Zhang, and H. Sun, "Graph embedding on biomedical networks: methods, applications and evaluations," *Bioinformatics*, vol. 36, no. 4, pp. 1241–1251, 10 2019. [Online]. Available: https://doi.org/10.1093/bioinformatics/btz718

[121] Z. Yu, F. Huang, X. Zhao, W. Xiao, and W. Zhang, "Predicting drug–disease associations through layer attention graph convolutional network," *Briefings in Bioinformatics*, vol. 22, no. 4, p. bbaa243, 10 2020. [Online]. Available: https://doi.org/10.1093/bib/bbaa243

[122] J. Y. Ryu, H. U. Kim, and S. Y. Lee, "Deep learning improves prediction of drug–drug and drug–food interactions," *Proceedings of the National Academy of Sciences*, vol. 115, no. 18, pp. E4304–E4311, 2018. [Online]. Available: https://www.pnas.org/doi/abs/10.1073/pnas.1803294115

[123] S. Lin, Y. Wang, L. Zhang, Y. Chu, Y. Liu, Y. Fang, M. Jiang, Q. Wang, B. Zhao, Y. Xiong, and D.-Q. Wei, "MDF-SA-DDI: predicting drug–drug interaction events based on multi-source drug fusion, multi-source feature fusion and transformer self-attention mechanism," *Briefings in Bioinformatics*, vol. 23, no. 1, p. bbab421, 10 2021. [Online]. Available: https://doi.org/10.1093/bib/bbab421

[124] G. Lee, C. Park, and J. Ahn, "Novel deep learning model for more accurate prediction of drug-drug interaction effects," *BMC Bioinformatics*, vol. 20, no. 1, p. 415, Aug. 2019.

[125] A. K. Nyamabo, H. Yu, and J.-Y. Shi, "SSI–DDI: substructure–substructure interactions for drug–drug interaction prediction," *Briefings in Bioinformatics*, vol. 22, no. 6, p. bbab133, 05 2021. [Online]. Available: https://doi.org/10.1093/bib/bbab133

[126] L. Yu, Z. Xu, M. Cheng, W. Lin, W. Qiu, and X. Xiao, "Mseddi: Multi-scale embedding for predicting drug—drug interaction events," *International Journal of Molecular Sciences*, vol. 24, no. 5, 2023. [Online]. Available: https://www.mdpi.com/1422-0067/24/5/4500

[127] C. Knox, M. Wilson, C. M. Klinger, M. Franklin, E. Oler, A. Wilson, A. Pon, J. Cox, N. E. L. Chin, S. A. Strawbridge, M. Garcia-Patino, R. Kruger, A. Sivakumaran, S. Sanford, R. Doshi, N. Khetarpal, O. Fatokun, D. Doucet, A. Zubkowski, D. Y. Rayat, H. Jackson, K. Harford, A. Anjum, M. Zakir, F. Wang, S. Tian, B. Lee, J. Liigand, H. Peters, R. Q. R. Wang, T. Nguyen, D. So, M. Sharp, R. da Silva, C. Gabriel, J. Scantlebury, M. Jasinski, D. Ackerman, T. Jewison, T. Sajed, V. Gautam, and D. S. Wishart, "DrugBank 6.0: The DrugBank knowledgebase for 2024," *Nucleic Acids Res.*, vol. 52, no. D1, pp. D1265–D1275, Jan. 2024.

[128] U. E. P. Agency, "Sustainable futures / p2 framework manual 2012 epa-748-b12-001 appendix f. smiles notation tutorial," https://www.epa.gov/sites/default/files/2015-05/documents/appendf.pdf, 2012, [Accessed 25-08-2024].

[129] B. Zdrazil, E. Felix, F. Hunter, E. J. Manners, J. Blackshaw, S. Corbett, M. de Veij, H. Ioannidis, D. M. Lopez, J. F. Mosquera, M. P. Magarinos, N. Bosc, R. Arcila, T. Kizilören, A. Gaulton, A. P. Bento, M. F. Adasme, P. Monecke, G. A. Landrum, and A. R. Leach, "The ChEMBL database in 2023: a drug discovery platform spanning multiple bioactivity data types and time periods," *Nucleic Acids Res.*, vol. 52, no. D1, pp. D1180–D1192, Jan. 2024.

[130] M. Ali, M. Berrendorf, C. T. Hoyt, L. Vermue, S. Sharifzadeh, V. Tresp, and J. Lehmann, "PyKEEN 1.0: A Python Library for Training and Evaluating Knowledge Graph

Embeddings," *Journal of Machine Learning Research*, vol. 22, no. 82, pp. 1–6, 2021. [Online]. Available: http://jmlr.org/papers/v22/20-825.html